

Final Projects

EE330 – Integrated Electronics

Iowa State University

Truong Hoai An Luu & Nhut Minh Quan Nguyen

Digital Potentiometer / Programmable Amplifier

Presentation Date: 05/07/2026 (TA: Sam Burns)

I) Introduction:

For the final project, we are required to build a multifunctional circuit that is controlled by a multiplexer. Based on the input of the control lines of the mux, our circuit can work as a programmable non-inverting (00)/inverting amplifier (01), a programmable potentiometer (10), and a digital-to-analog converter (11).

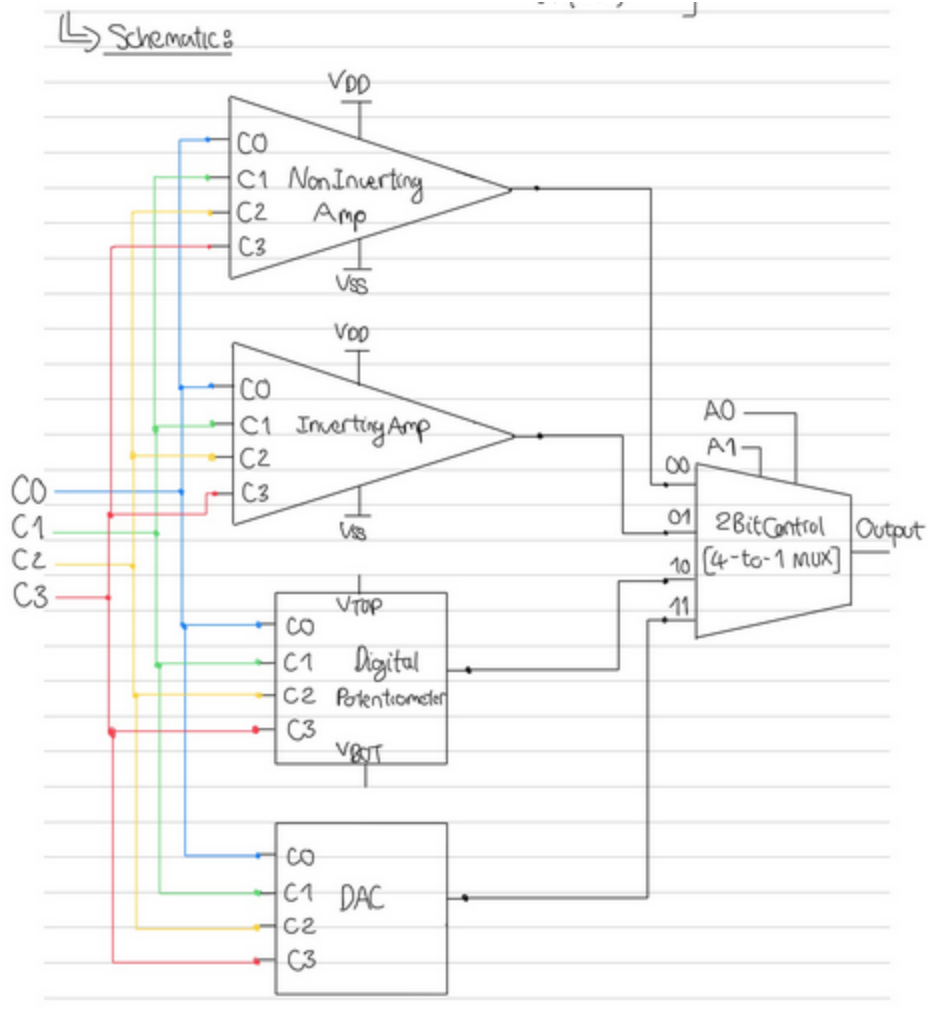
Besides the logic of the circuit, we are also required to complete the physical layout. This means that we have to use Cadence for the project and apply what we have learned in the labs throughout the semester to complete the project.

Additionally, because we have never actually built a potentiometer or an op-amp in cadence, extra research is also required for us to successfully build the circuit.

II) Project Process:

a) General understanding and sketch of the project

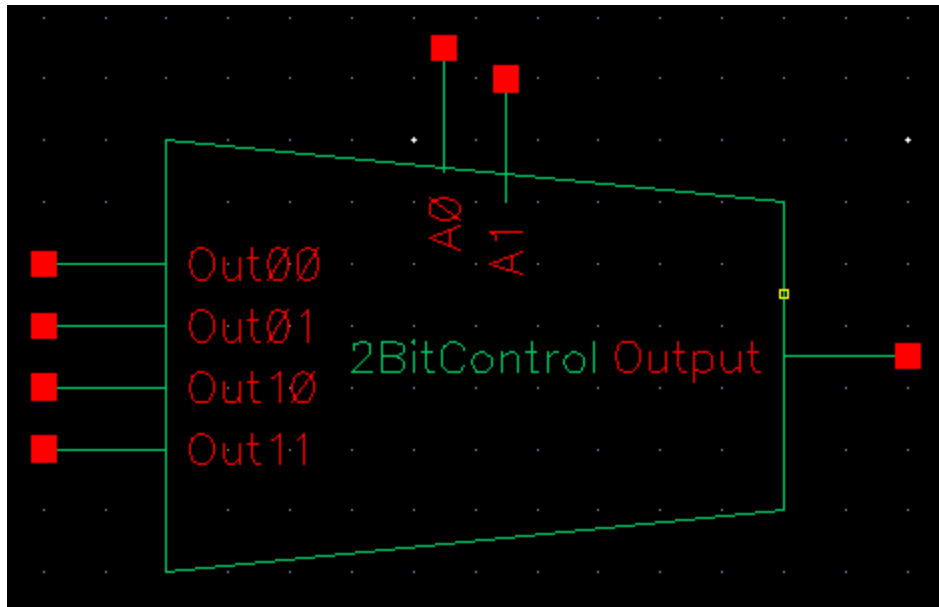
According to the reading about our Final Project from the project list, our final schematic should look like the draft for the high-level schematic that we sketched. This schematic consists of five main components: a non-inverting amplifier, an inverting amplifier, a digital potentiometer, a DAC, and a 2-bit mode control signal A0A1. There are four main inputs C3C2C1C0 (we mistakenly wrote down a different order in my draft), going into the first four components, as you can see in the figure below. After that, those four components will bring out four different outputs, and they're called 00, 01, 10, and 11, respectively. The reason for that is because the fifth component, which is the 2-bit mode control, acts as a 4-to-1 MUX with two selectors A0A1. The MUX here chooses the final output based on the two selectors. For example, if A0A1 is set to 00, the output of the non-inverting amplifier will be the final output.



(Figure 1: This is our high-level schematic sketch)

b) Schematic and layout completion

First, we started the project by building the 2-bit mode control signal A0A1 which is basically a 4-to-1 MUX with two selectors A0A1.



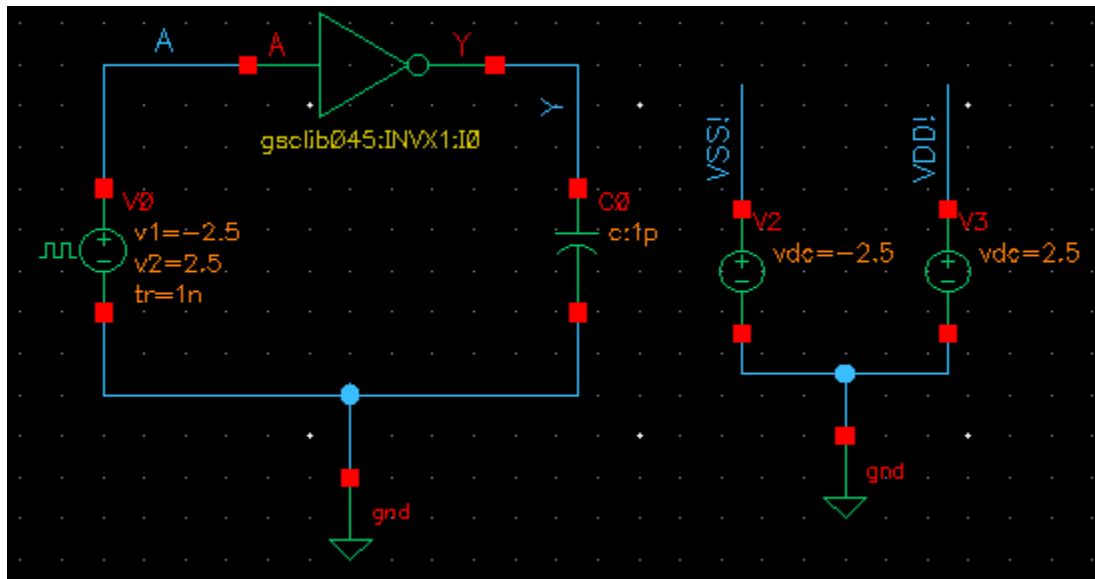
(Figure 2: This is 2-Bit Mode Control)

In order to build the 2-Bit Mode Control, we needed to build two components: a 2-to-4 Decoder and a Bidirectional Switch. So, we started by building the Bidirectional Switch.

Bidirectional Switch

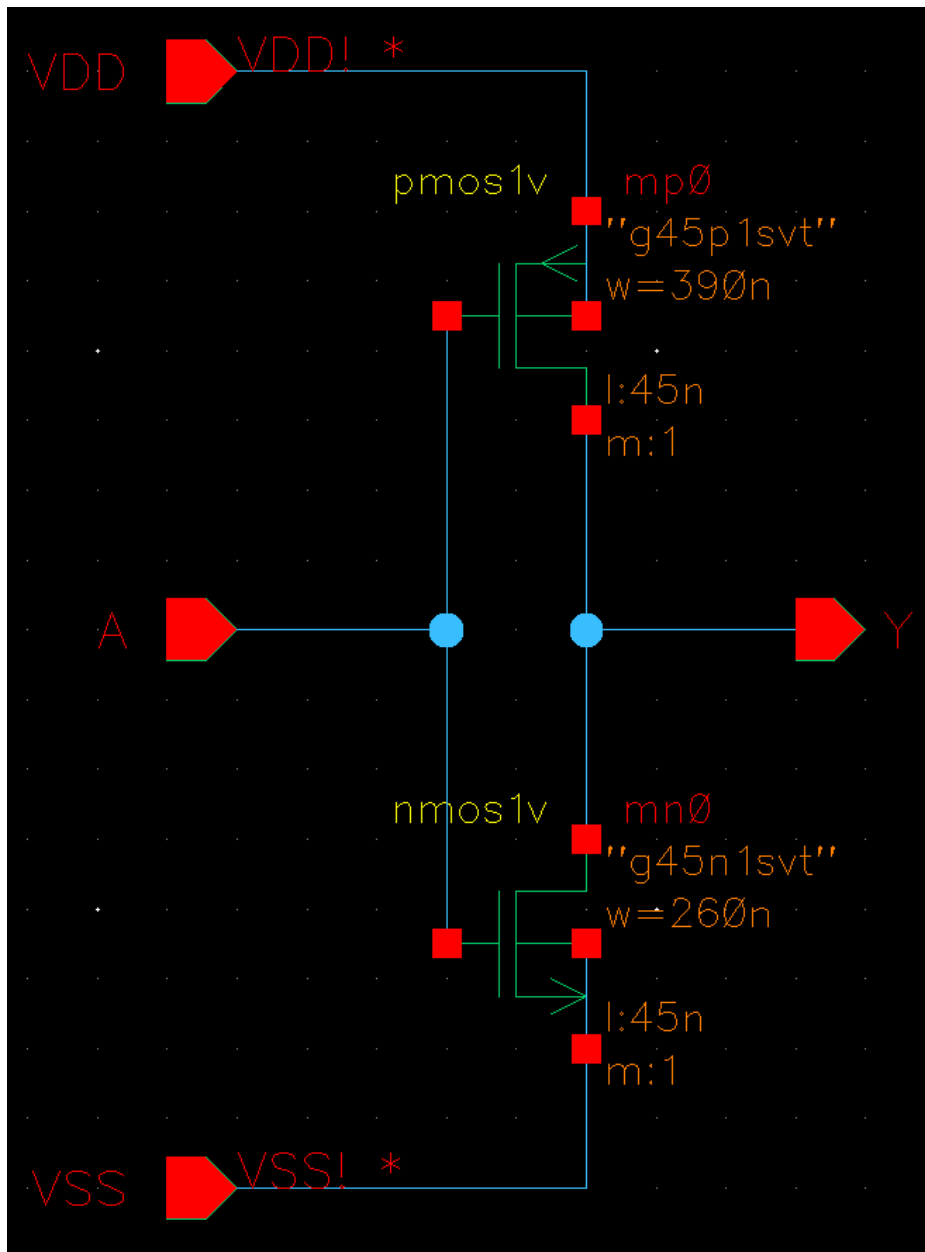
Bidirectional switch controls whether there is a voltage at the output or not based on the control signals. In order to achieve that function, we must use the combination of an NMOS and a PMOS (so it can pull up and down smoothly). Then we would want to tie one end of the MOS transistor (source / drain together) to the input while the other end is tied to the output. For the control, we can tie an input pin to the gate of both MOSFETs. However, as Sam reminded us before, to control the PMOS appropriately, we must not use the same signal for both the gates; instead, the control signal should be fed into a NOT gate before being connected to the gate of the PMOS. Thus, we have to explore the gsclib045 NOT gate a bit to see if we can use it for our circuit:

We first needed to make sure that the logic for the NOT gate was valid. To do so, we made a test bench for it to test if the NOT gate from gpdk45 worked.



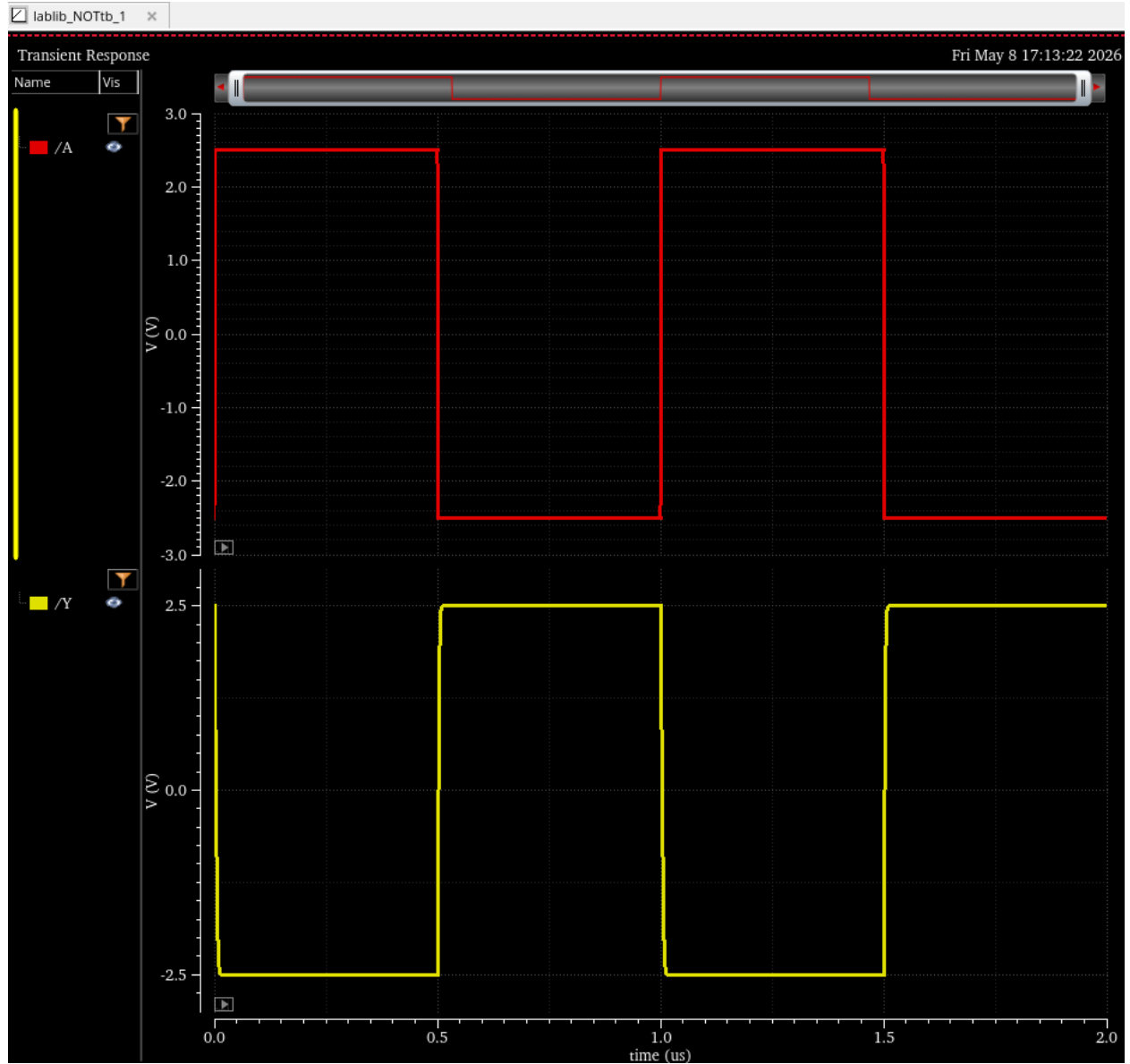
(Figure 3: The test bench circuit for NOT gate)

Since the schematic circuit for the NOT gate from the gsclib045 library was using the global supplies ($VSS!$ & $VDD!$), so in the test bench schematic, we also had to use those.



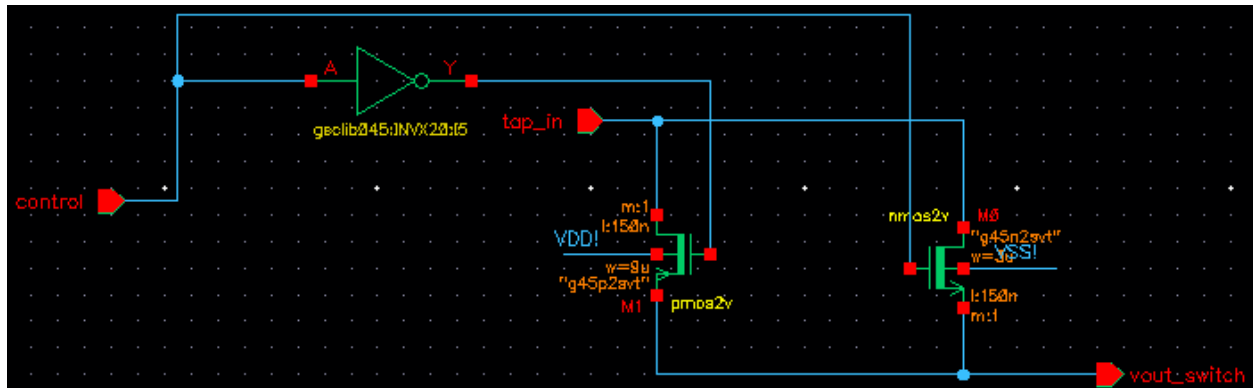
(Figure 4: The given schematic for NOT gate)

After running the simulation, we got this result from the test bench, and this proved that the NOT gate from the gsclib045 library was working properly.



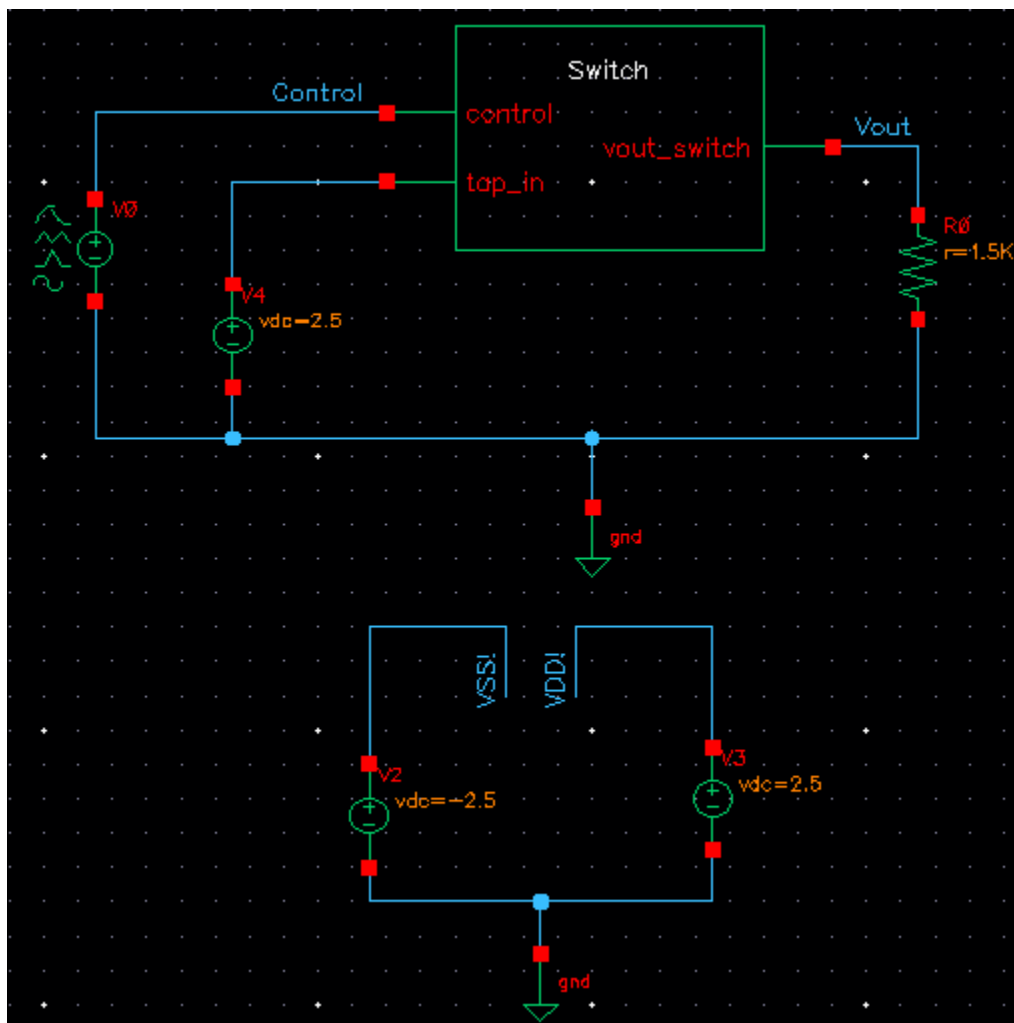
(Figure 5: This is the result for the test bench circuit for NOT gate)

Then, we used this NOT gate along with an NMOS and a PMOS to build the Bidirectional Switch with the logic we mentioned above:



(Figure 6: This is schematic for Bidirectional Switch)

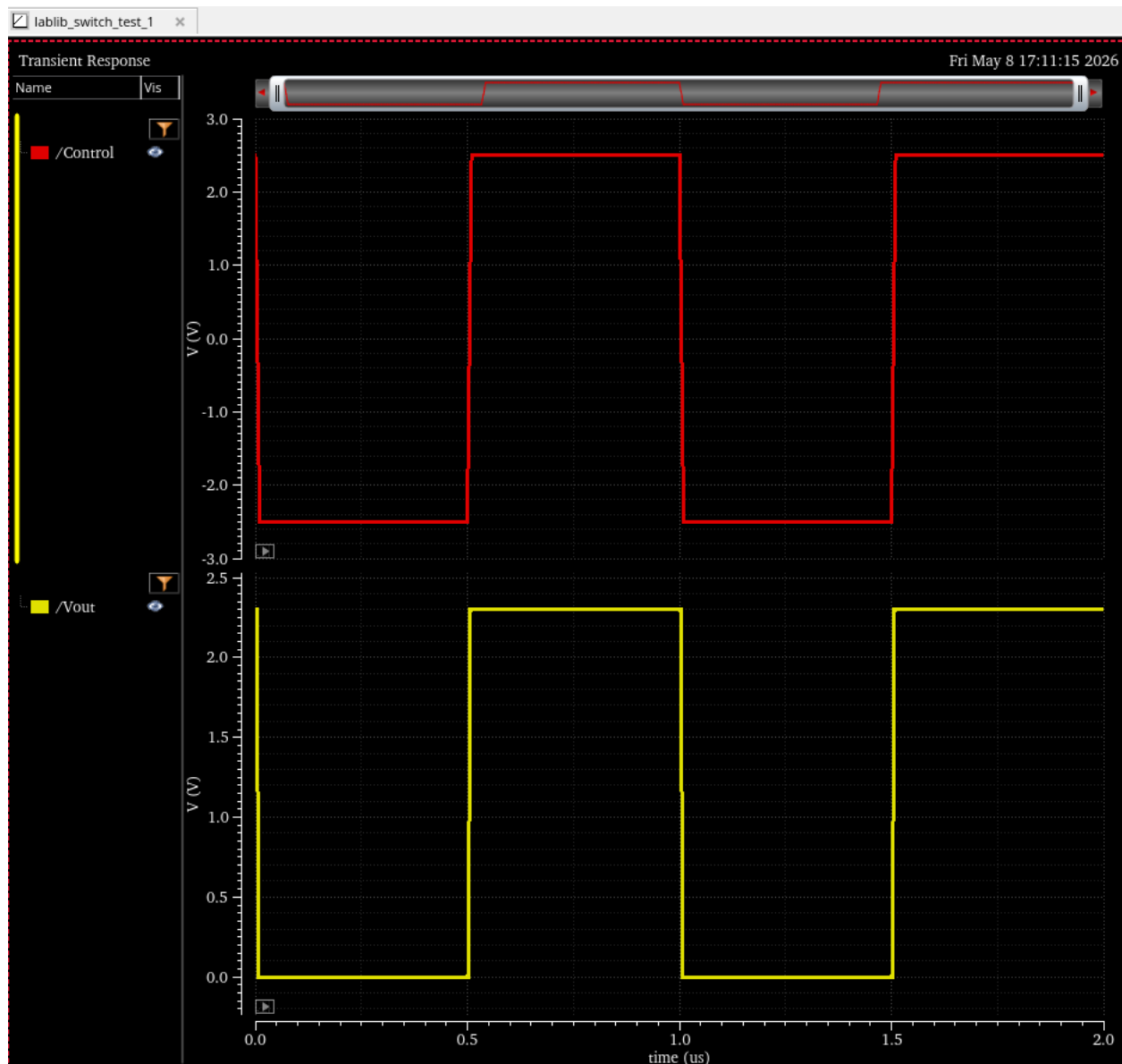
We also built a testbench for the switch. We just reused testbench from previous lab for this except one small change, instead of a capacitor at the output, we try to use a resistor to test if our circuit can work without the help of a capacitor.



(Figure 7: The test bench circuit for Bidirectional Switch)

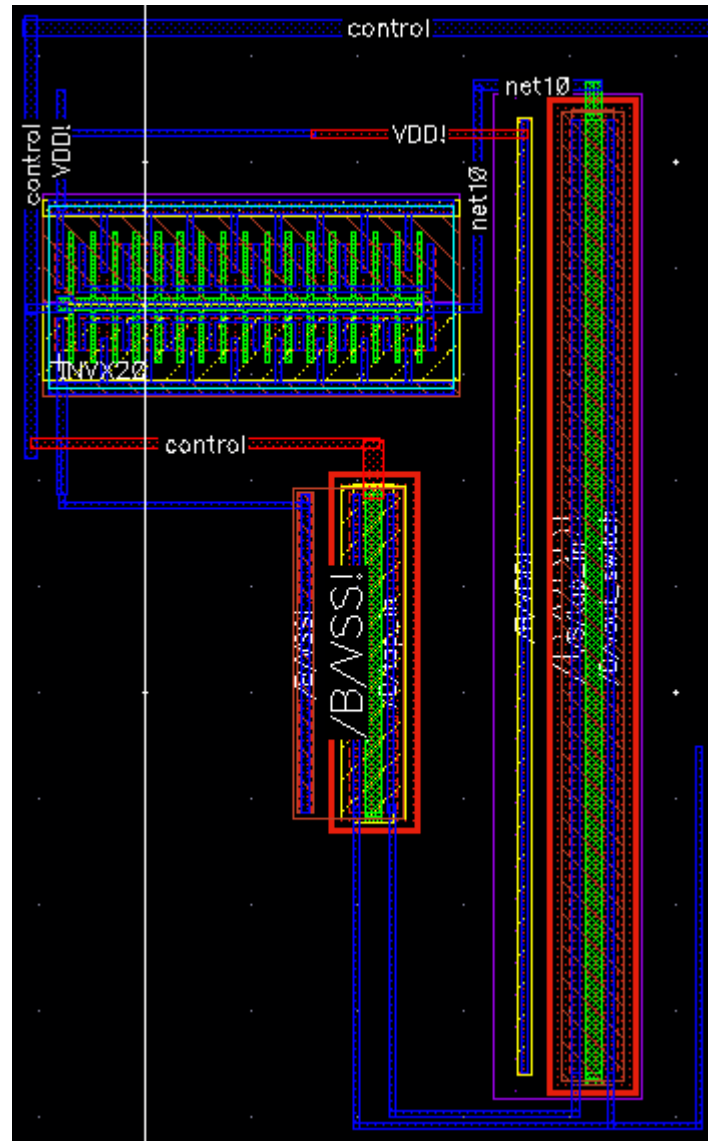
Using Maestro, we tested the bidirectional switch using transient analysis. This is shown below. When the control line has a voltage, the output should be on (or have some voltage); then, when the control has no voltage, the output should have no voltage.

In the waveform, whenever the control is high, V_{out} will almost be equal to V_{in} , which is 2.5 V DC voltage (it's a bit off because of the resistor). That matches what we expected. So, we concluded that it worked fine.

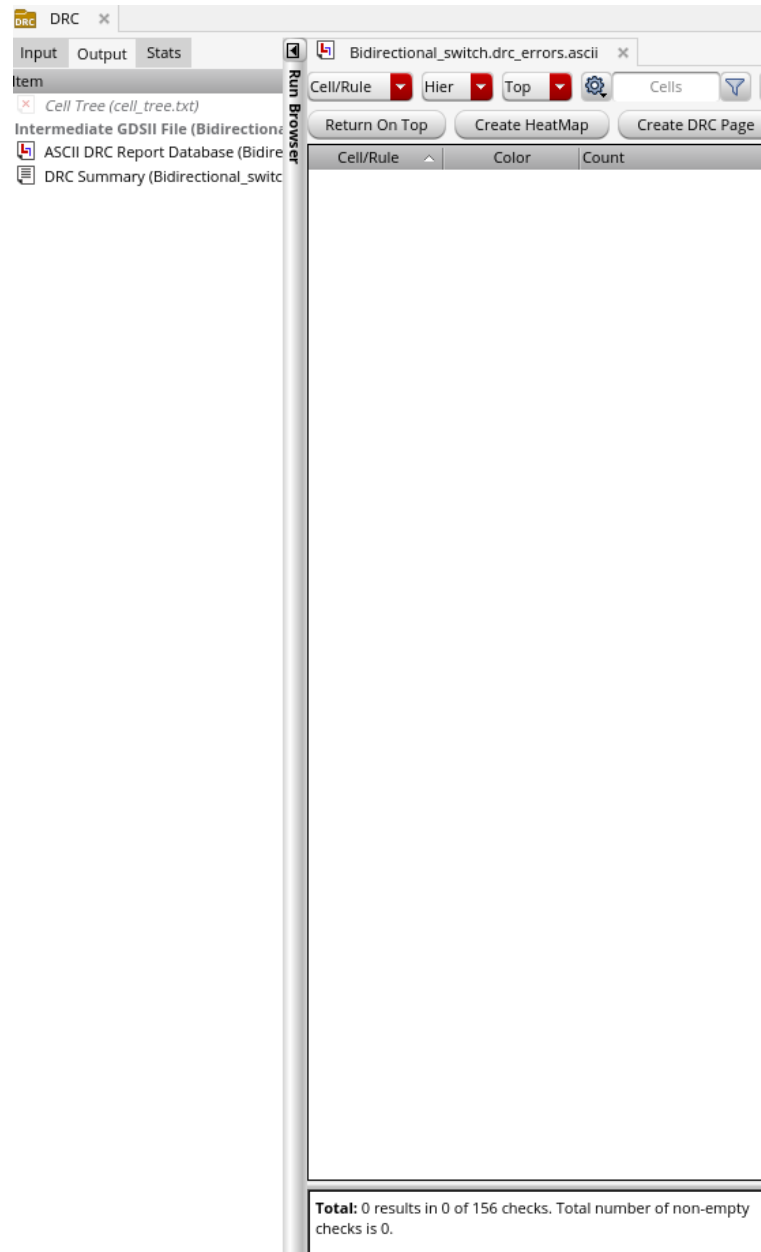


(Figure 8: This is the result for the test bench circuit for Bidirectional Switch)

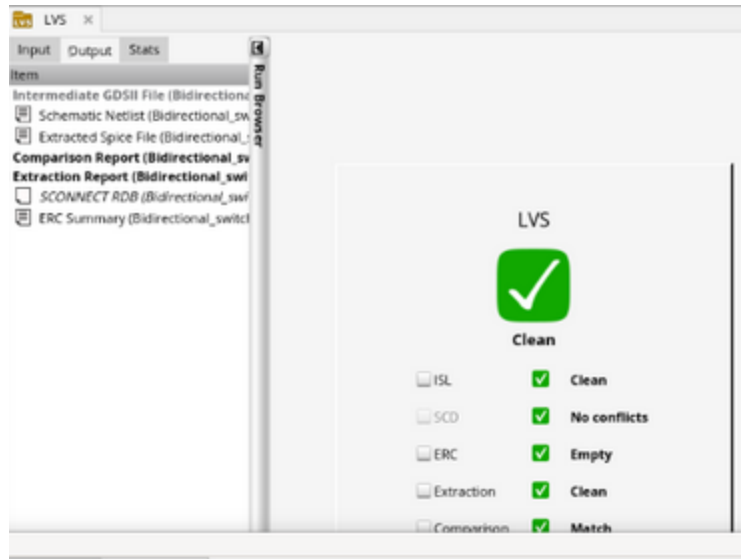
After making sure the schematic is correct, we went on to build the layout of the switch schematic and ran some checks to see if the layout was valid.



(Figure 9: This is the layout for Bidirectional Switch)



(Figure 10: This is the DRC result for the layout for Bidirectional Switch)



(Figure 11: This is the LVS result for the layout for Bidirectional Switch)

Once we finished building the Bidirectional Switch, we moved to building the 2-to-4 Decoder. And in order to build the 2-to-4 Decoder, we had to use a couple of NOT gates and four AND gates.

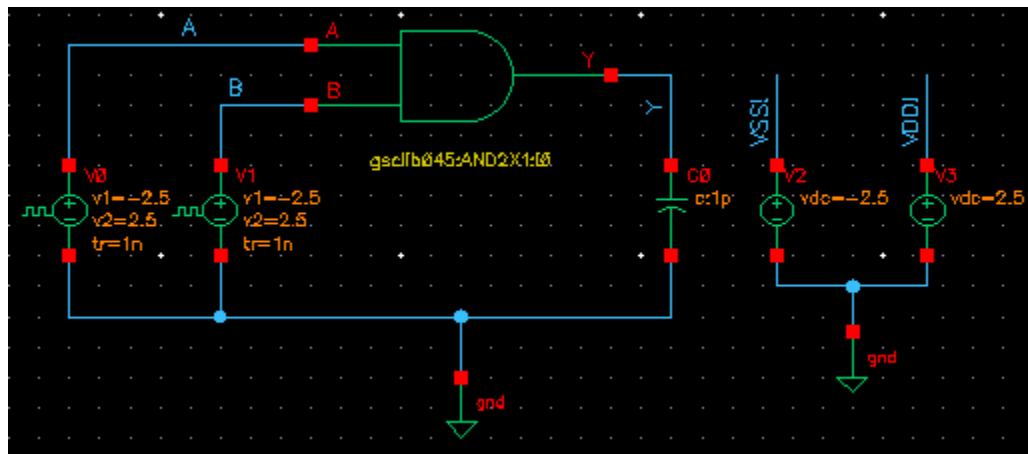
2-to-4 Decoder



(Figure 12: The symbol block for 2-to-4 decoder)

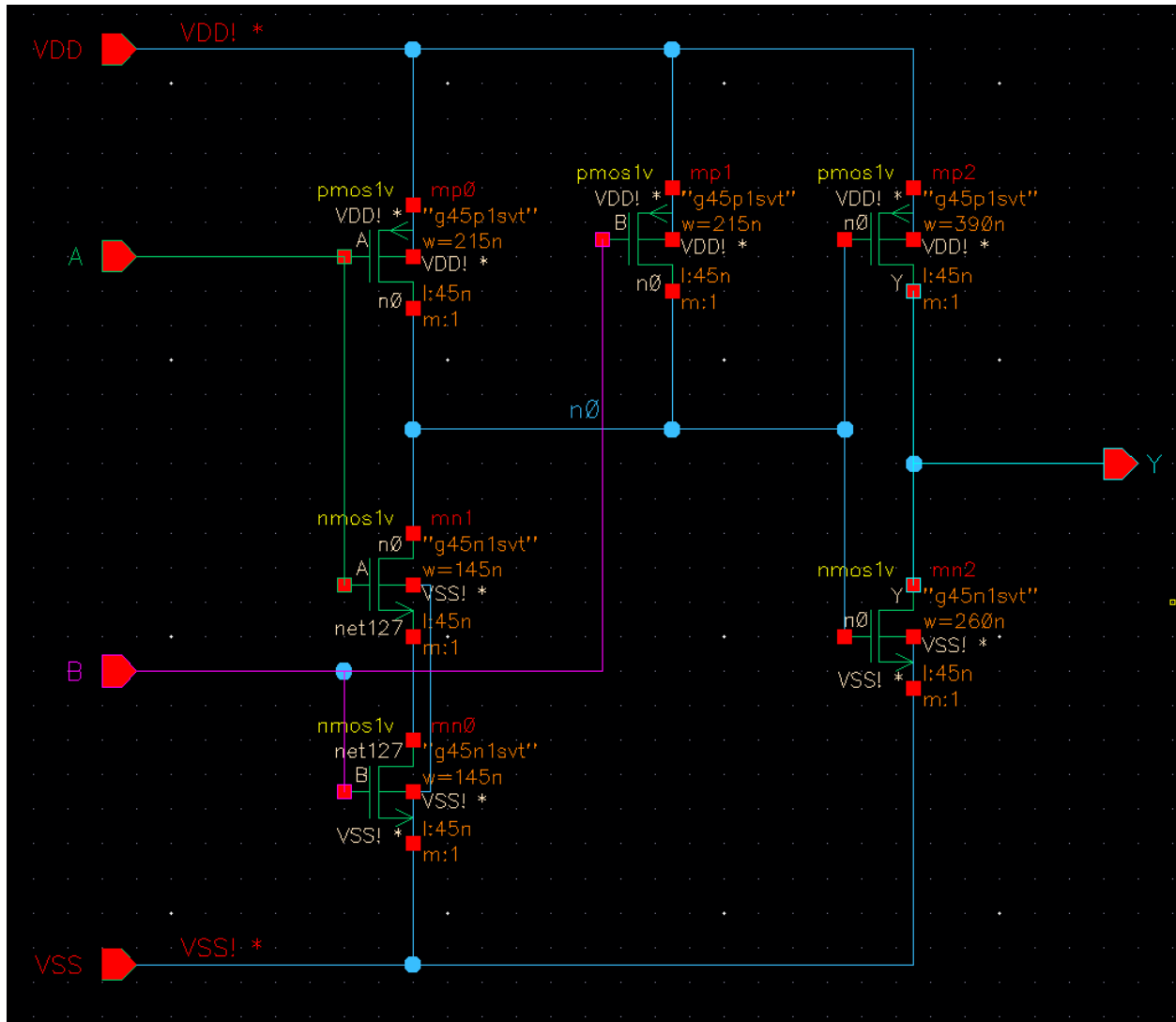
However, since we used gsclib045 library from gpdk45 process as we stated, we already tested out the NOT gate. So, we next needed to make sure that the logic for AND gate was valid. To do that, we made a testbench for it to test if the NOT gate from gpdk45 worked.

Test Bench for AND gate



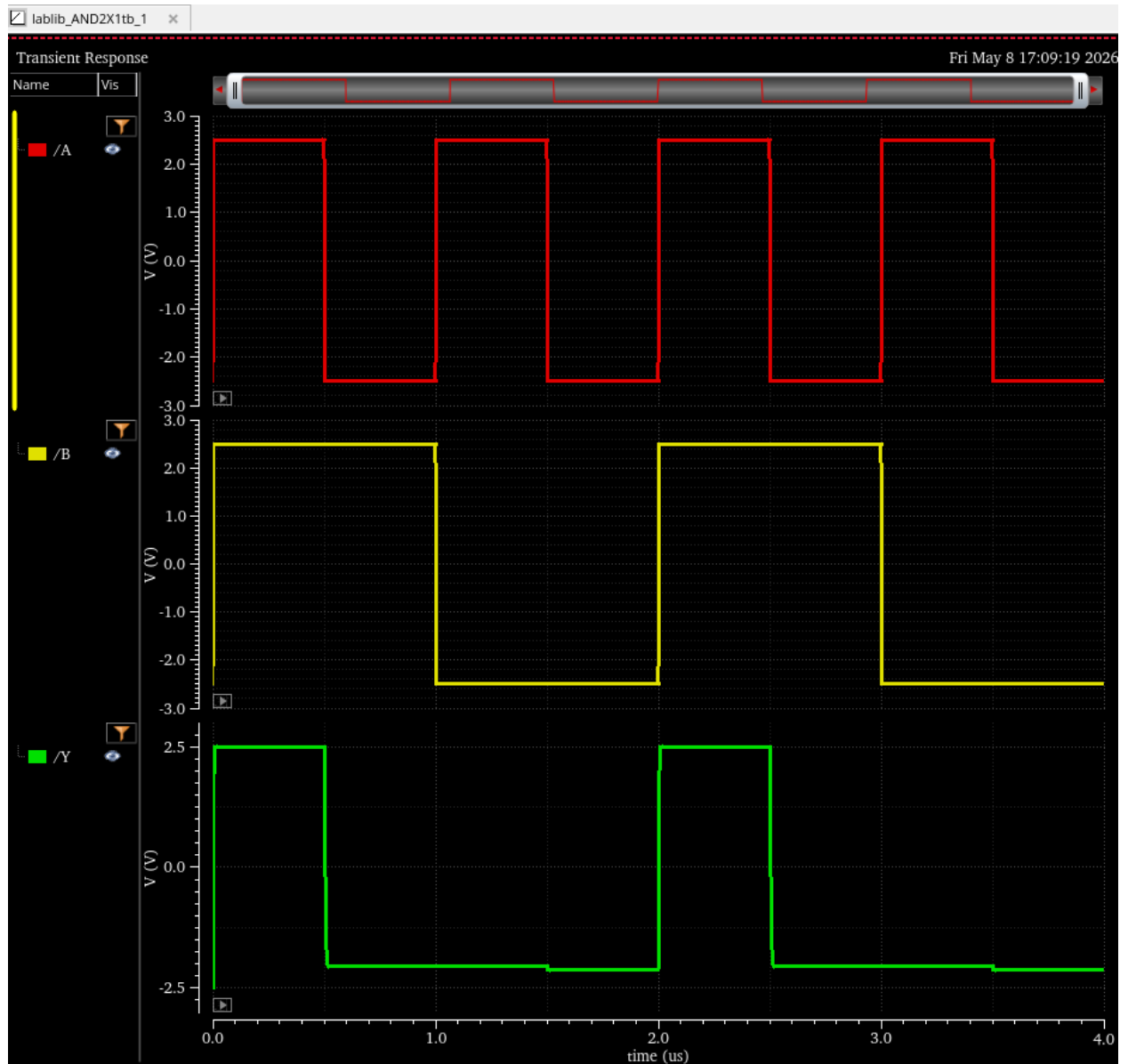
(Figure 13: This is the test bench circuit for AND gate)

Since the schematic circuit for the AND gate from gsclib045 library was using the global supplies (VSS! & VDD!), so in the test bench schematic, we also used those.



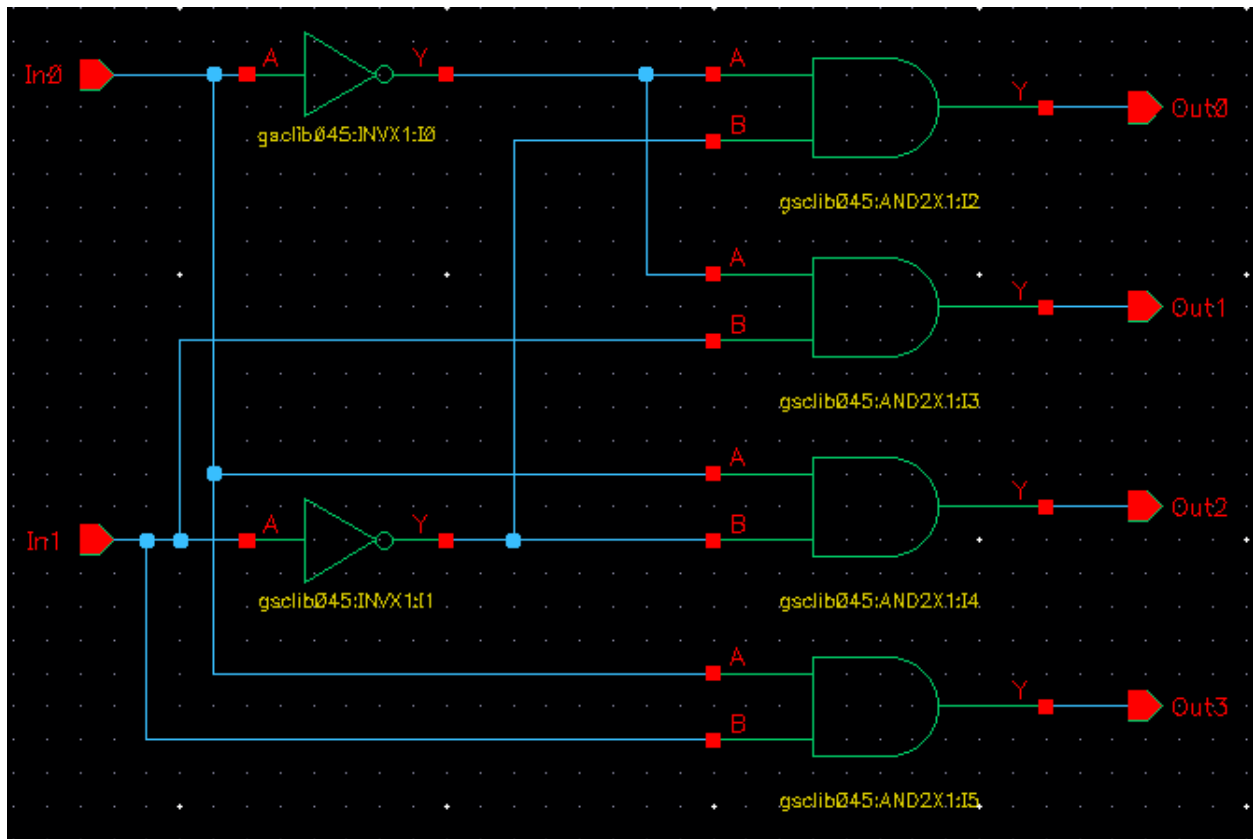
(Figure 14: This is the given schematic for AND gate)

Output only has voltage when both A and B are on. This result from the test bench proved that the AND gate from gsclib045 library was working properly.



(Figure 15: This is the test bench result for the test bench for AND gate)

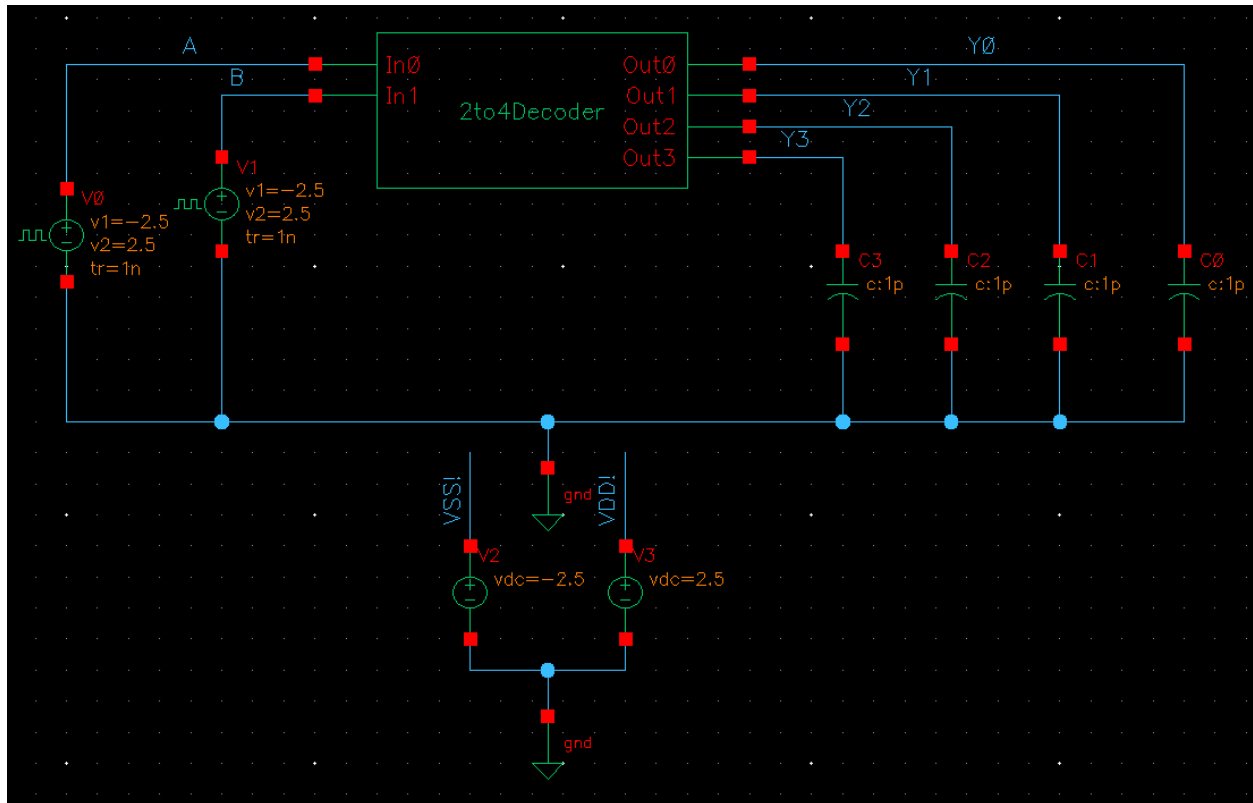
Then, we used this AND gate to build the 2-to-4 Decoder.



(Figure 16: This is the schematic for 2-to-4 decoder)

We also built the layout of this schematic and ran some checks to see if the layout was valid.

After that, we next built a test bench to see if the logic of our schematic worked as intended as the logic of the 2-to-4 Decoder.

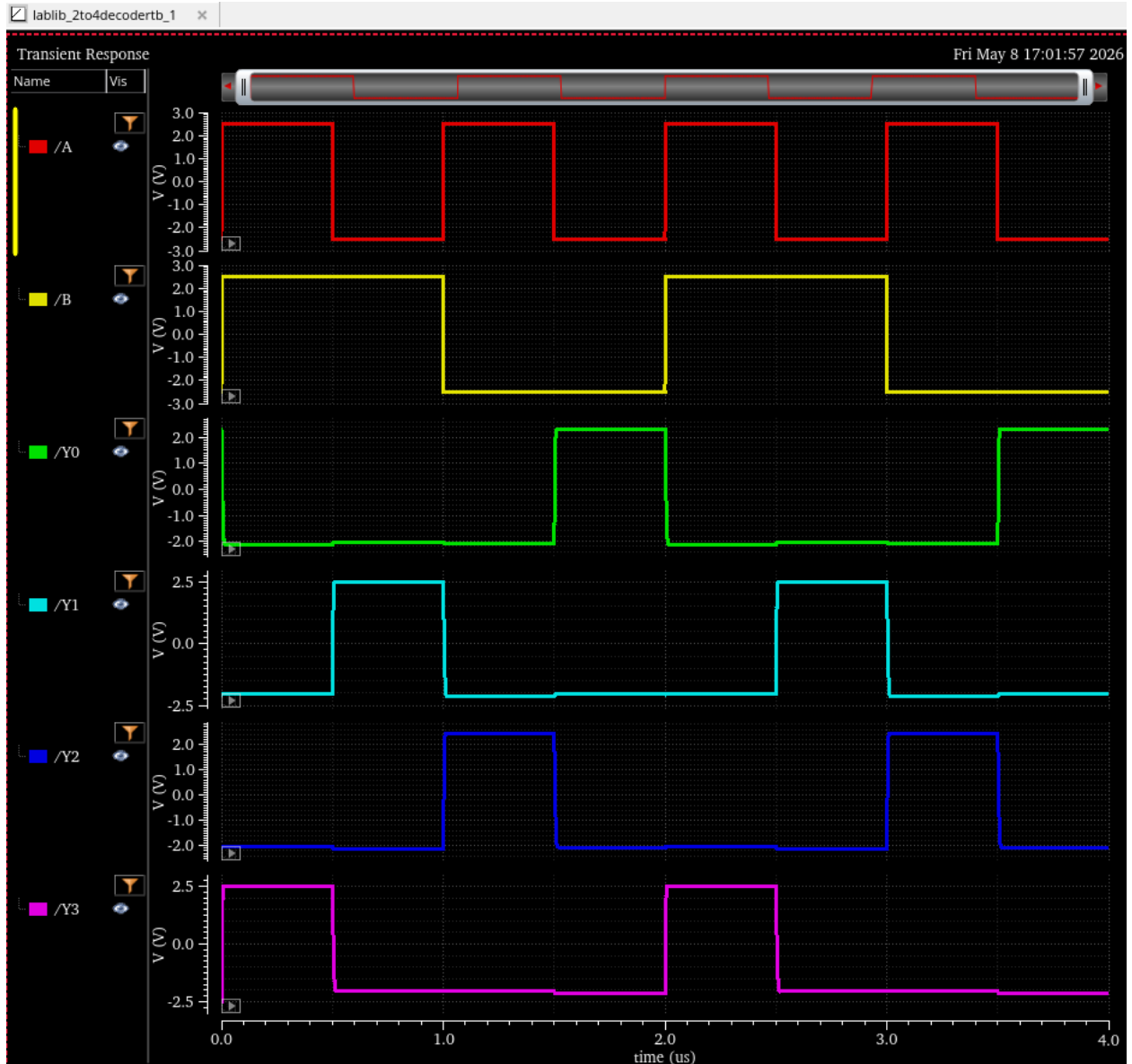


(Figure 17: This is the test bench circuit for 2-to-4 decoder)

The testbench is quite similar to all tb in previous projects. The input will be square wave input with different periods. Using this testbench schematic, we used a waveform maestro to run a transient analysis of one of the highest input periods. This allowed us to view the input and output waveform response and see if it ran correctly.

Initially, we expected B to be the bit 1 while A is bit 0. Thus, we expected those cases to happen in our waveform:

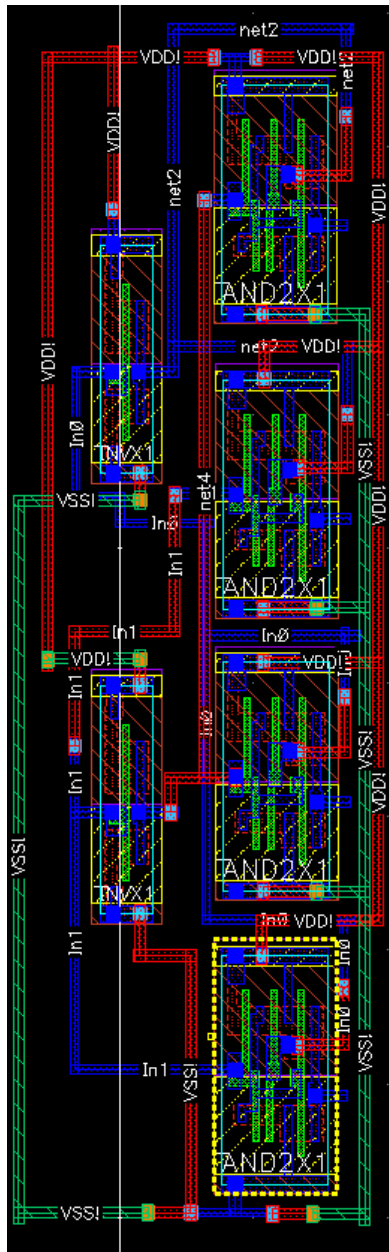
- _ When A and B are both off: Y0 should be on.
- _ When A and B both on: Y3 should be on.
- _ When A is on and B is off: Y1 should be on.
- _ When A is off and B is on, Y2 should be on.



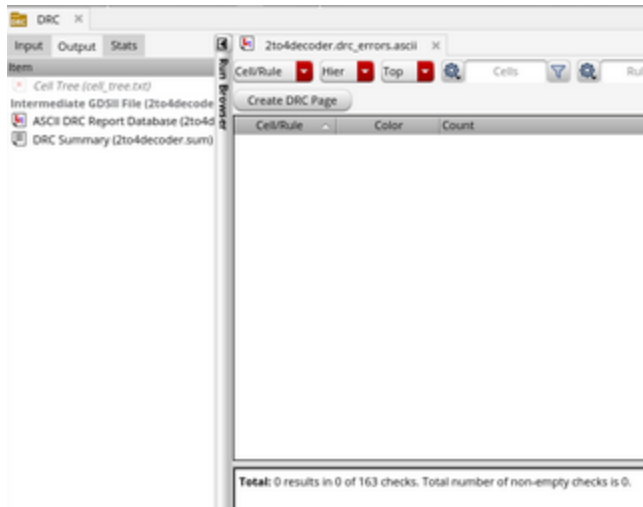
(Figure 18: This is the result for the test bench for 2-to-4 decoder)

The result matches what we expected above, so we were convinced that our 2-to-4 decoder function appropriately.

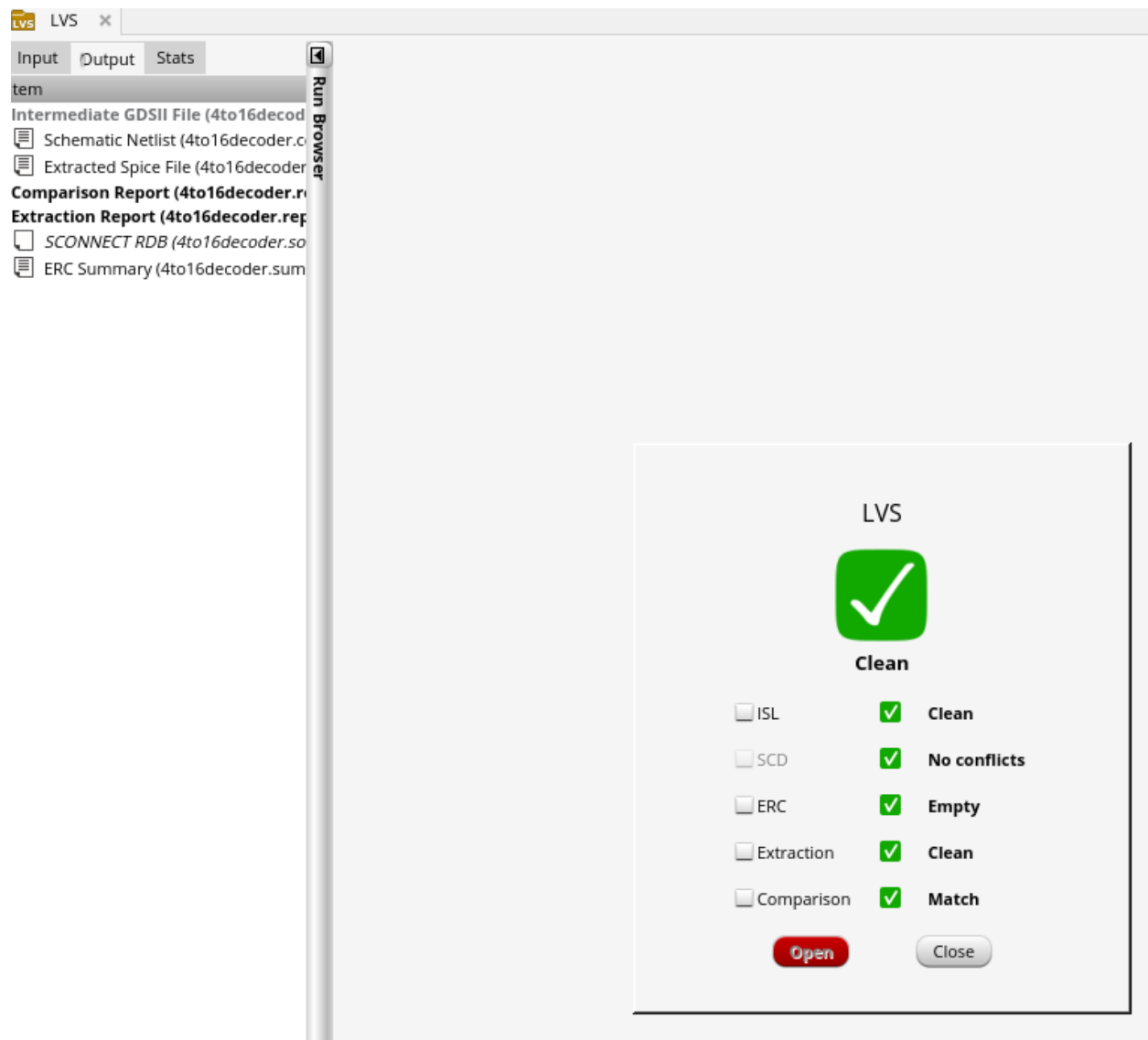
After making sure the schematic function is correct, we went on to complete the layout and run LVS + DRC test.



(Figure 19: This is the Layout for 2-to-4 decoder)

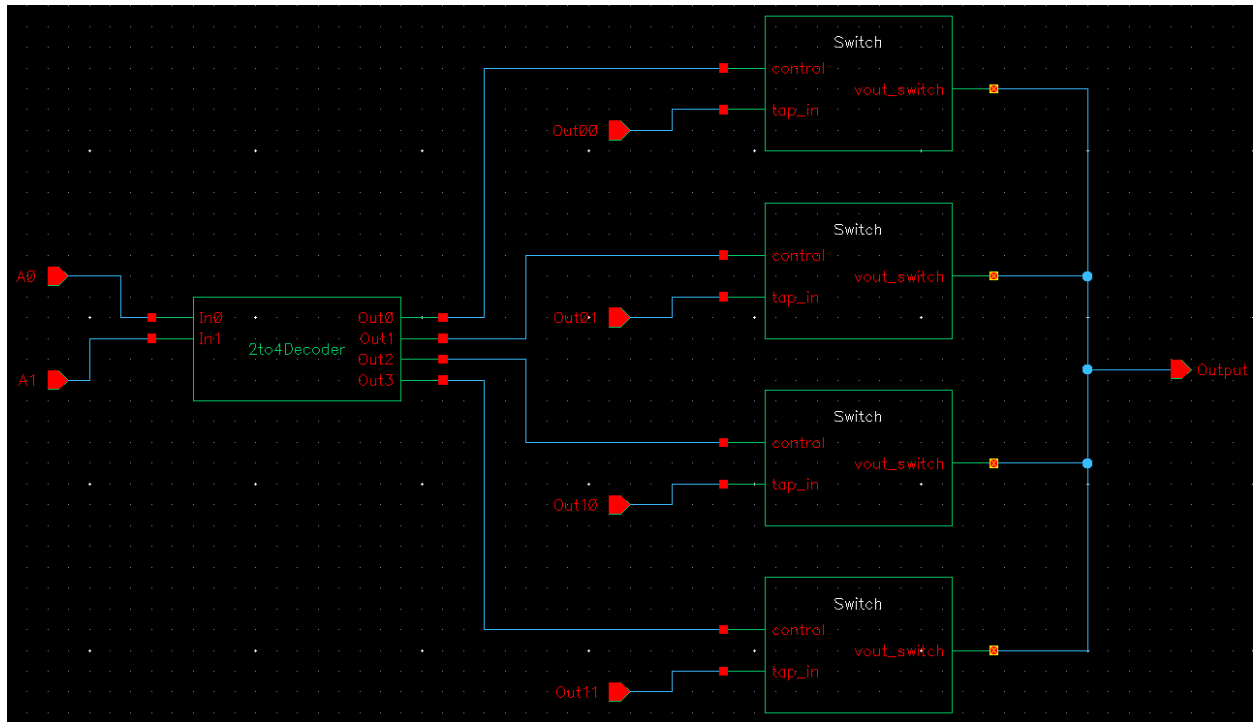


(Figure 20: This is the DRC result for the layout for 2-to-4 decoder)



(Figure 21: This is the LVS result for the layout for 2-to-4 decoder)

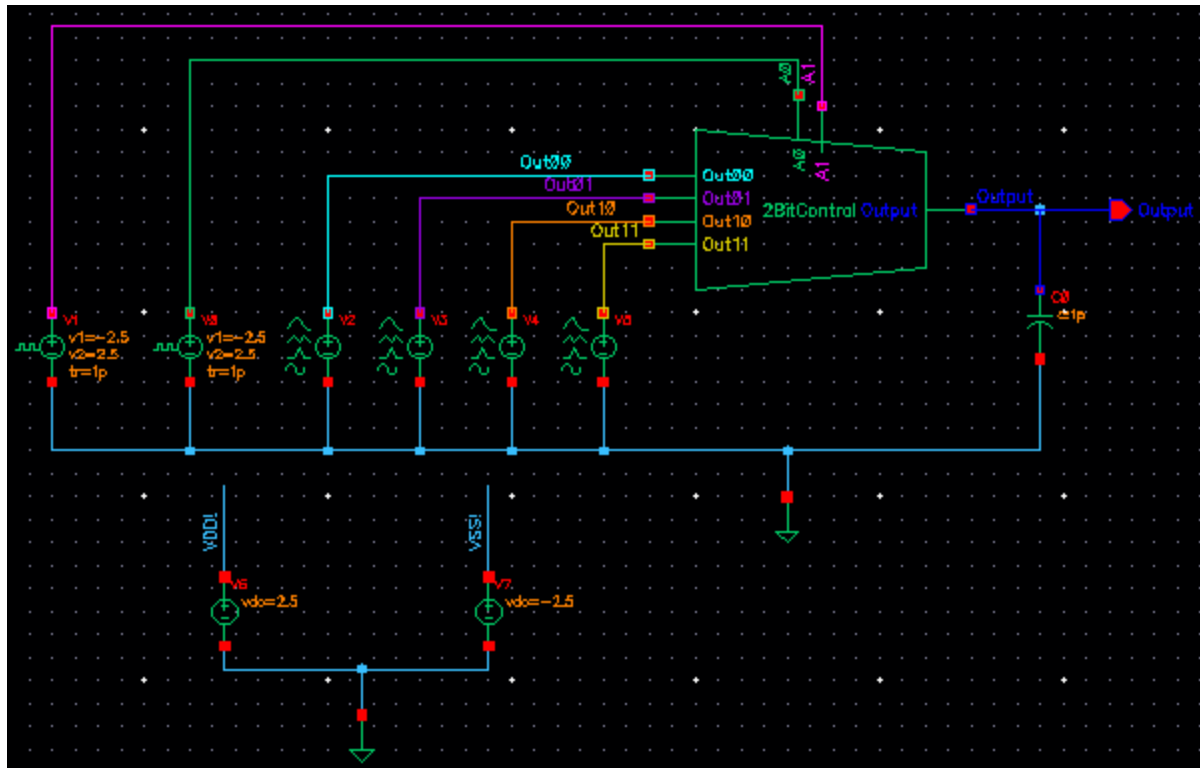
After building the two components – Bidirectional Switch and 2-to-4 Decoder, we started building the 2-Bit Mode Control. As mentioned above, this is just a 2-to-4 Mux, depends on the combination of the input, the corresponding switch will be on and allows the output to have the voltage waveform of the input at that terminal. Our schematic is below:



(Figure 22: This is the schematic for 2-Bit Mode Control)

After that, we next built a test bench to see if the logic of our schematic worked as intended as the logic of the 2-Bit Mode Control.

We built the testbench with four different inputs, including two sin waves with different period and two square waves with different period at the Vin (input voltage that output will carry). For the control line, we also had two square waves with the same amplitude but different periods to simulate the control lines. The testbench schematic is shown below.



(Figure 23: The test bench circuit for 2-Bit Mode Control)

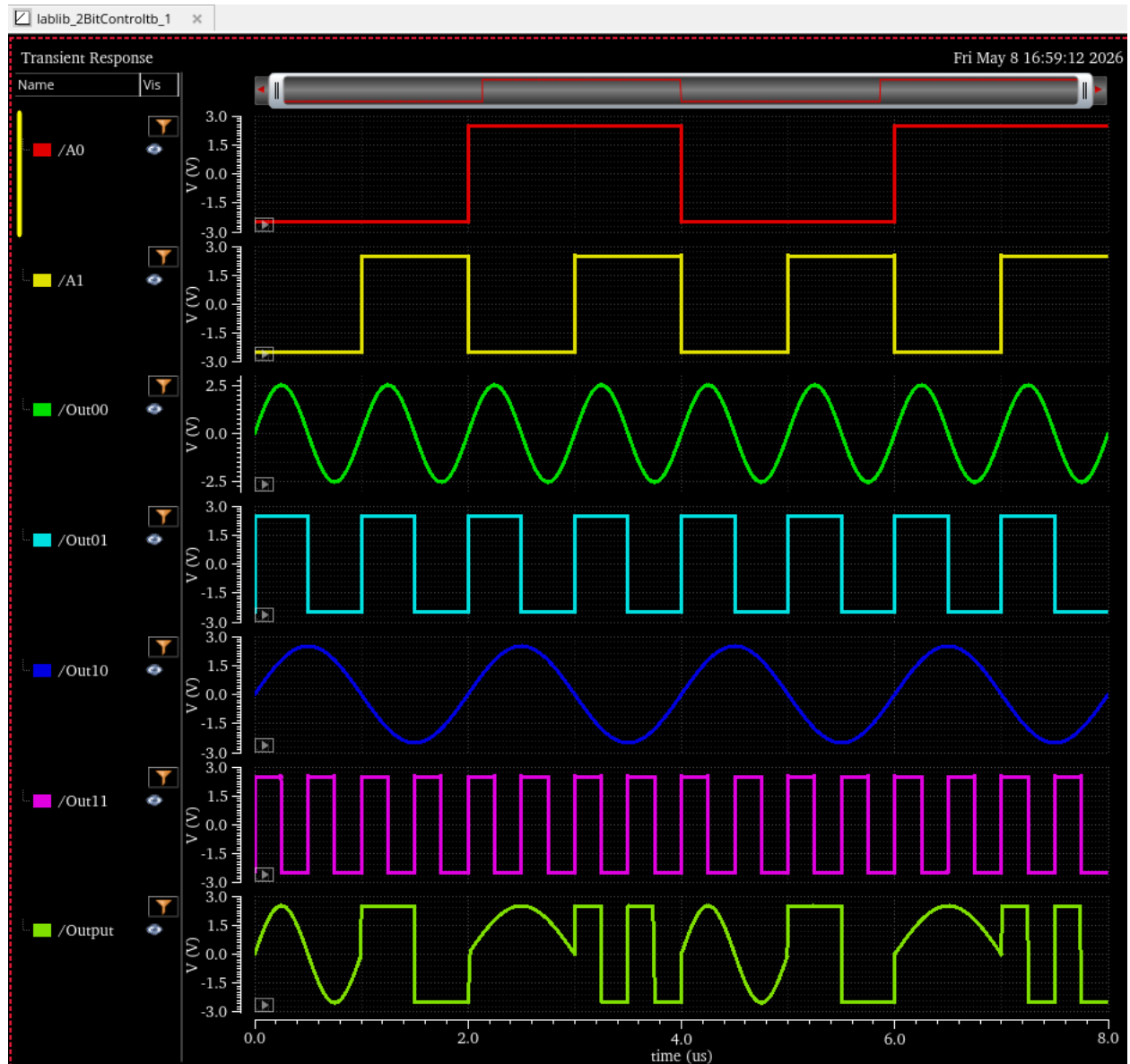
For each possible control input, it allowed a different wave through (A0 is bit 1 while A1 is bit 0) :

When A0 and A1 is off it should be a short-period sine wave.

When A0 is off and A1 is on , the output should be long period sine wave

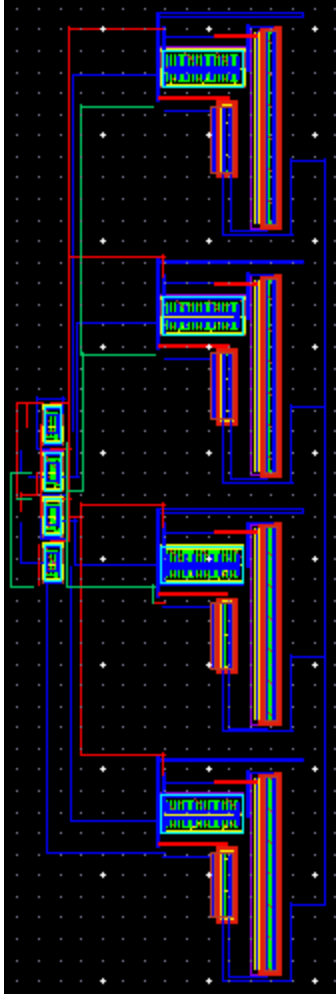
When A0 is on and A1 is off , output should be a long period sine wave.

When A0 and A1 are on , the output should be short period square wave

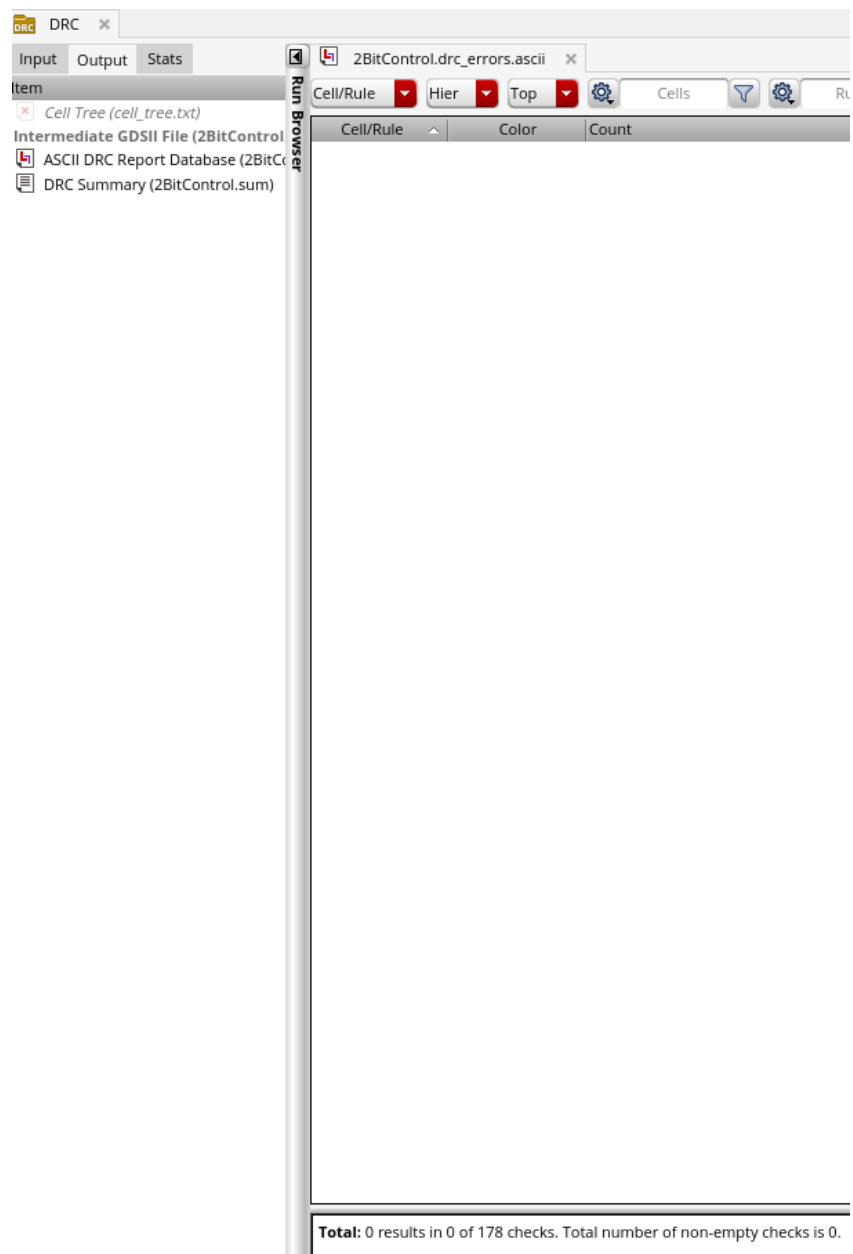


(Figure 24: The result for the test bench circuit for 2-Bit Mode Control)

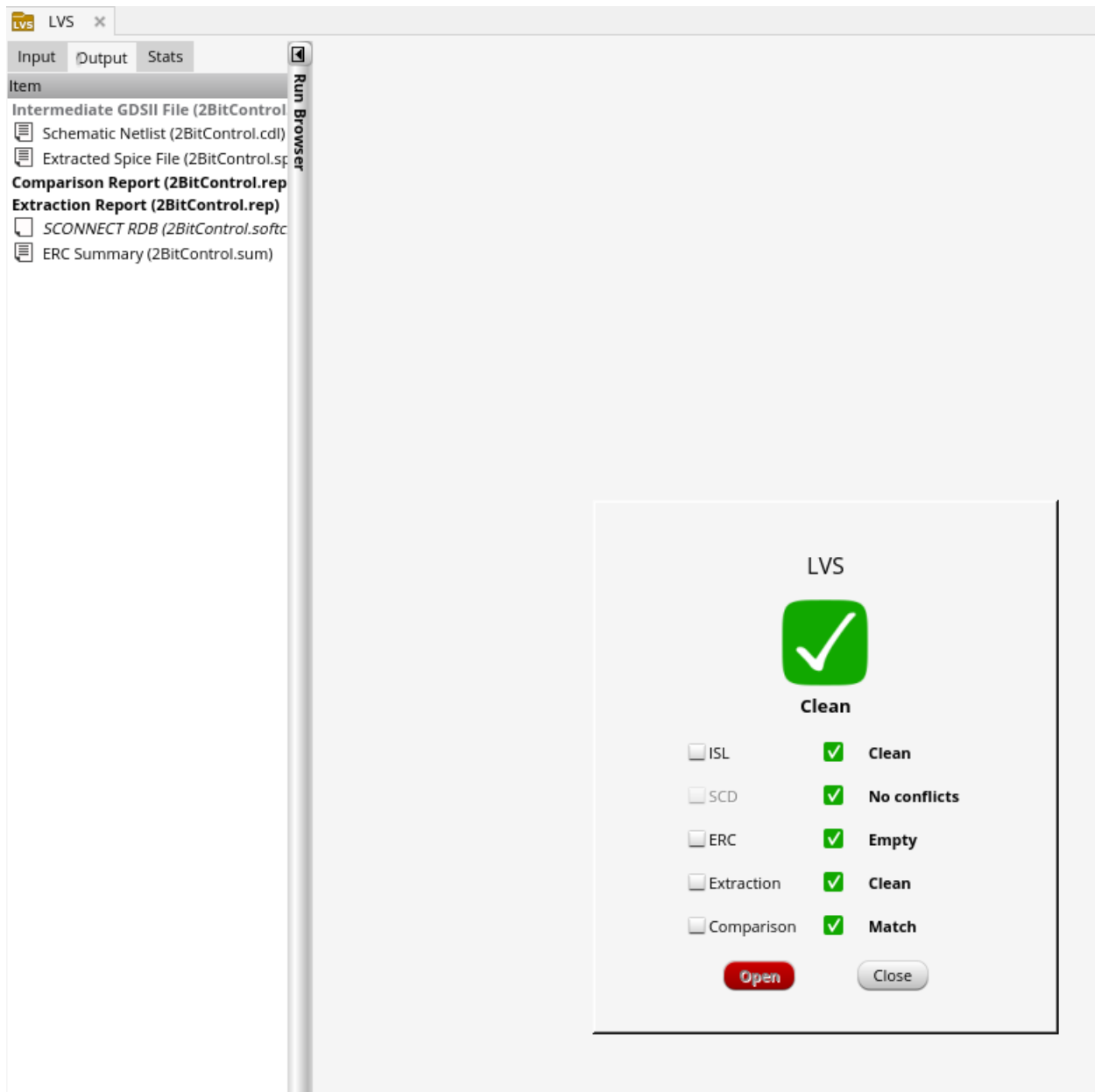
The result matched what we expected so we moved on to build the layout.



(Figure 25: This is the layout for 2-Bit Mode Control)



(Figure 26: DRC result for the layout for 2-Bit Mode Control)



(Figure 27: LVS result for the layout for 2-Bit Mode Control)

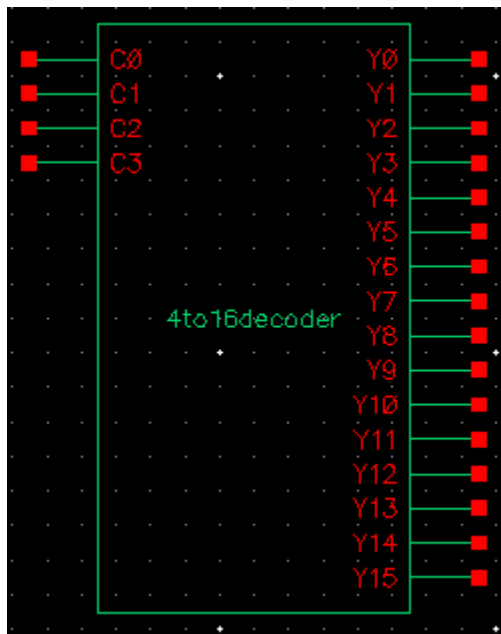
Digital Potentiometer

Once we finished building the Bidirectional Switch, we moved to the Digital Potentiometer. So the potentiometer is just a string of 15 resistors. That will give us 16 voltage taps which will be connected to 16 bidirectional switches. And based on which switch is on, the voltage output will be different (which

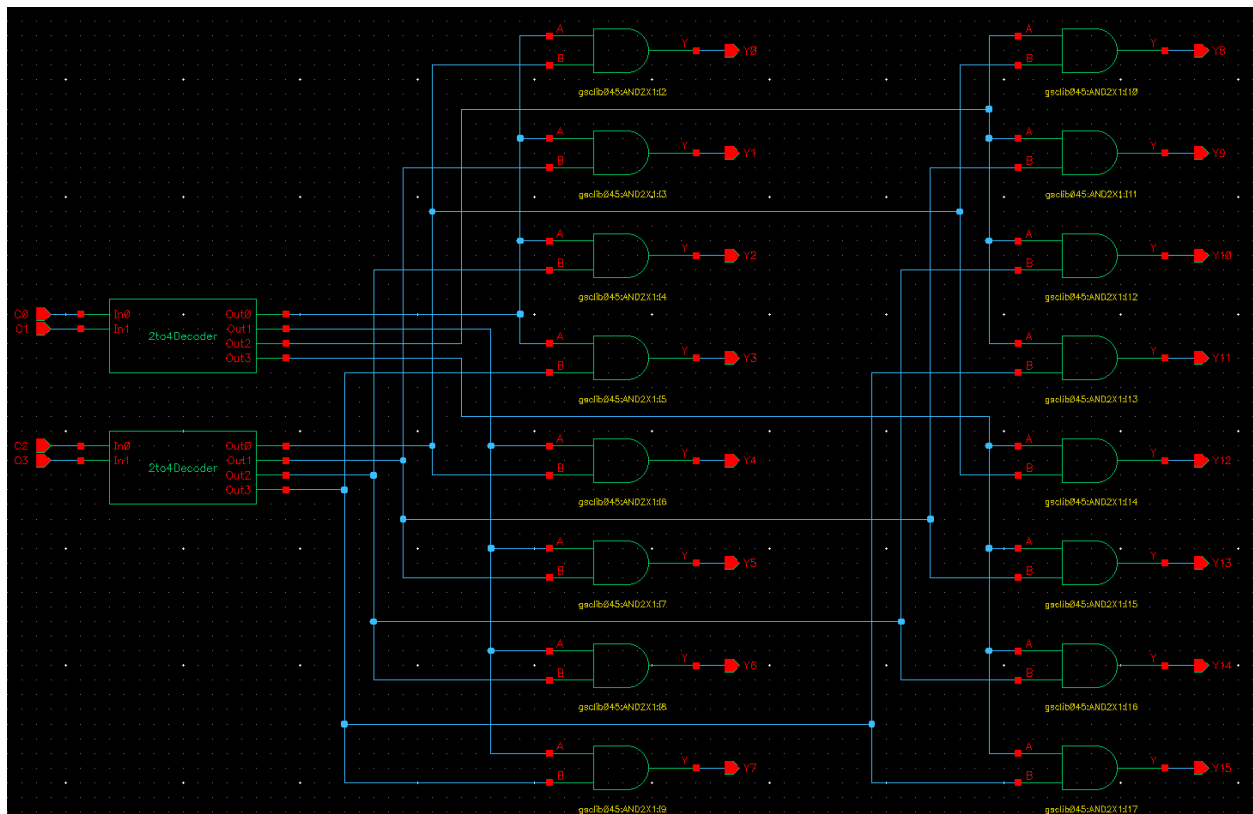
makes it feel like different resistance value) . We can make the potentiometer to be programmable by using a 4 to 16 decoder to control which switch is on. Thus , that is what we decided to build next.

4-to-16 Decoder

To build a 4-to-16 Decoder using a couple of 2-to-4 Decoders that we built above along with sixteen AND gates.

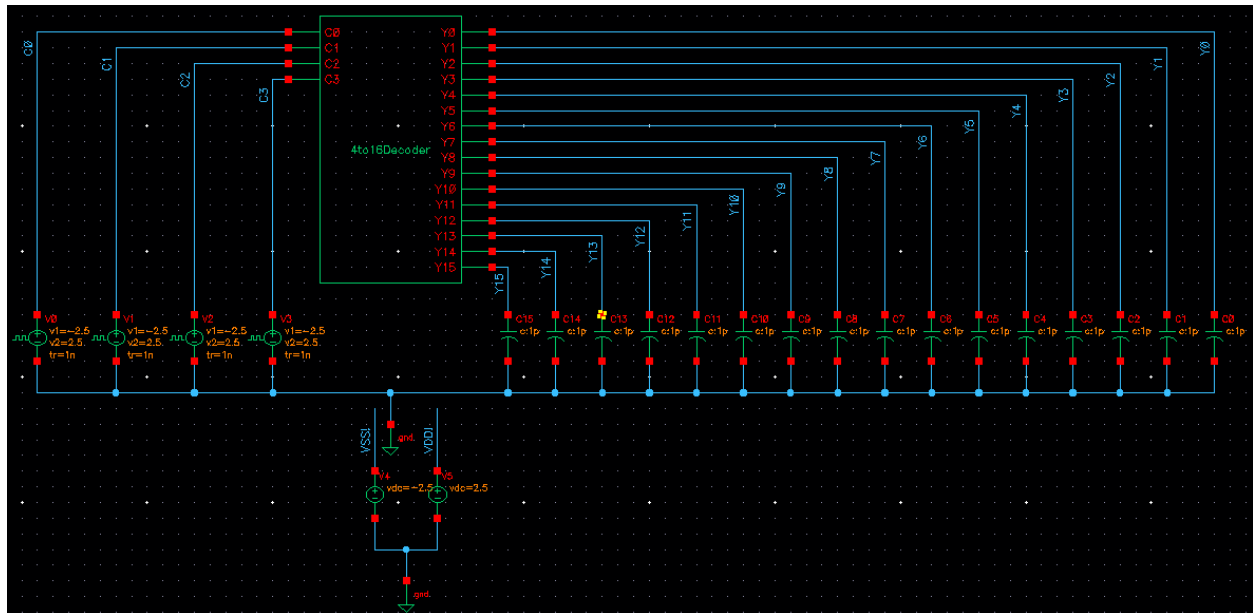


(Figure 28: This is the symbol block for 4-to-16 decoder)



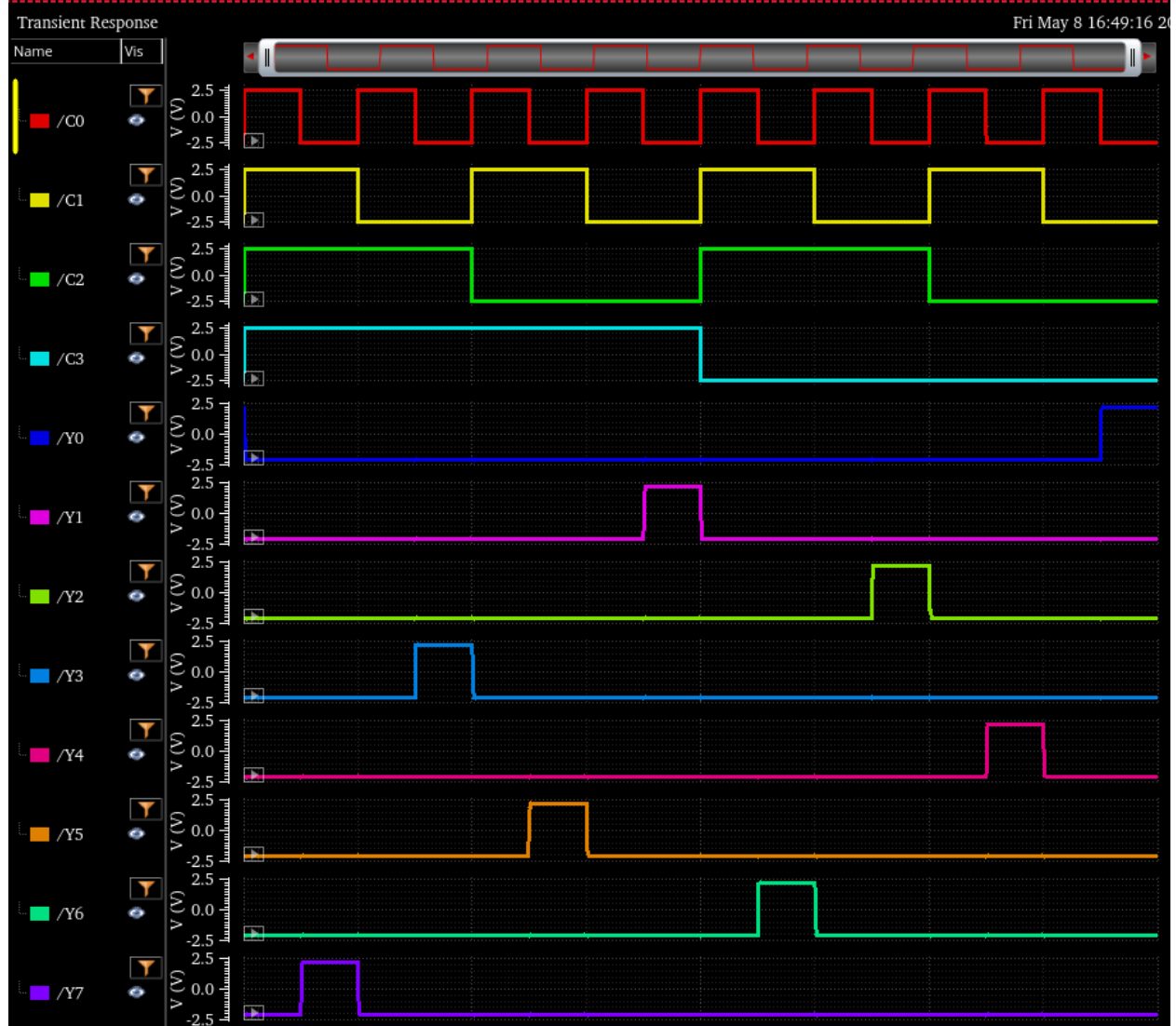
(Figure 29: This is the schematic for 4-to-16 decoder)

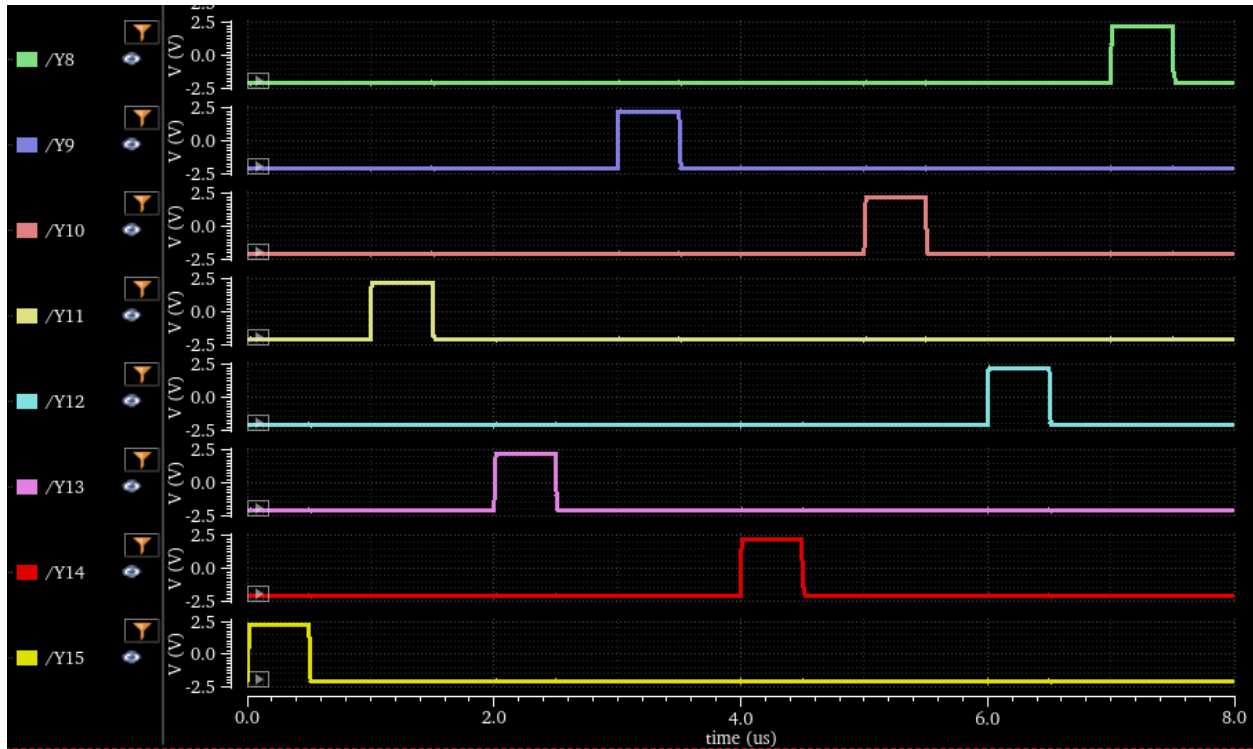
After that, we next built a test bench to see if the logic of our schematic worked as intended as the logic of the 4-to-16 Decoder.



(Figure 30: The test bench circuit for 4-to-16 decoder)

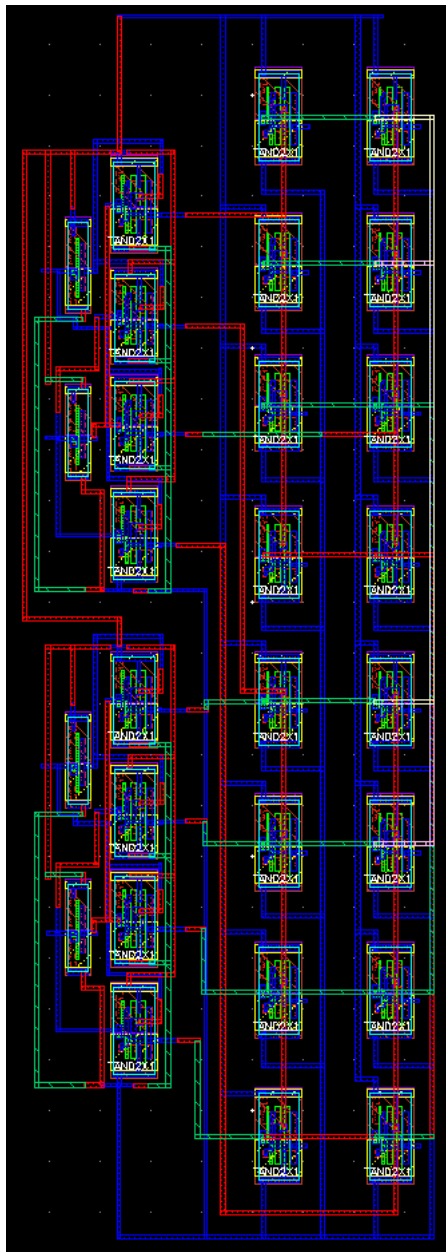
The expectation is pretty straightforward, based on the combination of the input, the corresponding output will be on. There are a lot of combinations, so we did not really realize that we put the input order kinda opposite (instead of A3A2A1A0, we use A0A1A2A3) until test it with the potentiometer. However, we had built the layout at that time and it did not really affect the final result as long as we knew the order so those are what we got below :



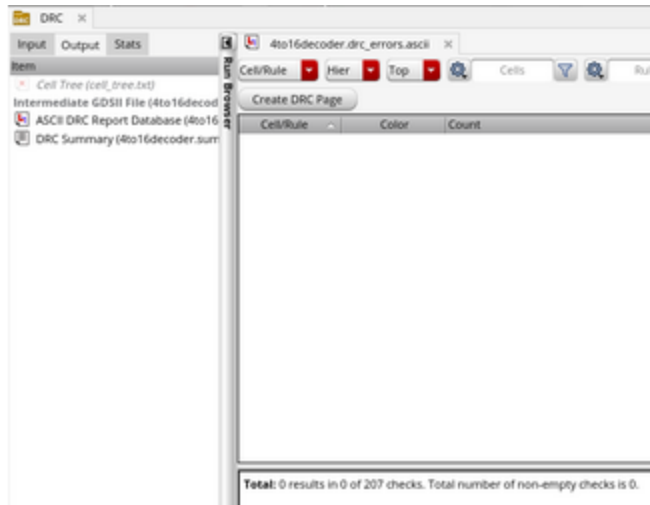


(Figures 31 & 32: These are the result for the test bench circuit for 4-to-16 decoder)

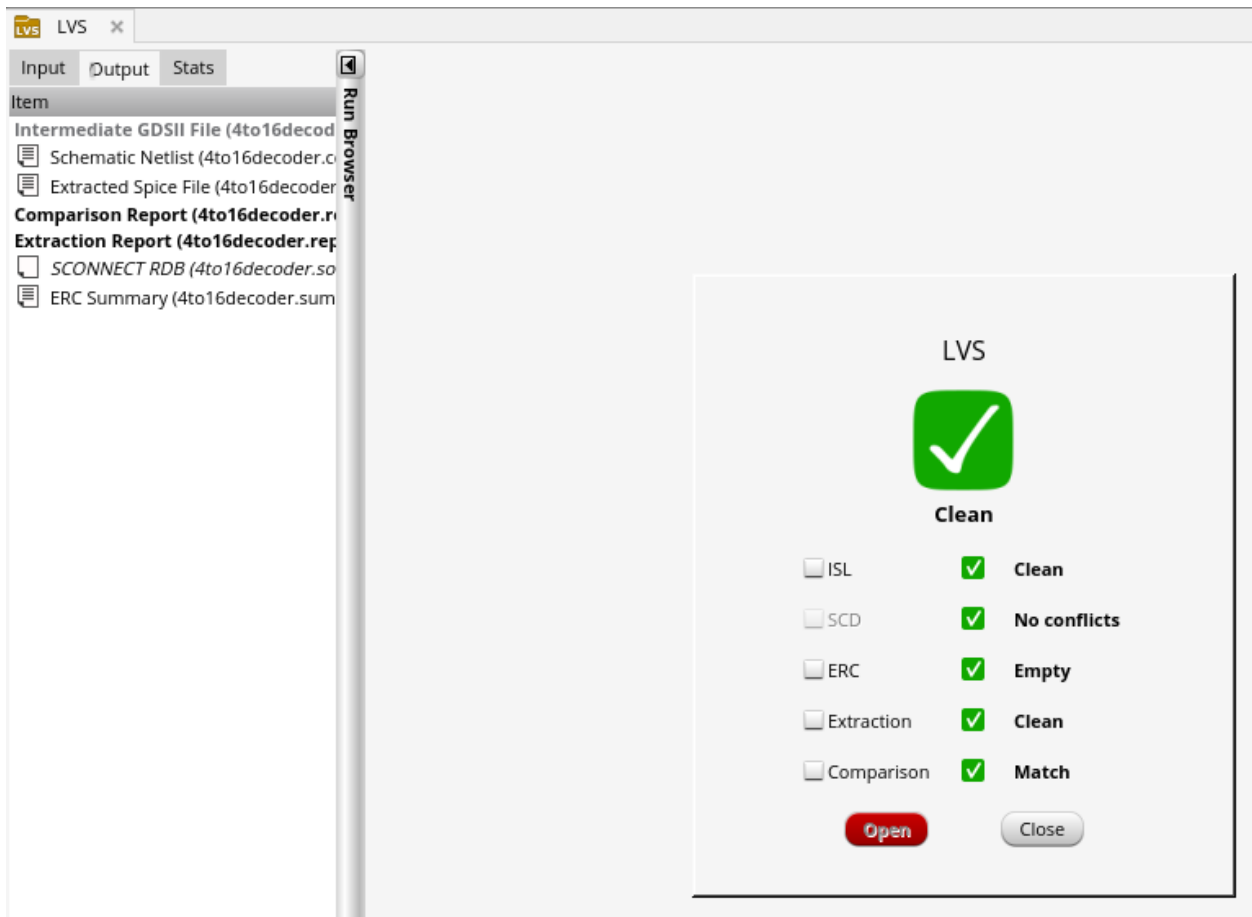
We also built the layout of the 4-to-16 Decoder schematic and ran some checks to see if the layout was valid.



(Figure 33: This is the layout for 4-to-16 decoder)

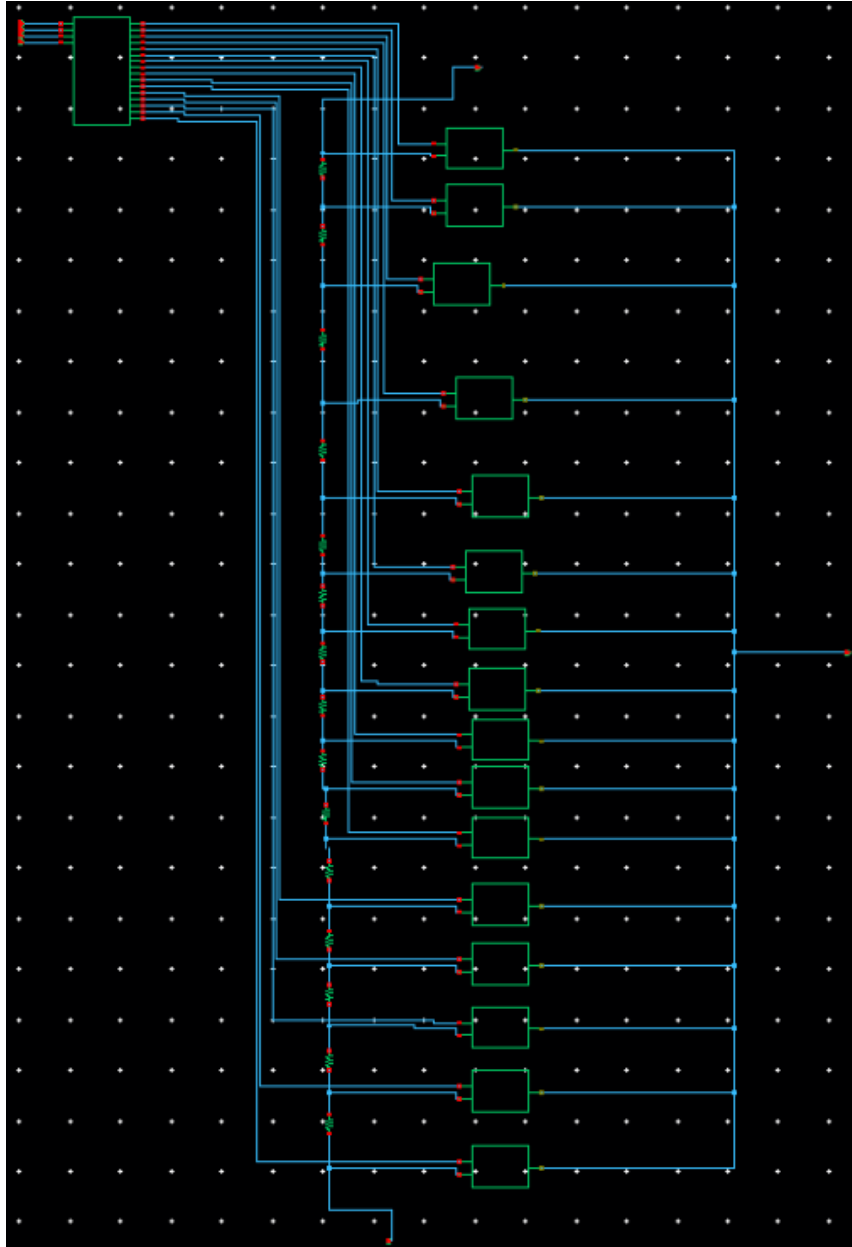


(Figure 34: DRC result for the layout for 4-to-16 decoder)

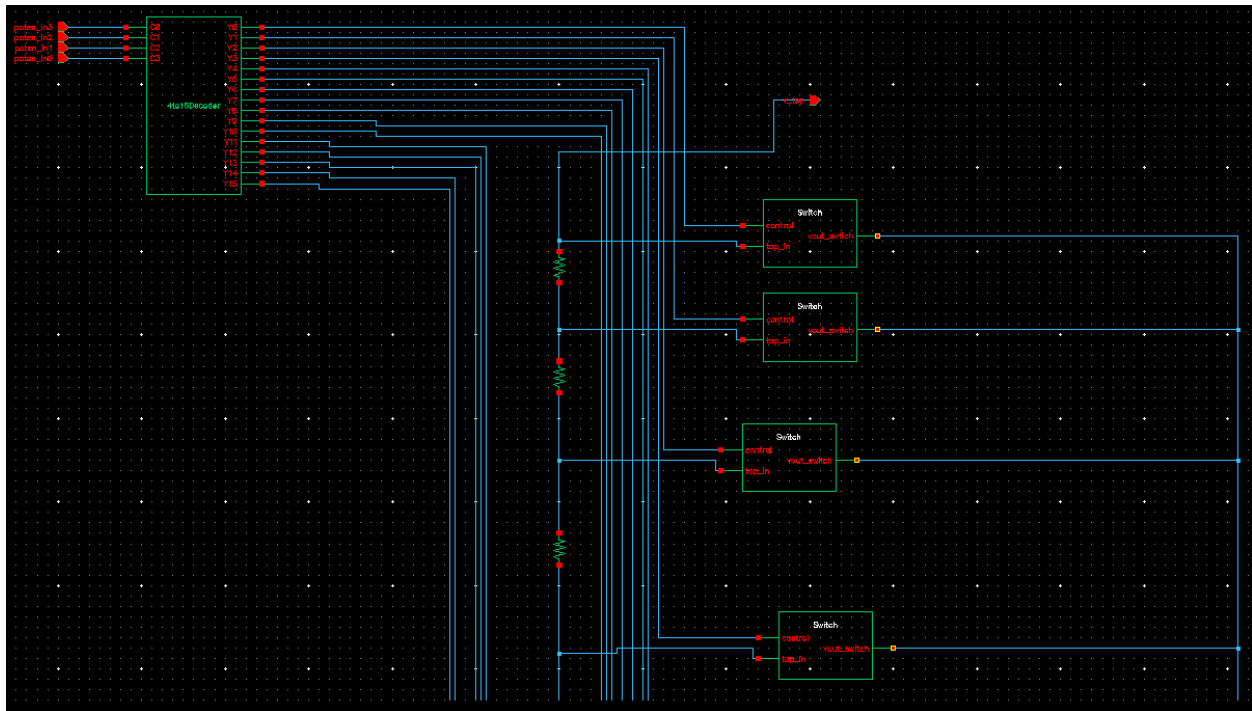


(Figure 34: LVS result for the layout for 4-to-16 decoder)

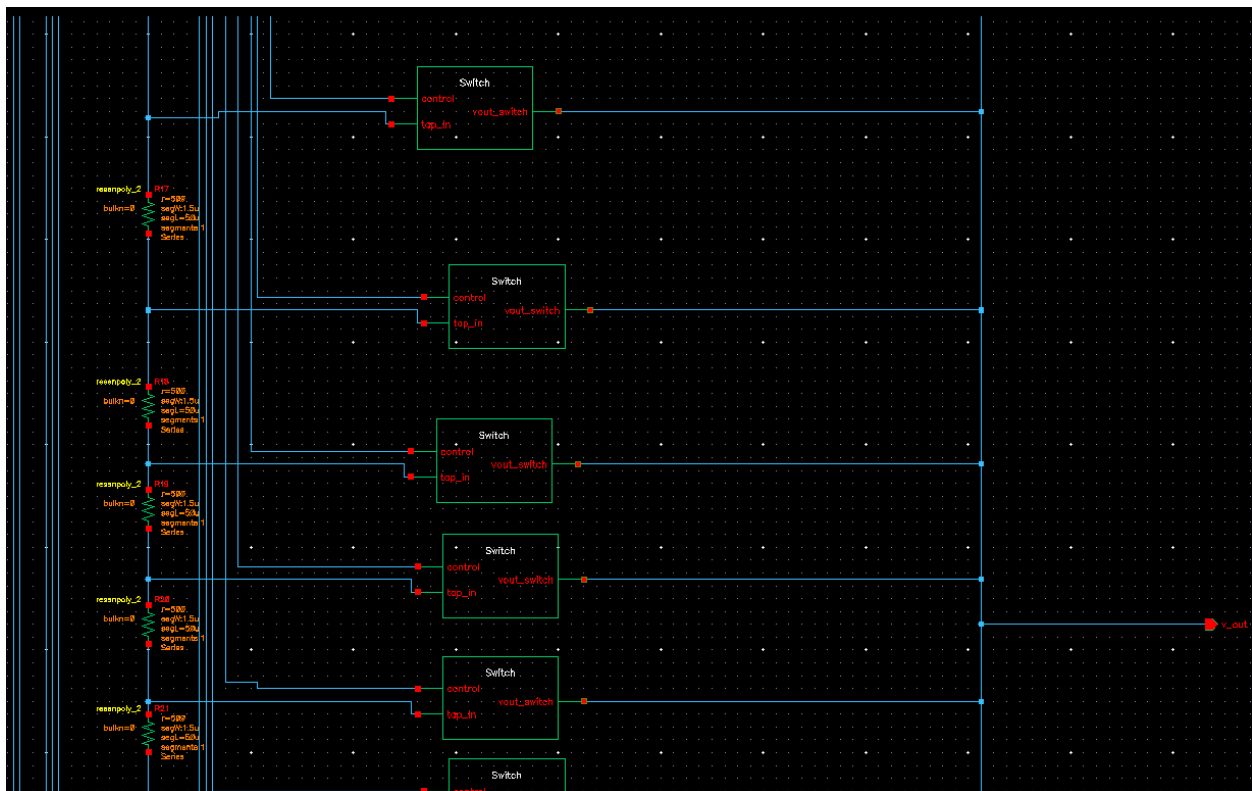
After completing the 4 to 16 decoder, we combined it with the string of resistors and bidirectional switch as I mentioned above to make the potentiometer.



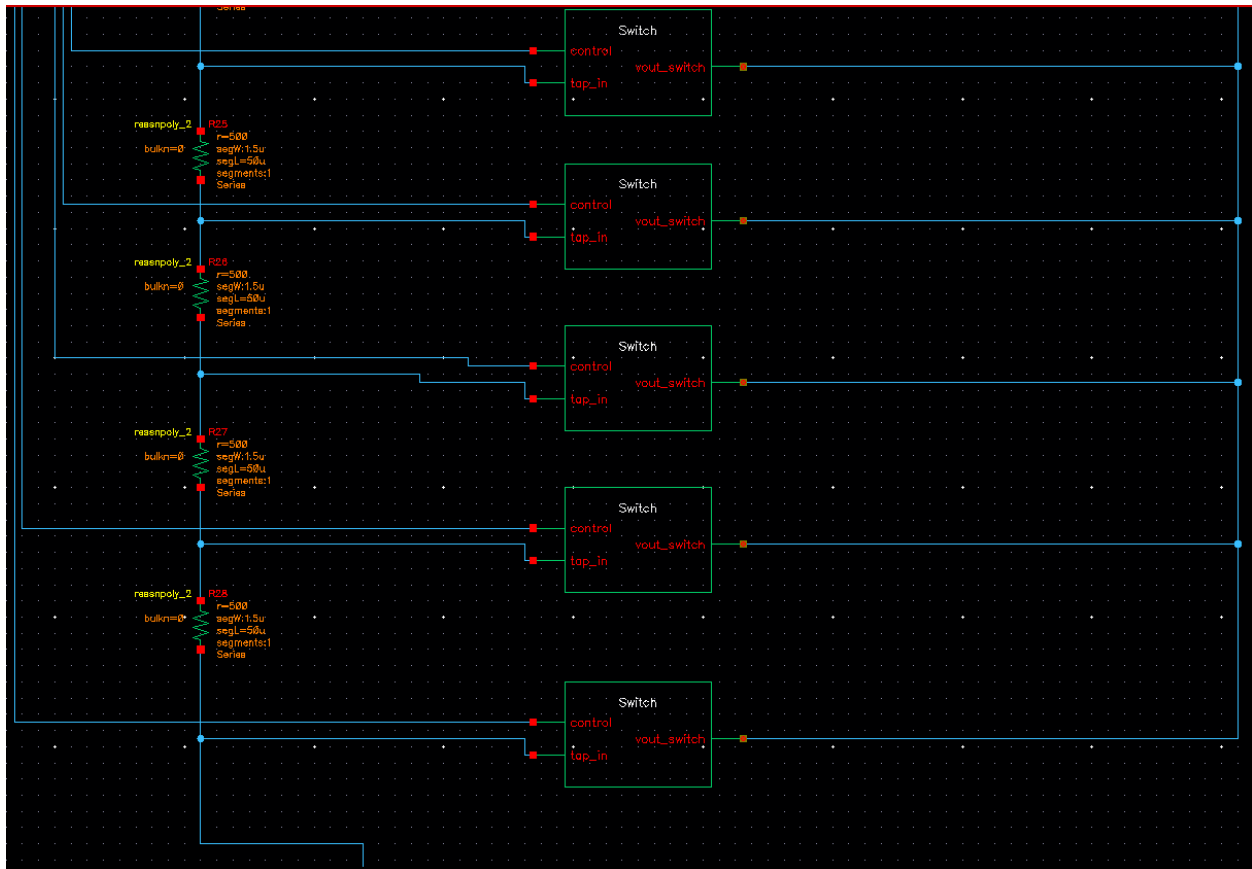
(Figure 35: This is the schematic for Potentiometer)



(Figure 36: Zoomed-In upper part of the schematic)

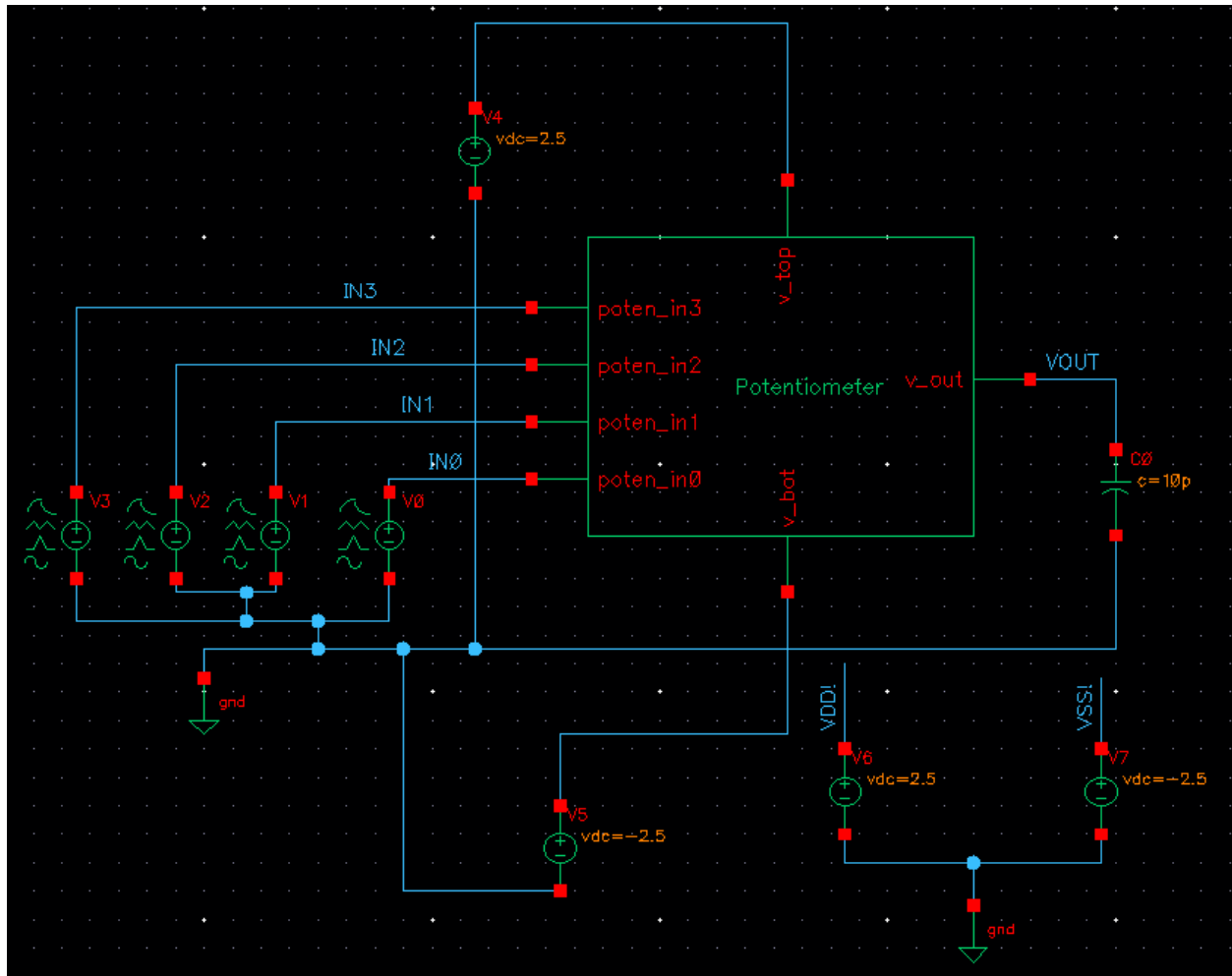


(Figure 37: Zoomed-In middle part of the schematic)



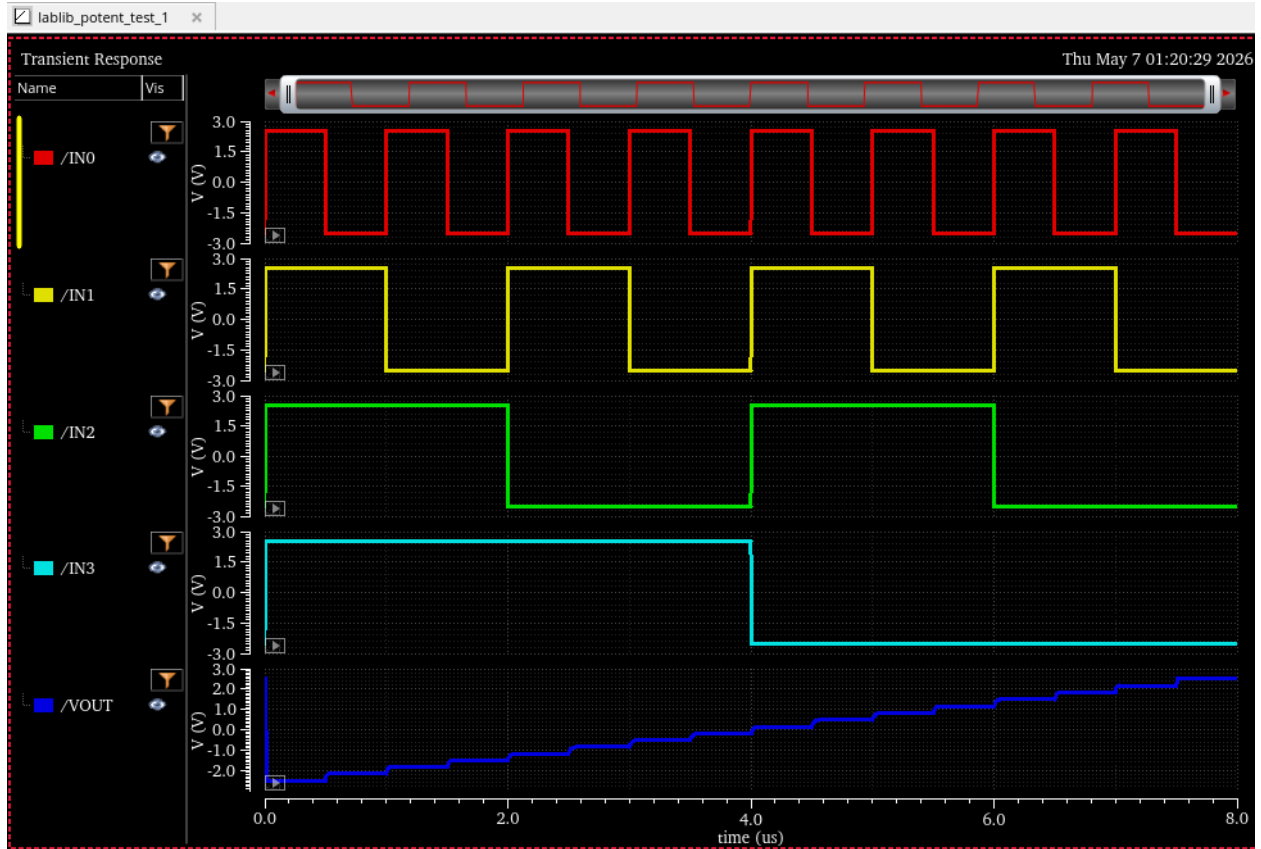
(Figure 38: Zoomed-In lower part of the schematic)

For the testbench, we had 4 square waves with the same amplitude and different periods—each one was twice as much as the previous one, so each possible scenario was allowed—for the control lines. We then Had VTOP be 2.5 V and VBOT be -2.5 as those were the farthest values required of the potentiometer. This schematic is shown below.



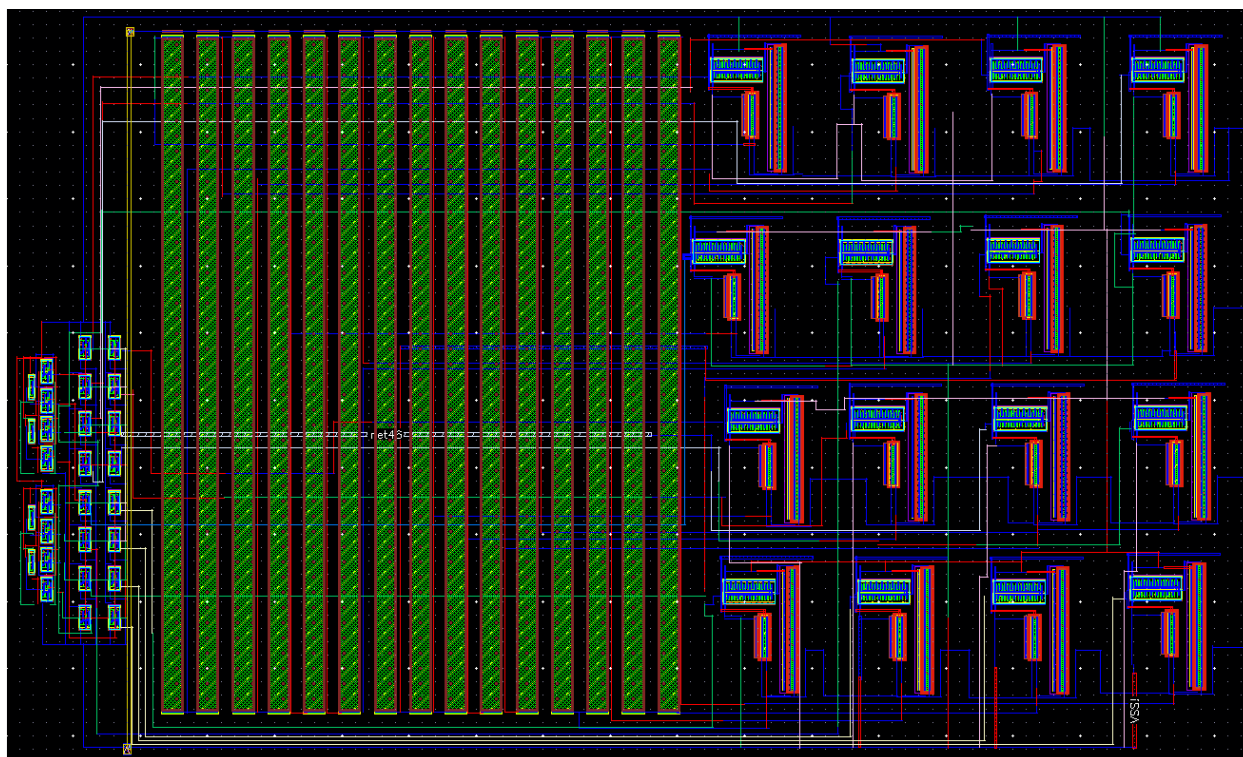
(Figure 39: This is the test bench circuit for Potentiometer)

What we expected from the result would be an upward ladder at Vout , since the lower the input is (lower here means 0000 < 1111 sth like that) , the voltage should be closer to 2.5 V which should be higher than the previous step.

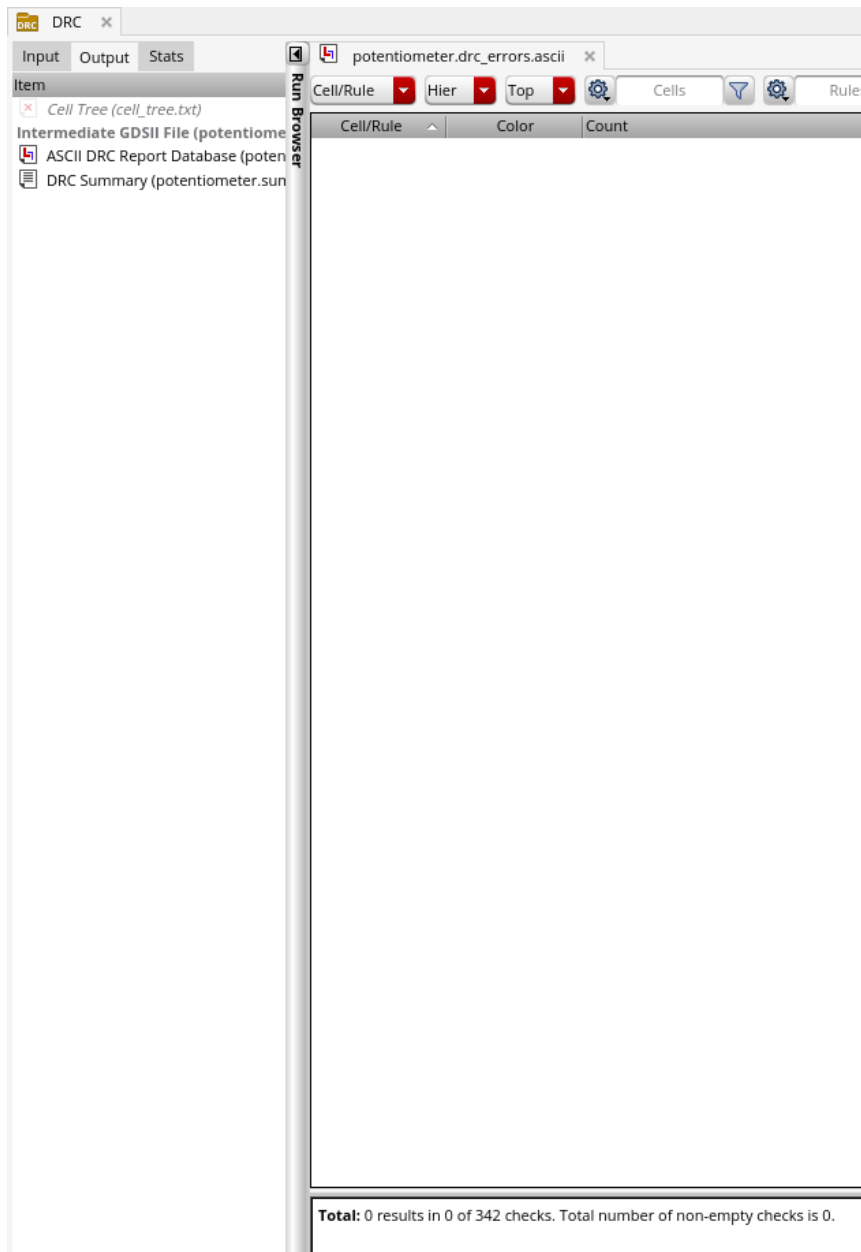


(Figure 40: This is the result for the test bench circuit for Potentiometer)

We got a voltage ladder in the range from -2.5 V to 2.5 V which matched our expectation so we moved on to complete the layout



(Figure 41: This is the layout for Potentiometer)



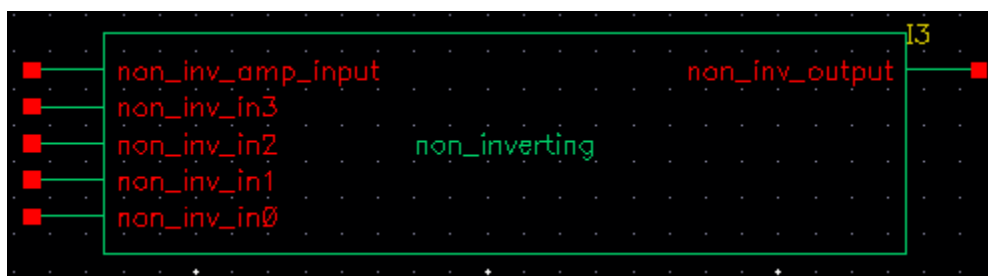
(Figure 42: This is the DRC result for the layout for Potentiometer)



(Figure 43: This is the LVS result for the layout for Potentiometer)

Programmable Non-Inverting Amplifier

Once we finished building the Digital Potentiometer, we moved to the Programmable Non-Inverting Amplifier.



(Figure 44: This is the symbol block for the Non-Inverting Amp)

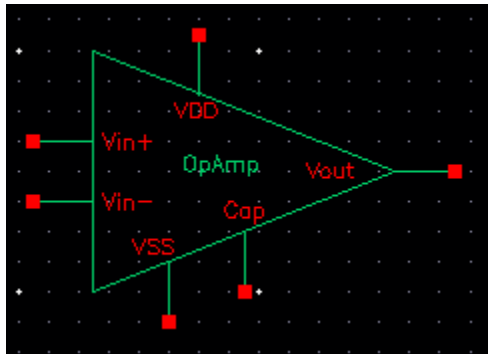
Operational Amplifier

Firstly, to get the amplifier configuration, we must build an opamp

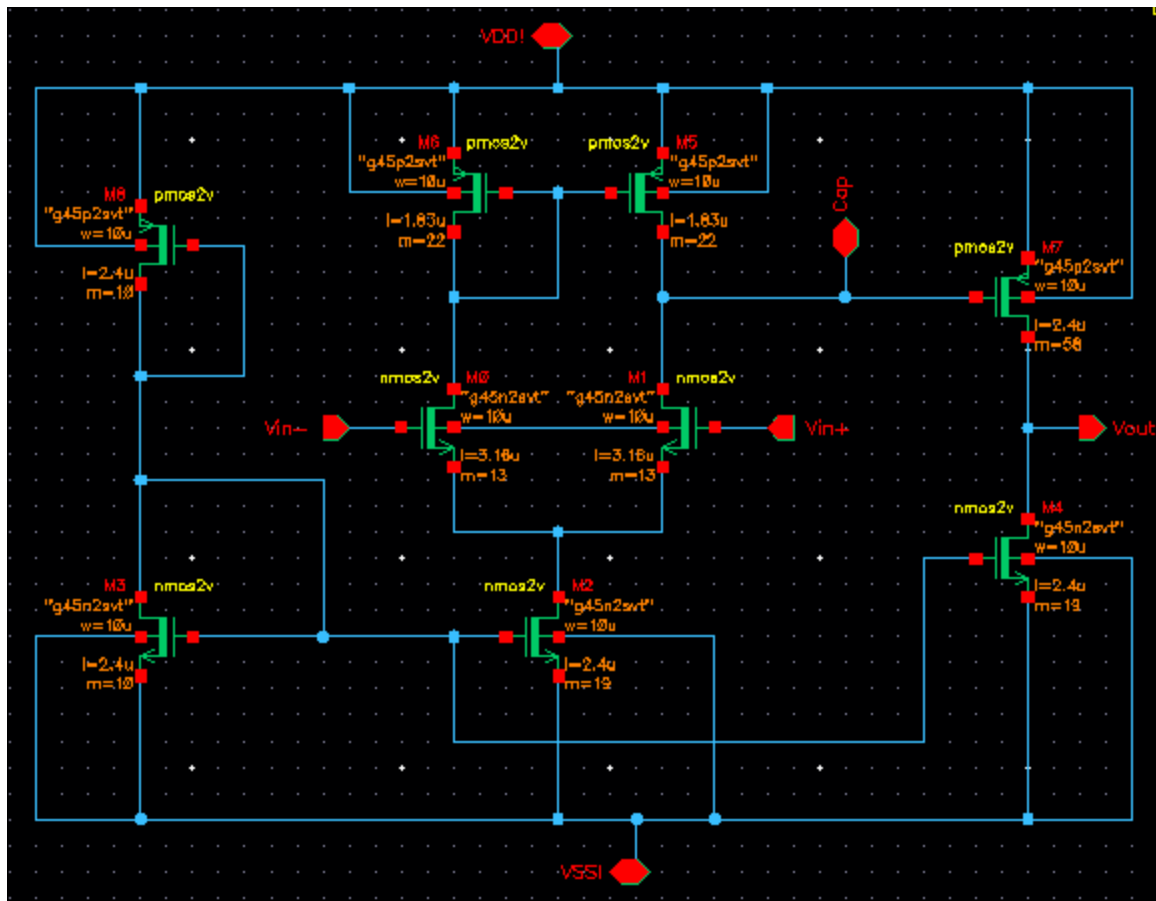
We used the two-stage externally compensated op-amp file provided to create our op amp architecture. During our op amp design, we used the spreadsheet to get the Ucox for our MOSFETs and combine it with some formula from the

instruction sheets to build our opamp. I think the task is achievable but we have limited of time (exams and stuff) and professor Gegier does not really talk about this too much in the lecture , so we use a bit of help from ChatGPT + other peers to fasten the calculation (there are tons of other things to do with the opamp so we really need it to be done fast to get other things done.)

After a few times changing the sizing , we finally achived the sizing as below:



(Figure 45: This is the symbol block for Op Amp)

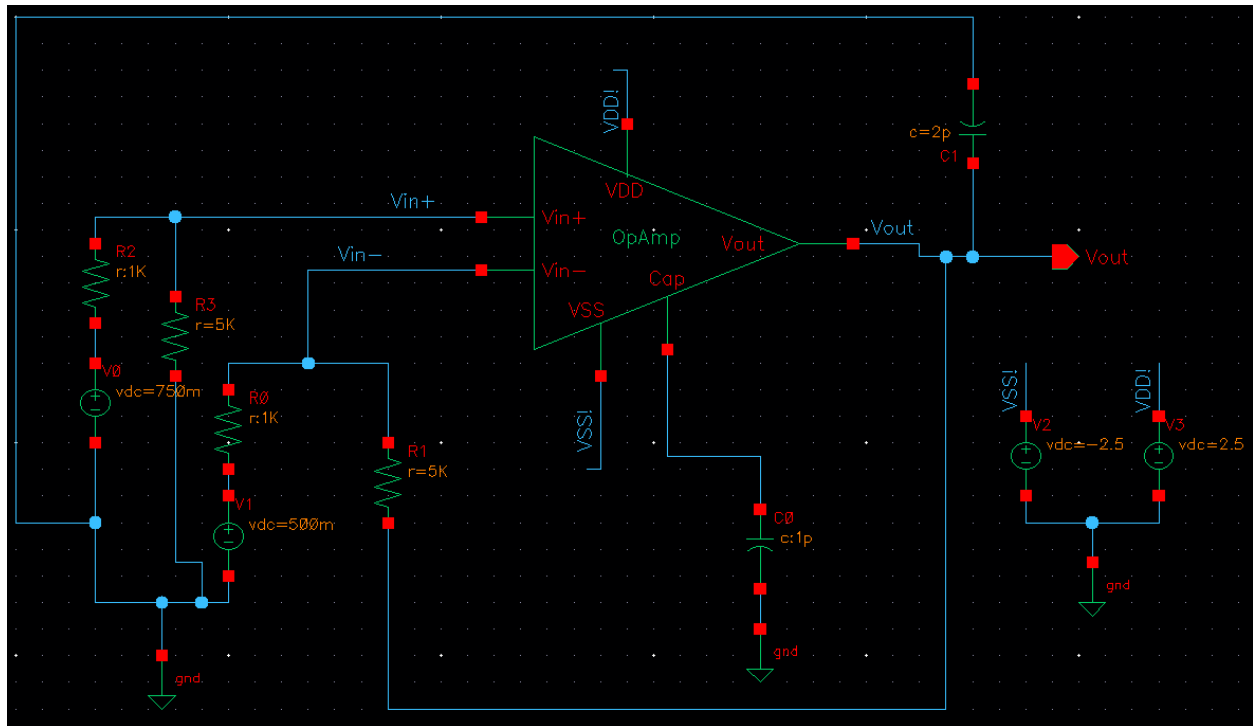


(Figure 46: This is the schematic for Op Amp)

After that, we next built a test bench to see if the logic of our schematic worked as intended as the logic of the op amp.

For the testbench, we created a simple noninverting op-amp to ensure that the op-amp worked correctly and that all transistors were in saturation during operation. With the test bench, we were able to see through a transient sweep that we were getting the desired gain based on our R_{in} and R_f ratio.

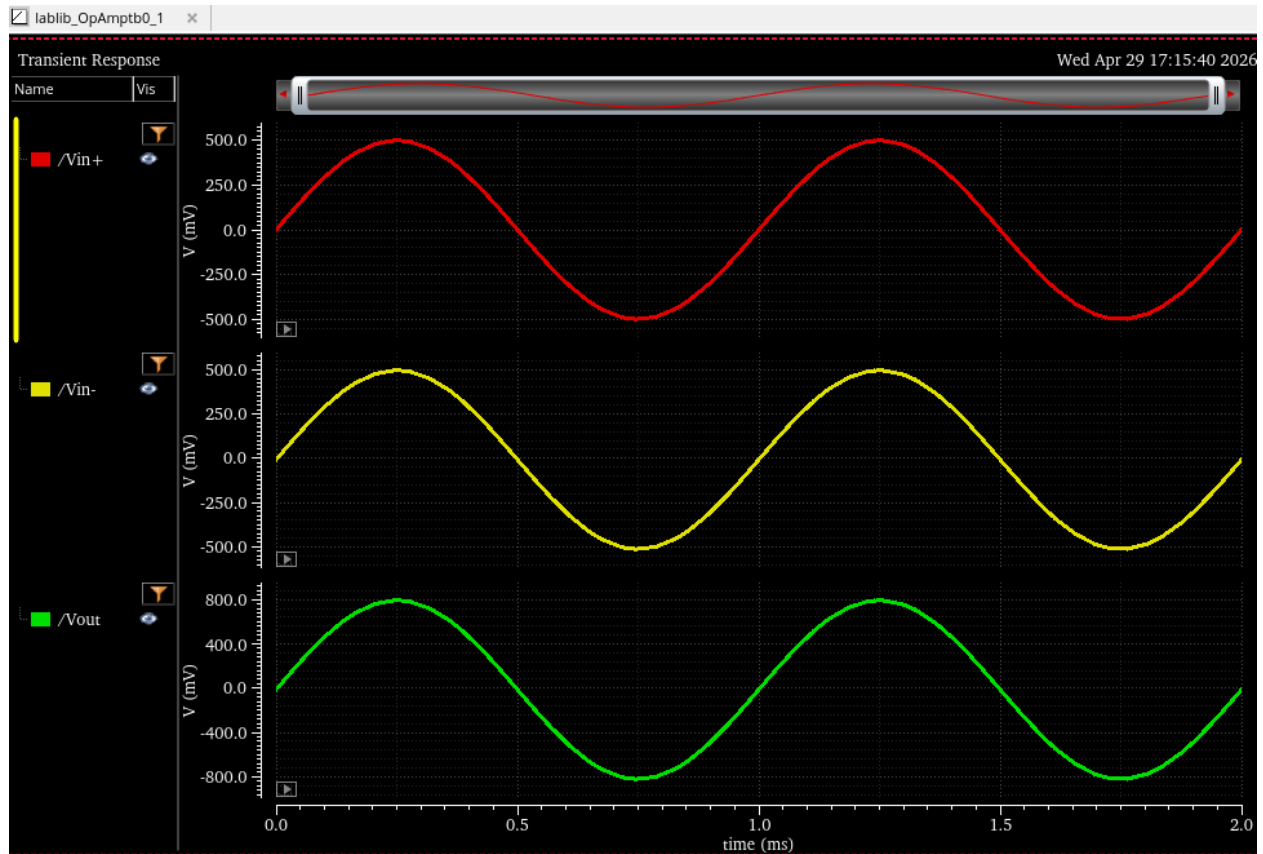
$$\text{Gain} = 1 + R_f / R_{in} = 1 + 6k / 10k = 1.6 \text{ V/V}$$



(Figure 47: This is the test bench circuit for Op Amp)

Based on the result below, we concluded that the Op Amp worked as intended.

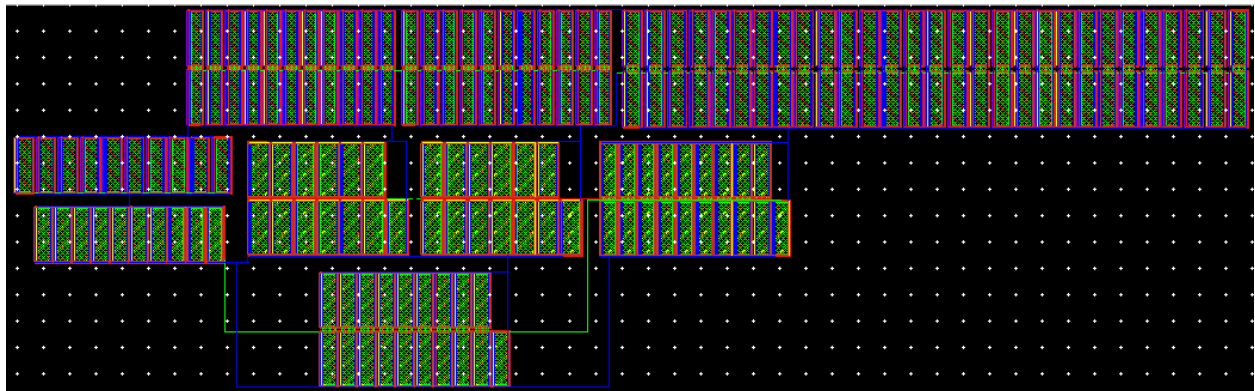
$$\text{Gain} = V_{\text{out}} / V_{\text{in}} = 800\text{mV} / 500\text{mV} = 1.6 \text{ V/V}$$



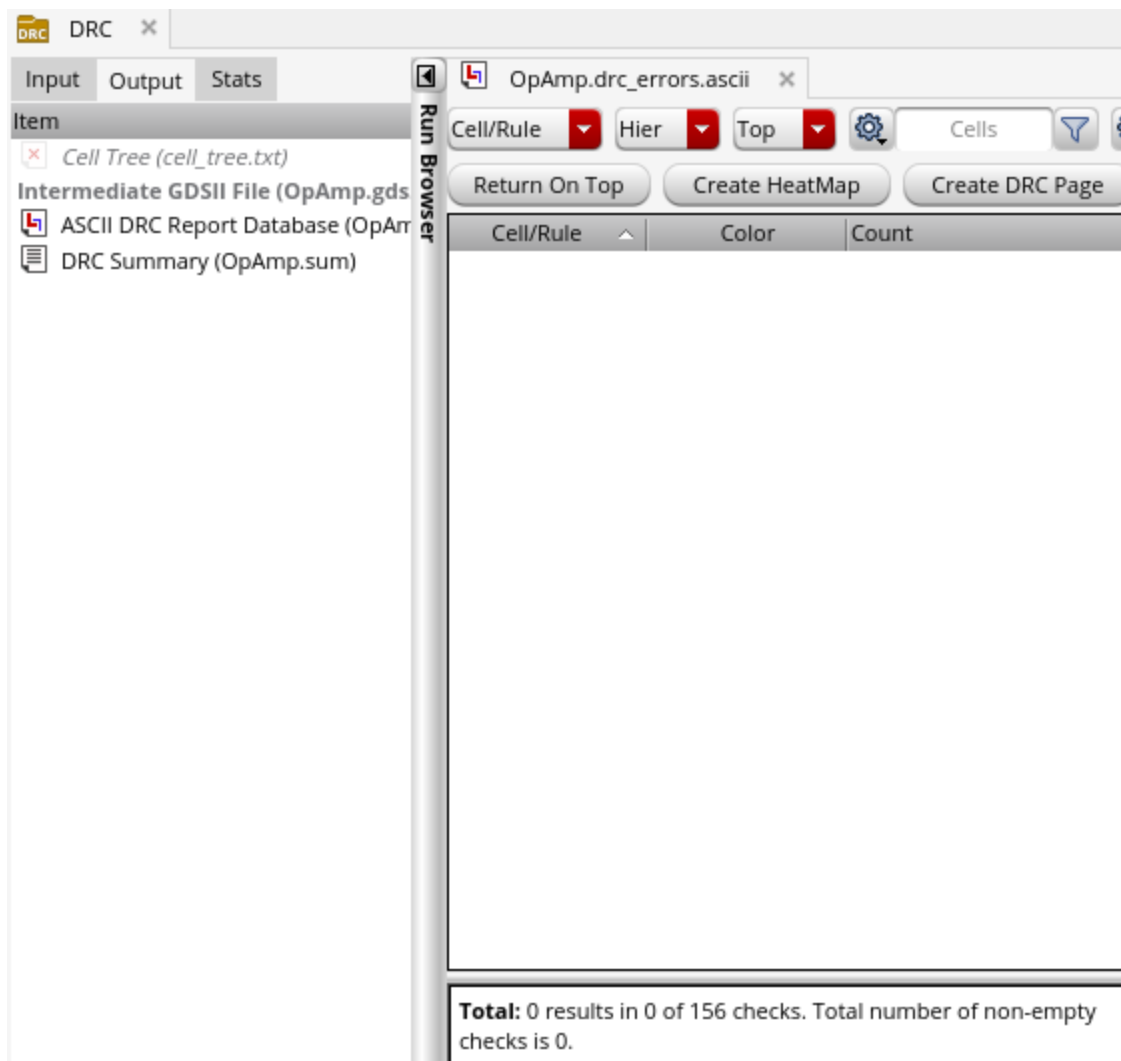
(Figure 48: This is the result for the test bench circuit for Op Amp)

The result matches what we expected so we thought our opamp functioned well.

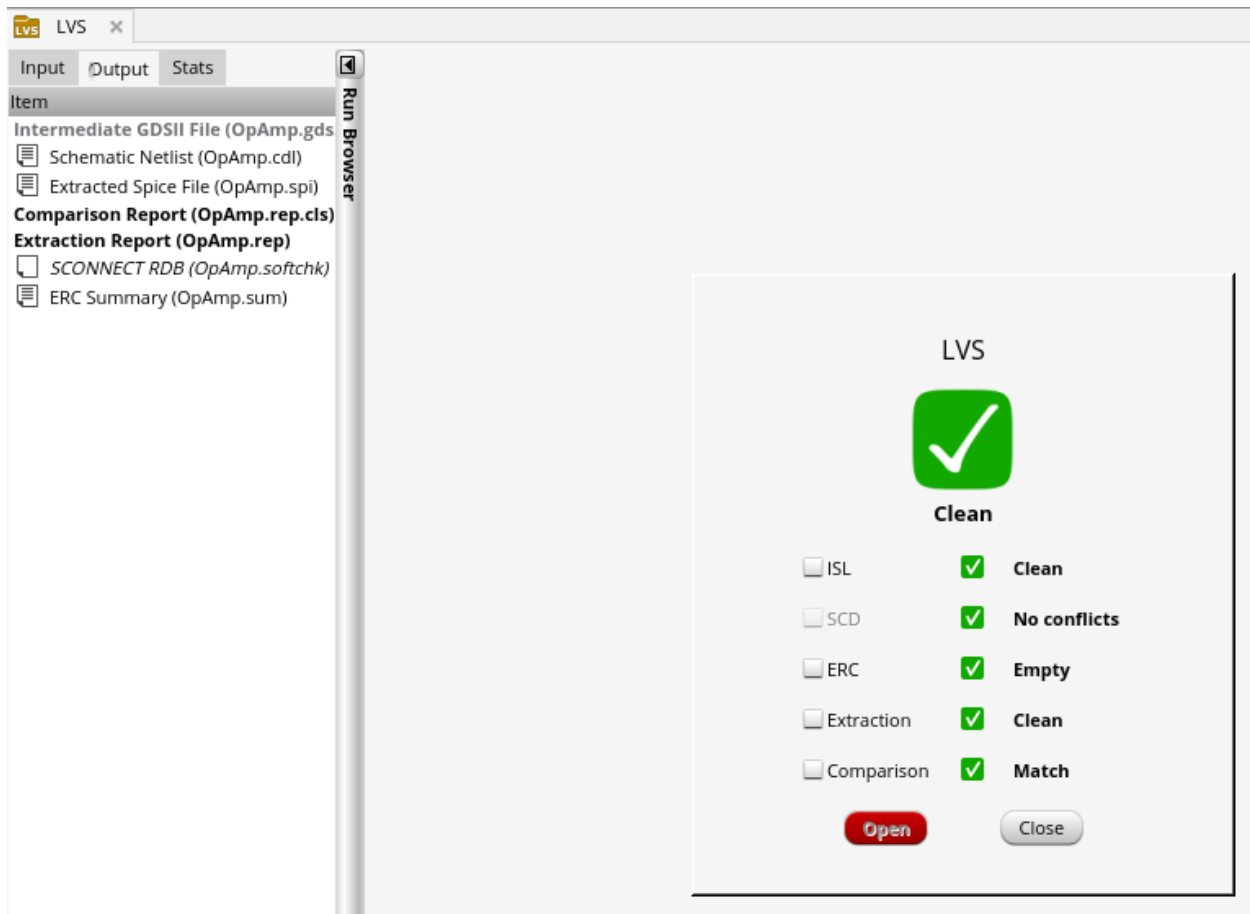
We also built the layout of the op amp schematic and ran some checks to see if the layout was valid.



(Figure 49: This is the layout for Op Amp)



(Figure 50: This is the DRC result for the layout for Op Amp)



(Figure 51: This is the LVS result for the layout for Op Amp)

Once we finished building the Basic 2-stage Operational Amplifier, we moved to build the Programmable Non-Inverting Amplifier.

Firstly , it was just a basic non-inverting amplifier but to make it programmable we must use the potentiometer as R_f so that we can control the gain of our circuit. (which is $1 + R_f/R_{in}$)

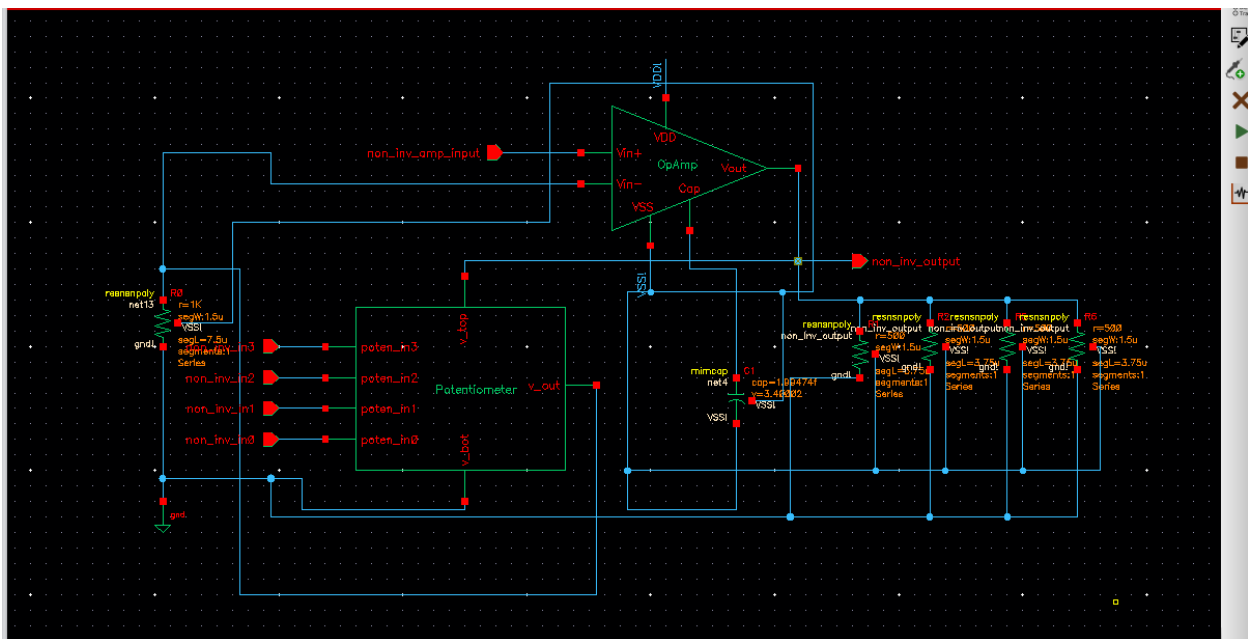
Additionally , since the working voltage range of our circuit is just from 2.5V to -2.5 V (which is not really large), we cannot set the gain to high since it will make the waveform look ugly (clipped), so we decided to use a reasonably large value of R_{in} which is 1K (although it still made the output waveform clipped when the the resistance is max).

Additionally , our op-amps are externally compensated opamp which requires large value capacitance to compensate which is not offered in gpdk45 library

(they actually have mimcap but we have to use multiplier like 100000 to get the value of 50pF and we actually are not able to build the layout for it – literally impossible, it made cadence crash everytime I treid to do it).

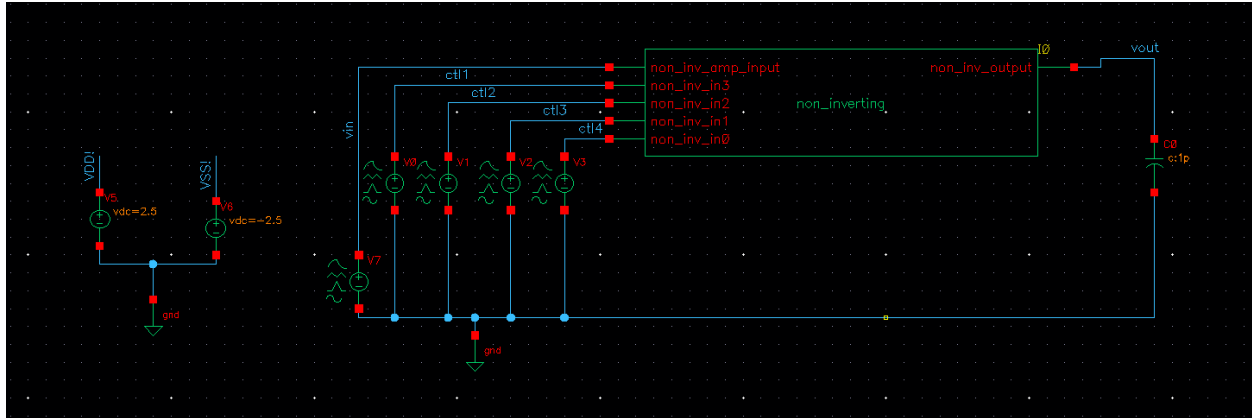
Therefore ,we came up with a solution that we should include a system of 4 parallel resistors at the output voltage node, so that they can steal some current from the feedback loop.Additioanlly , adding resistors from output to ground stabilizes the circuit because it provides resistive loading and damping, which shifts the output pole and improves phase margin.

This solution although can impact the gain of our circuit a bit , but because our circuit range is not quite large (only 2.5 to -2.5 V) , this does not strongly impact our efficiency that much



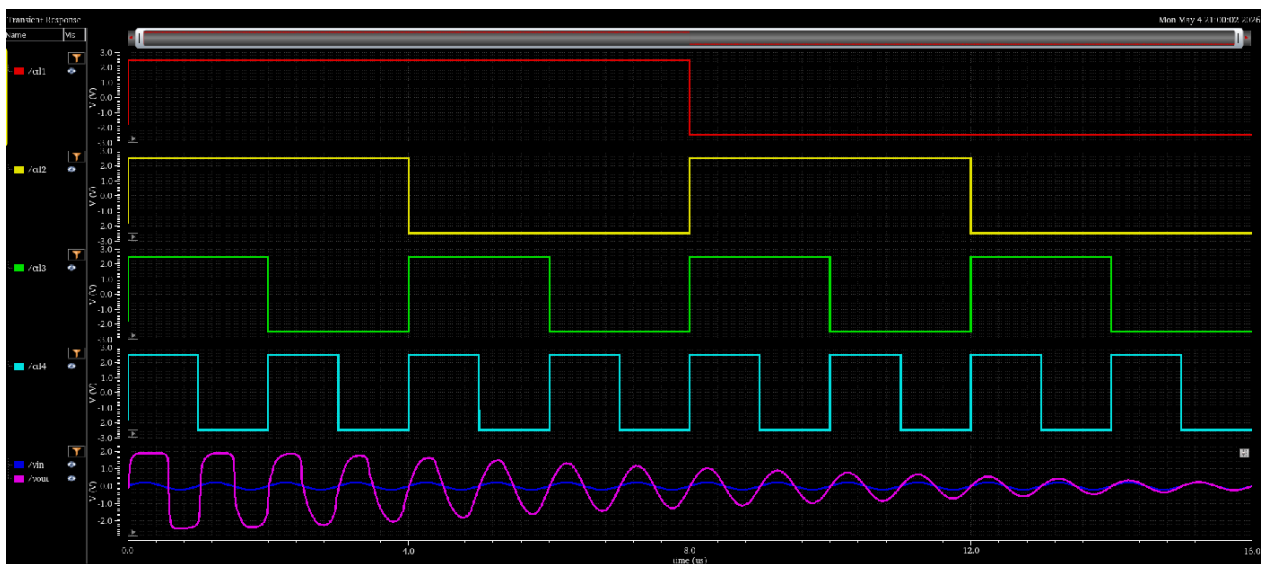
(Figure 52: schematic for Non-Inverting Amp)

We also made a testbench for the circuit to make sure it work:



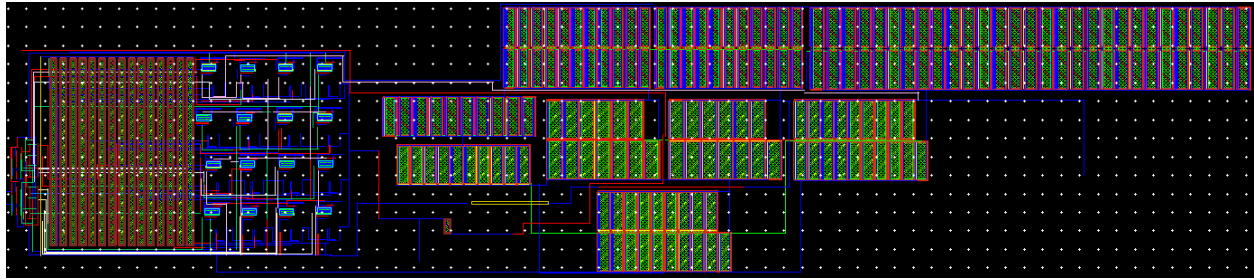
(Figure 53: TB for Non-Inverting Amp)

As we use the potentiometer as R_f , we expected the gain to increase as the input is increasing. This is because when input increases, the resistor line is longer $\rightarrow$ more resistance $\rightarrow$ gain $= 1 + R_f/R_{in}$ will be larger



(Figure 54: This is the result for the test bench circuit for Non-Inverting Amp)

The waveform does follow the behavior I mentioned above, so we moved on to build the layout

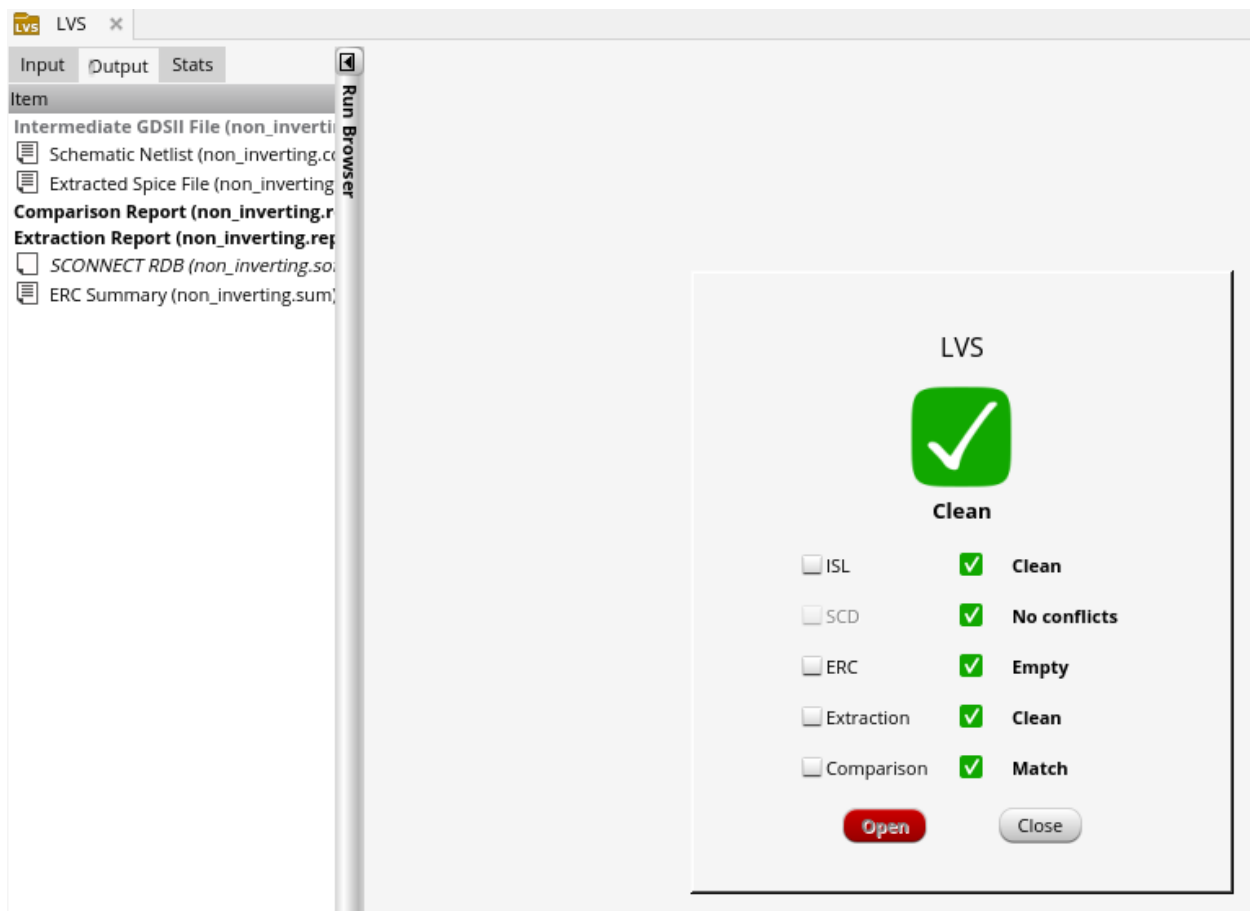


(Figure 55: This is the layout for Non-Inverting Amp)

The screenshot displays the DRC (Design Rule Check) results window. The window has a tabbed interface with 'DRC' selected. The 'Input' tab is active, showing a list of items: 'Cell Tree (cell_tree.txt)', 'Intermediate GDSII File (non_inverting.gds)', 'ASCII DRC Report Database (non_inverting.drc_errors.ascii)', and 'DRC Summary (non_inverting.summary)'. The 'Output' tab is also visible. The 'Stats' tab shows the results of the DRC check. The 'Run Browser' panel on the right side of the window is open, displaying a table with columns 'Cell/Rule', 'Color', and 'Count'. The table is currently empty. Below the table, a summary message states: 'Total: 0 results in 0 of 374 checks. Total number of non-empty checks is 0.'

Cell/Rule	Color	Count
Total: 0 results in 0 of 374 checks. Total number of non-empty checks is 0.		

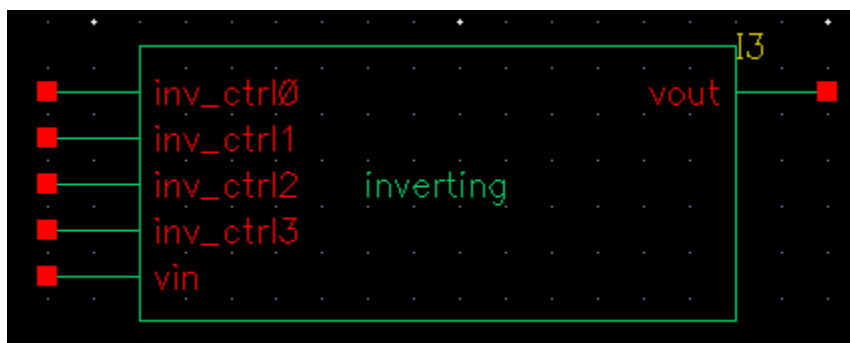
(Figure 56: This is the DRC result for the layout for Non-Inverting Amp)



(Figure 57: This is the LVS result for the layout for Non-Inverting Amp)

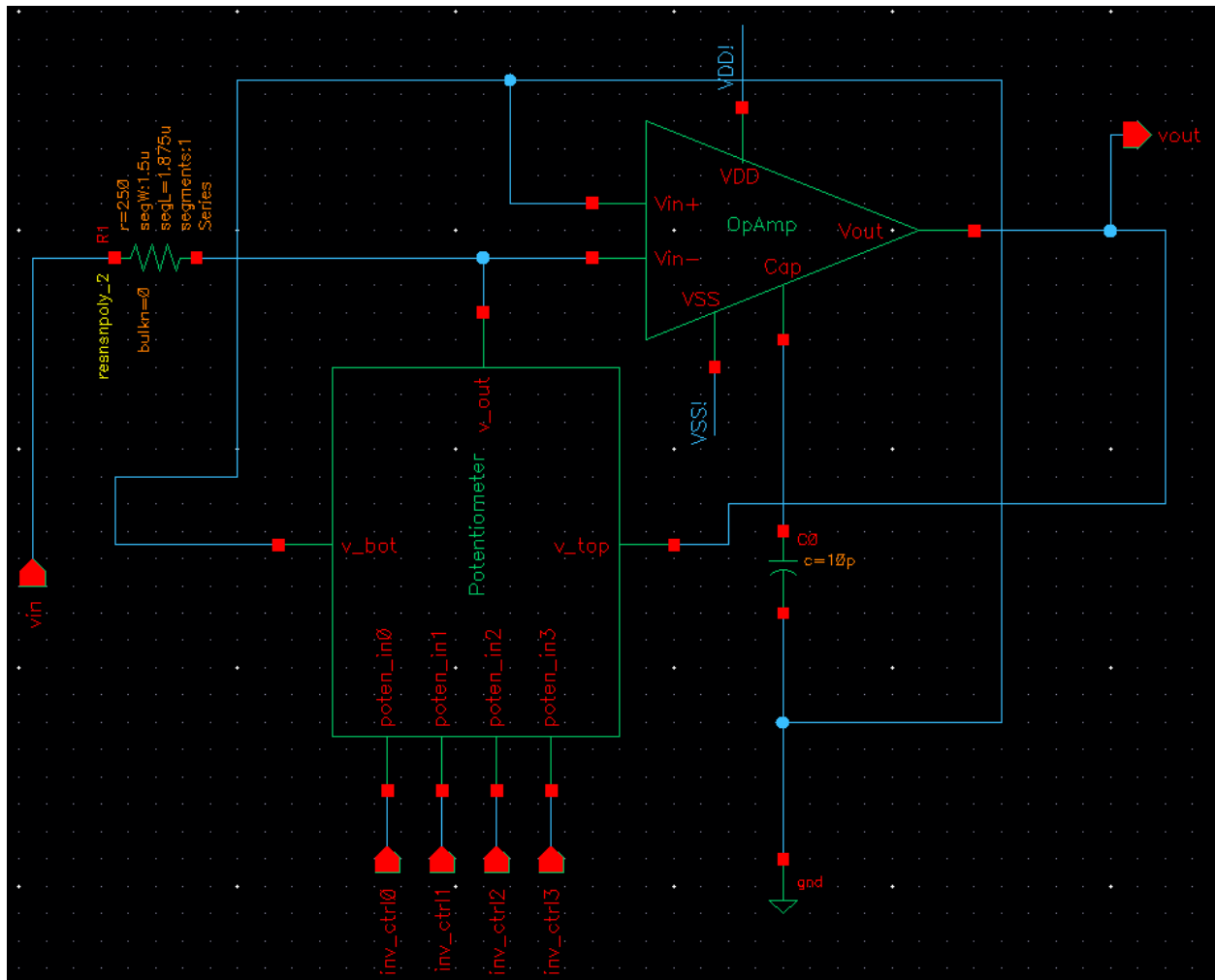
Programmable Inverting Amplifier

Once we finished building the Programmable Non-Inverting Amplifier, we moved to the Programmable Inverting Amplifier.



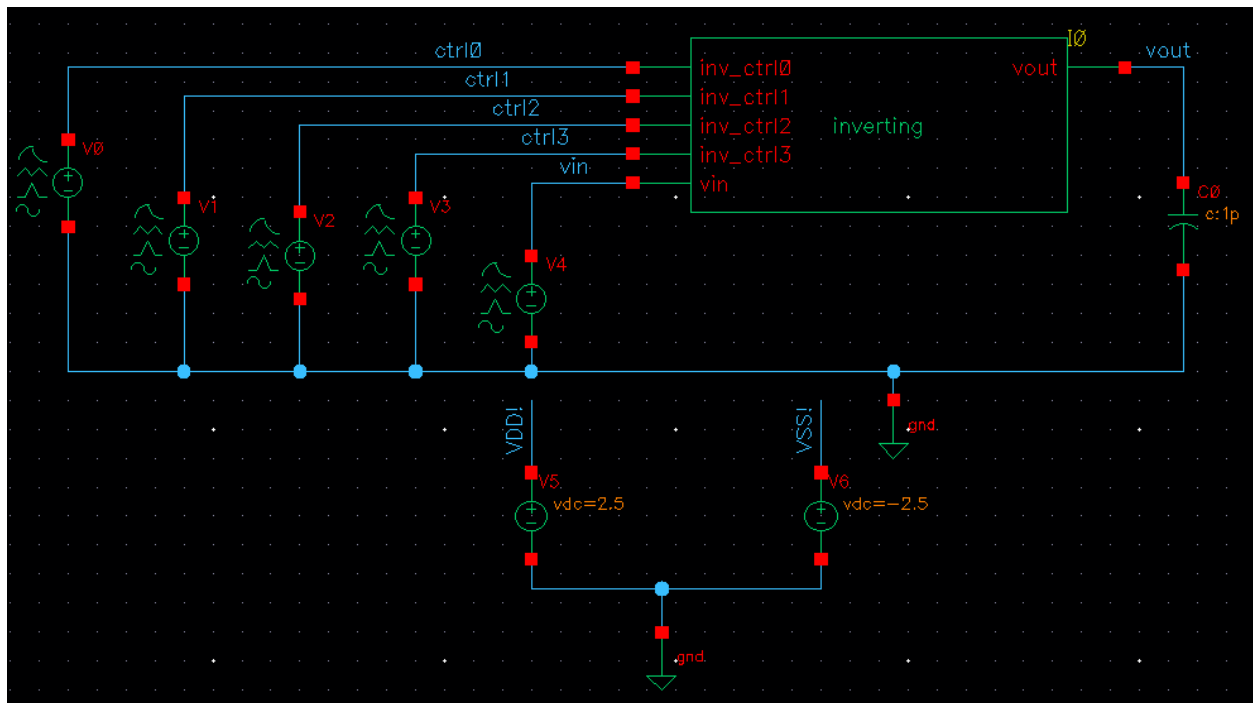
(Figure 58: This is the symbol block for the Inverting Amp)

The circuit is quite similar to a non-inverting opamp config. However, for the inverting opamp, fortunately we don't need to worry about stability of the circuit that much since inverting configuration has way less gain noise at high frequencies compared to non-inverting configuration.



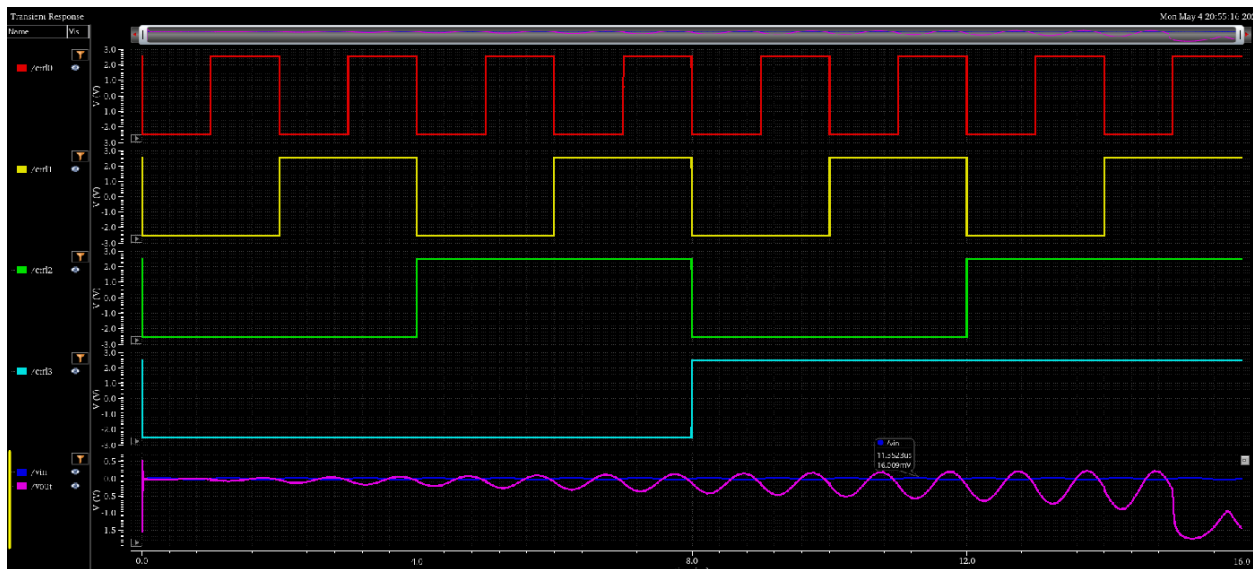
(Figure 59: This is the schematic for Inverting Amp)

We then build a testbench to make sure the circuit work



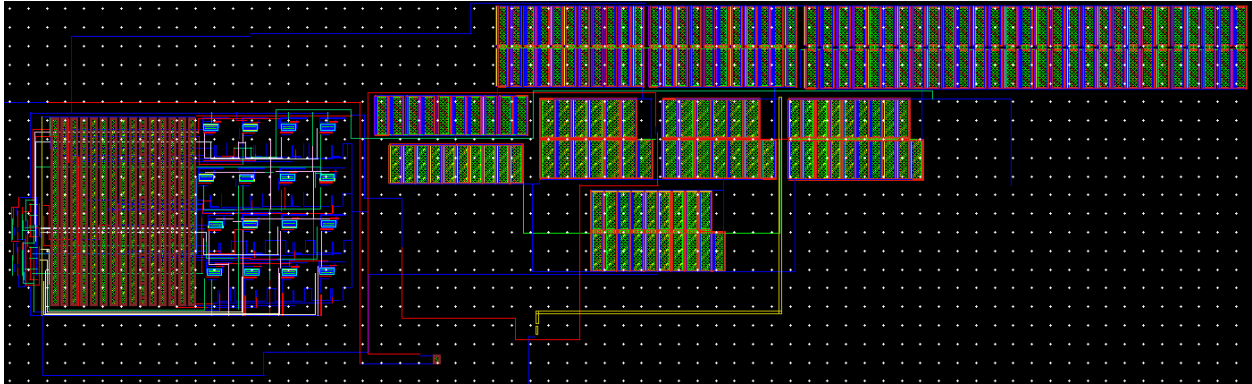
(Figure 60: This is the test bench circuit for Inverting Amp)

Based on the result below, we concluded that the inverting amp worked as intended.

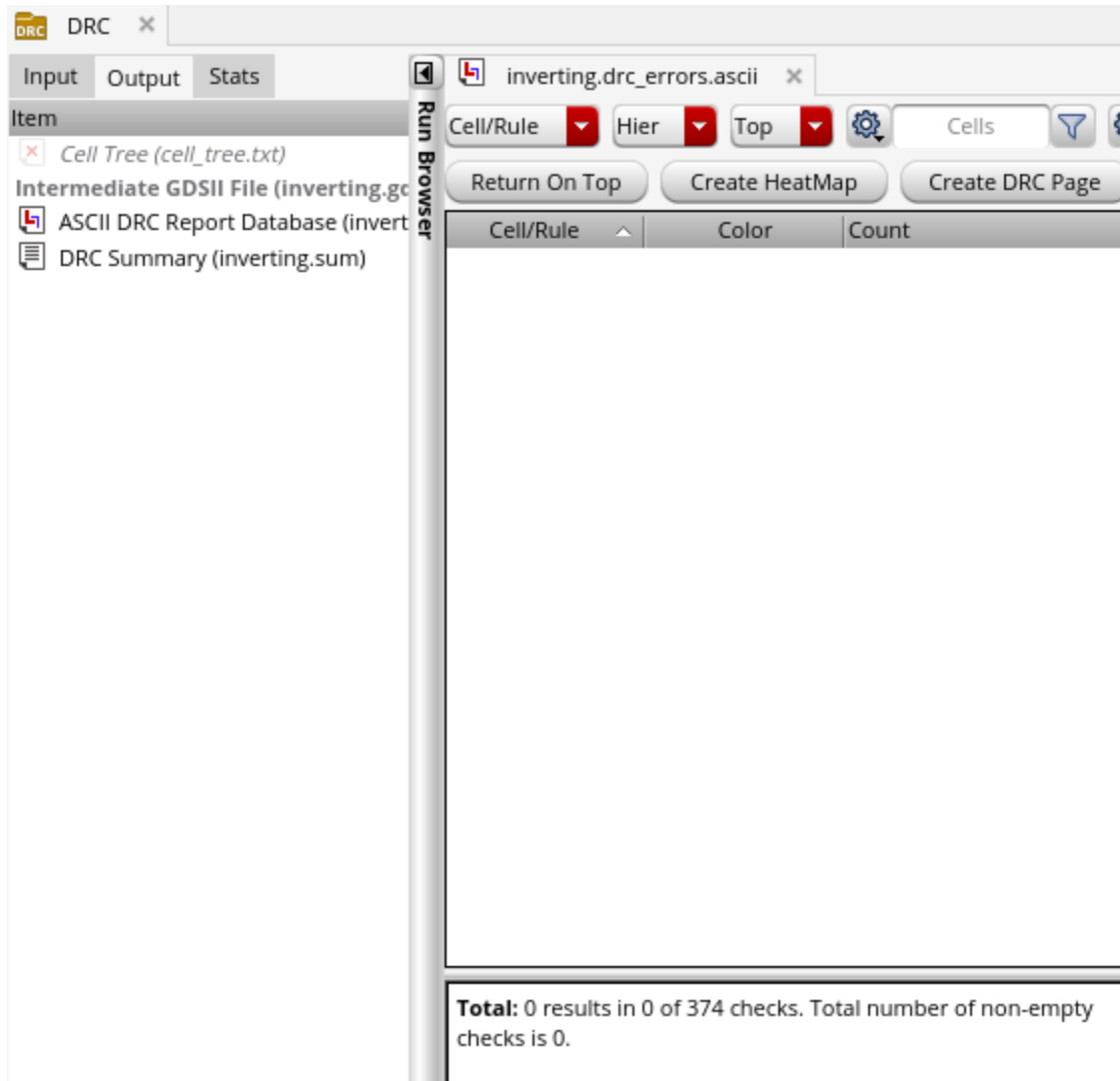


(Figure 61: This is the result for the test bench circuit for Inverting Amp)

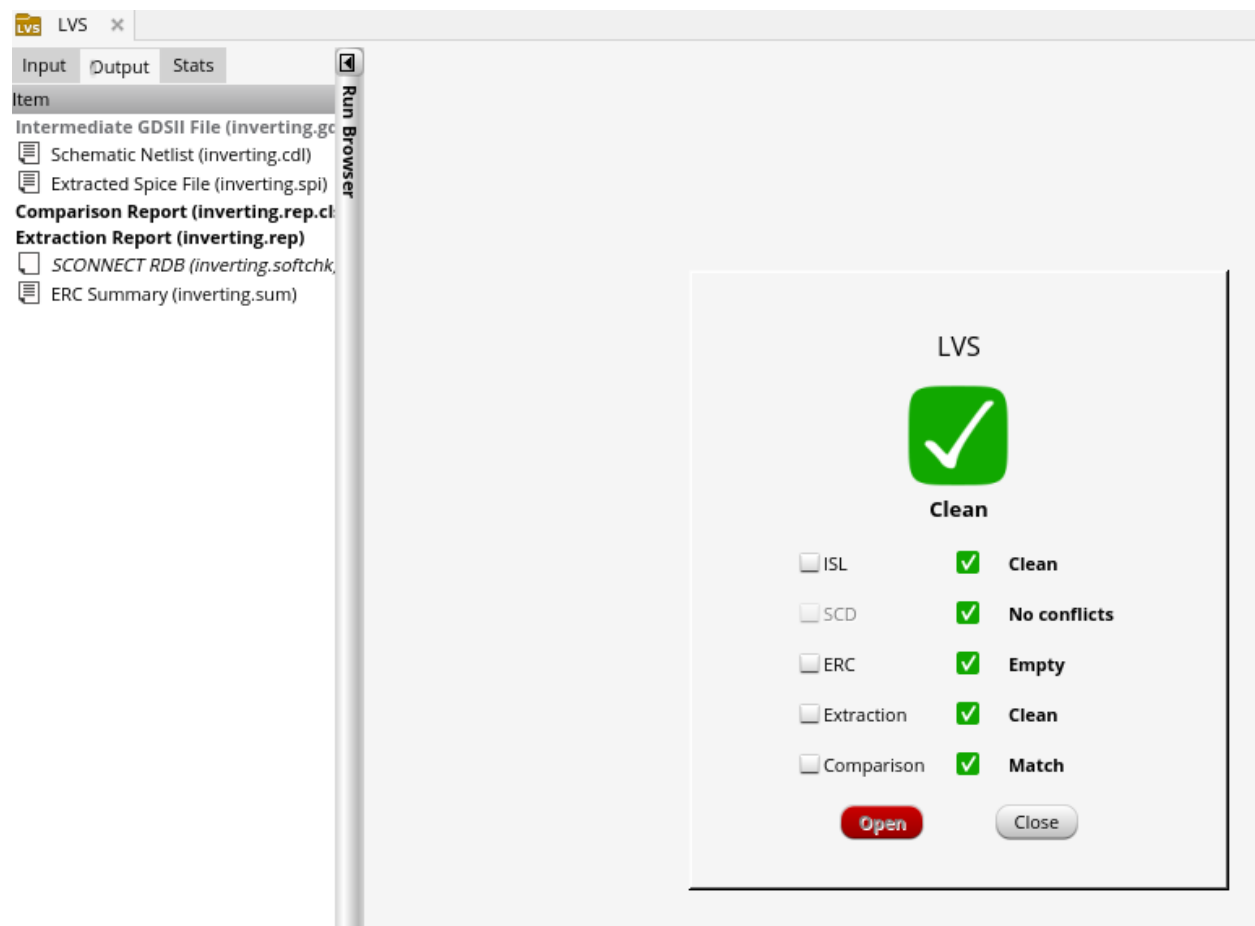
We then moved on to complete the layout



(Figure 62: This is the layout for Inverting Amp)



(Figure 63: This is the DRC result for the layout for Inverting Amp)



(Figure 64: This is the LVS result for the layout for Inverting Amp)

DAC

Once we finished building the Programmable Inverting Amplifier, we moved to the DAC.

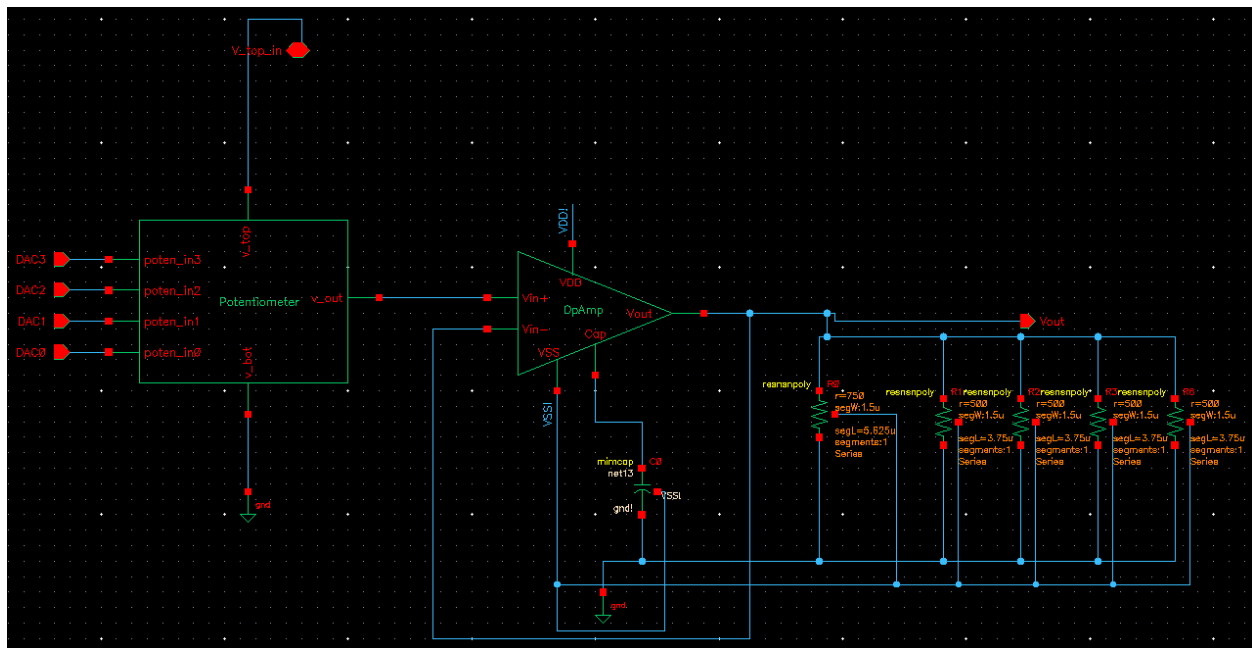
The DAC is basically our programmable potentiometer. Different combination of input signals will allow different taps to be on which give us different voltage (analog value). However, since the step in DAC is usually way smaller (only from 0 to 1, range --> smaller step) and DAC is usually used for other type of circuit (like ADC) we must feed the output through a buffer to make sure the signal is accurate and stable.

Additionally, when opamp is used as a buffer (almost the most extreme use case), we need a big capacitance value to compensate the signal (which is not available). Therefore, we have to apply the same solution as in the non-inverting configuration.

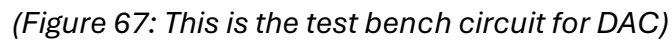
Below is our schematic :



(Figure 65: This is the symbol block for the DAC)



(Figure 66: This is the schematic for DAC)



The oscilloscope displays five waveforms over a 10.0 μs time interval. The red waveform is a high-frequency square wave. The yellow, green, and cyan waveforms are square waves with increasing widths. The blue waveform is a sawtooth wave that ramps down and then resets.

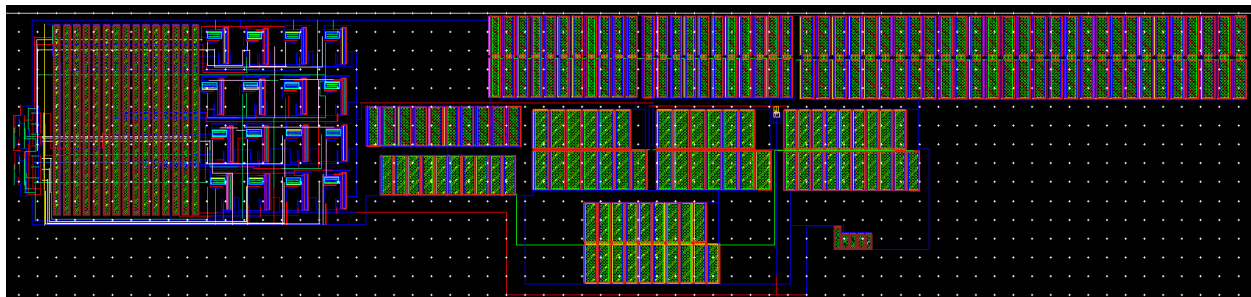


(Figures 68 & 69: This is the result for the test bench circuit for DAC)

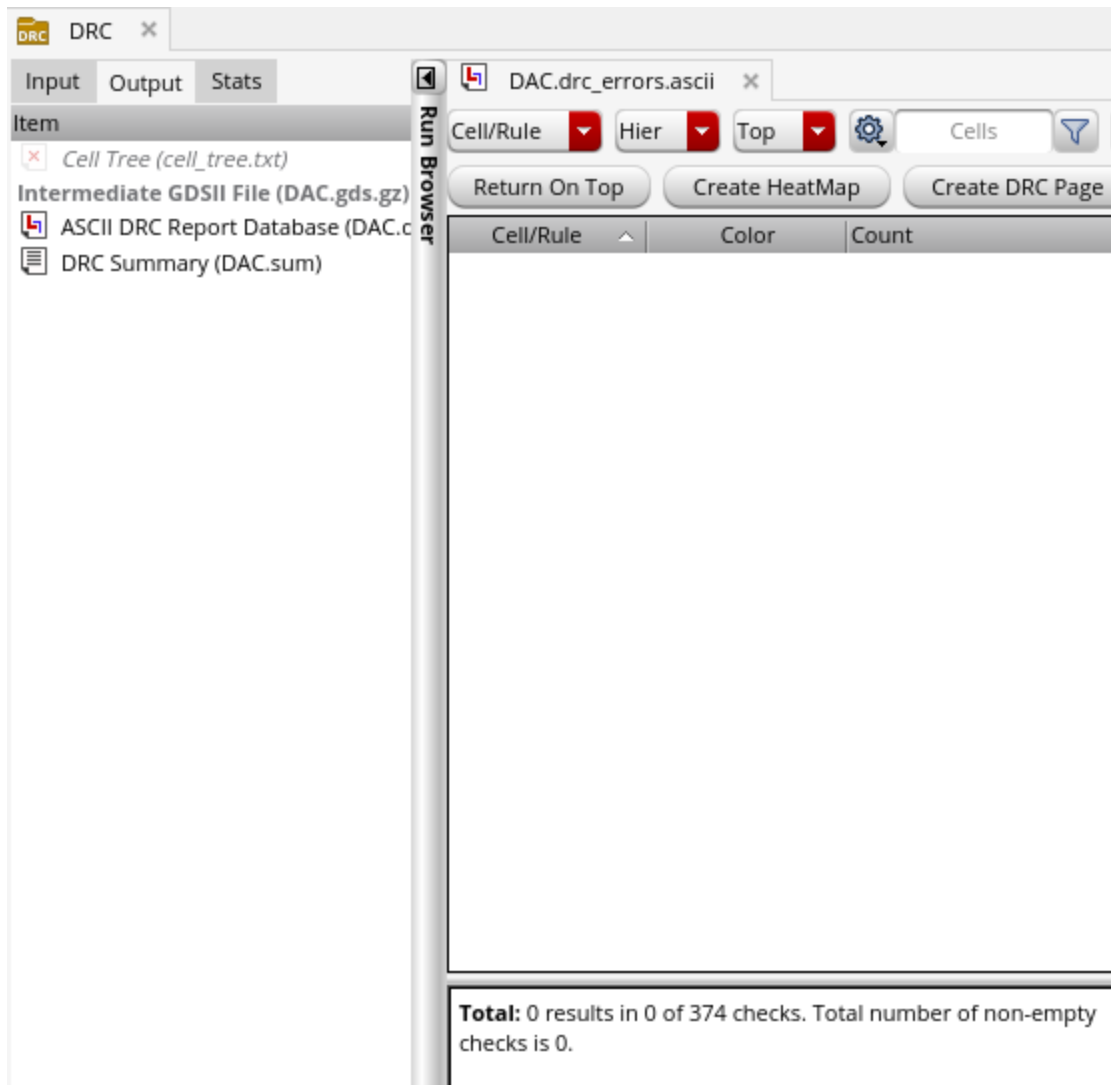
From both two waveform ,we can observe that as the input increases , the voltage is lower (which makes sense because it is nearer to the bot voltage).

The solution turned out to be quite efficient (the signal is stabilized, although not perfect). Additionally, because we split the current from Vout , the function range of the DAC is also limited a bit (about 30 mV). This is not really good in real life manufacturing (because 30 mV can be like 1 or 2 steps in a small-range DAC like this), but we did not really have any other solutions.

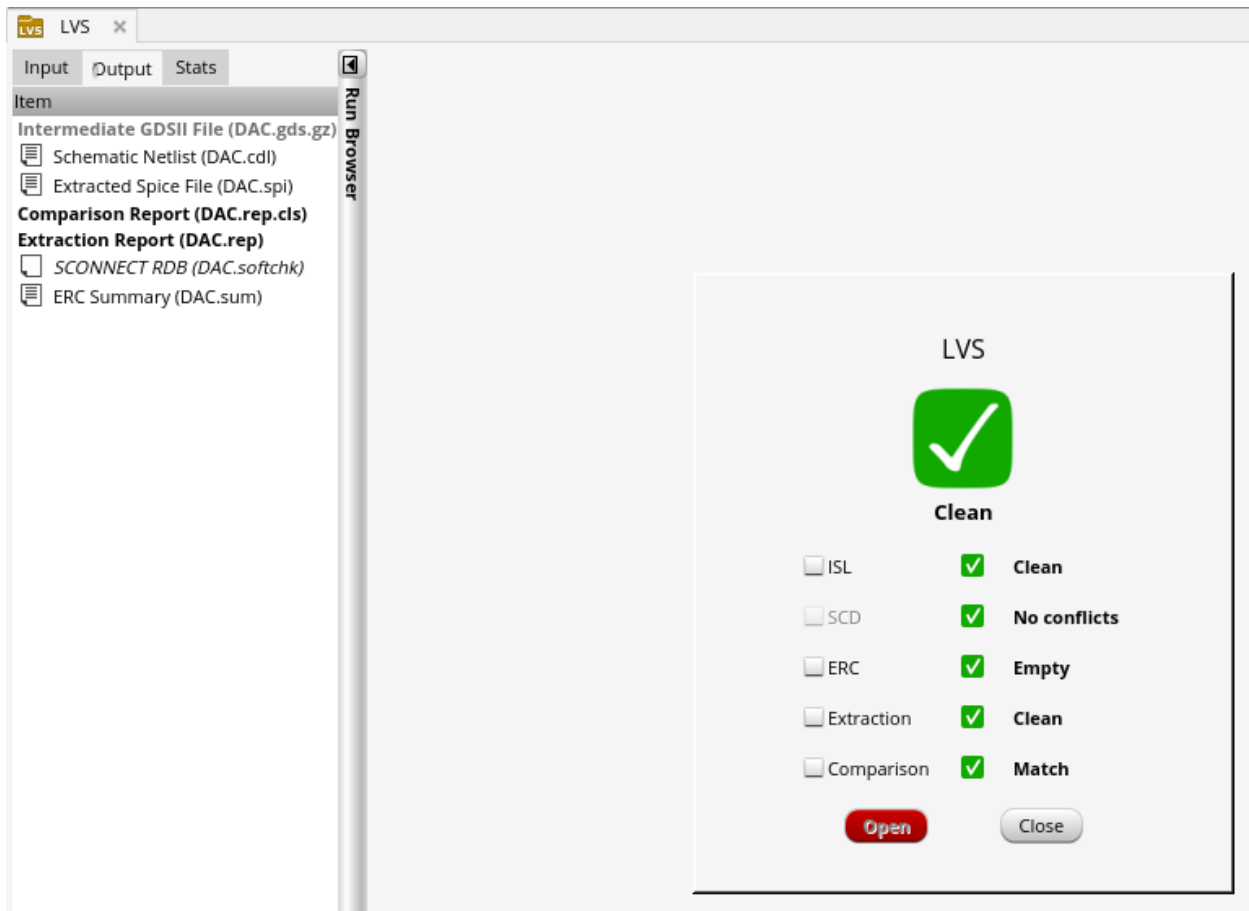
We then moved on to complete the layout for DAC :



(Figure 70: This is the layout for DAC)



(Figure 71: This is the DRC result for the layout for DAC)



(Figure 72: This is the LVS result for the layout for DAC)

Final Project

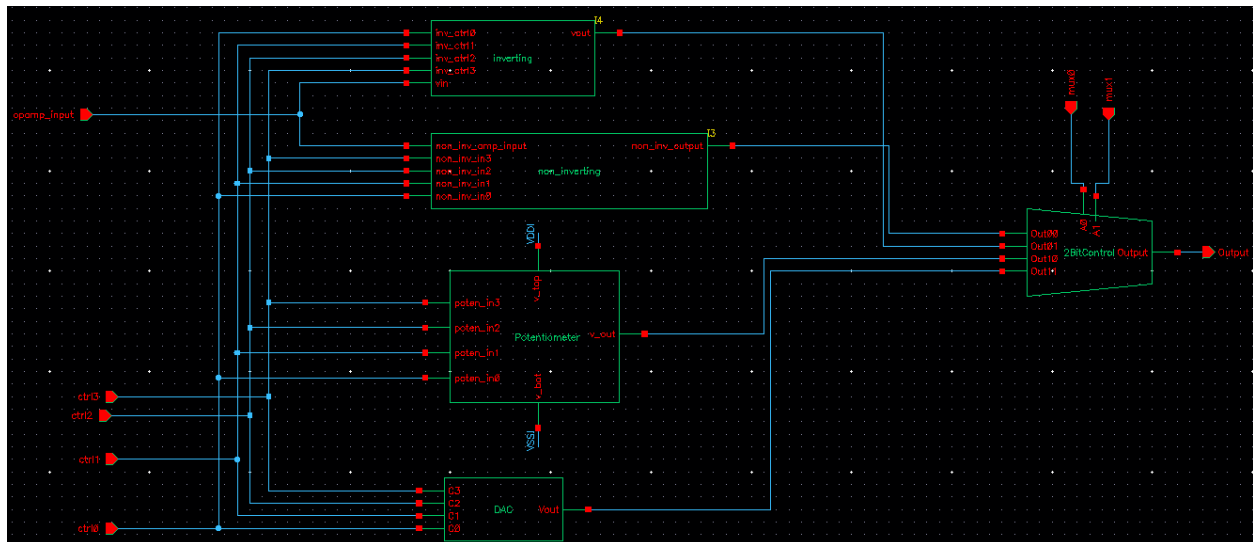
After finishing all the smaller components we just combined them all together in a final schematic and layout .

In addition to the control input and mux input , we also have a opamp_input for case 00 or 01 when the circuit behave as an amplifier.

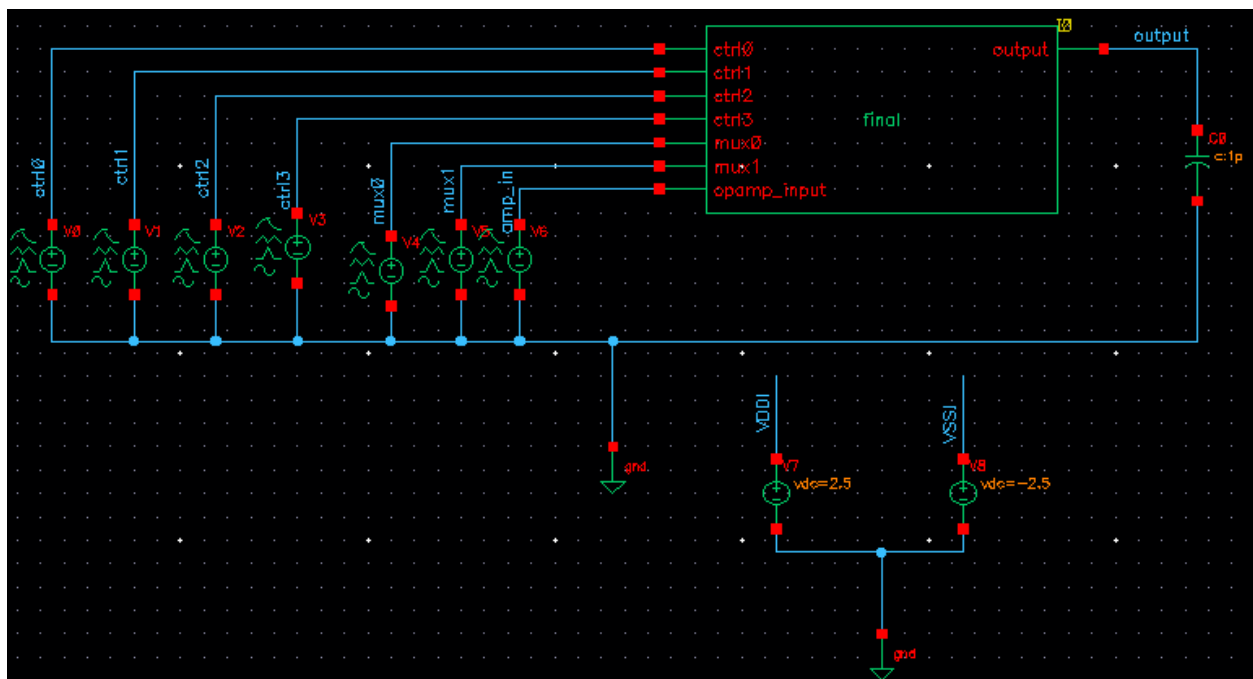
The mux will determine which function will the circuit behaves as mentioned in the introduction:



(Figure 73: This is the symbol block for the Final Project)

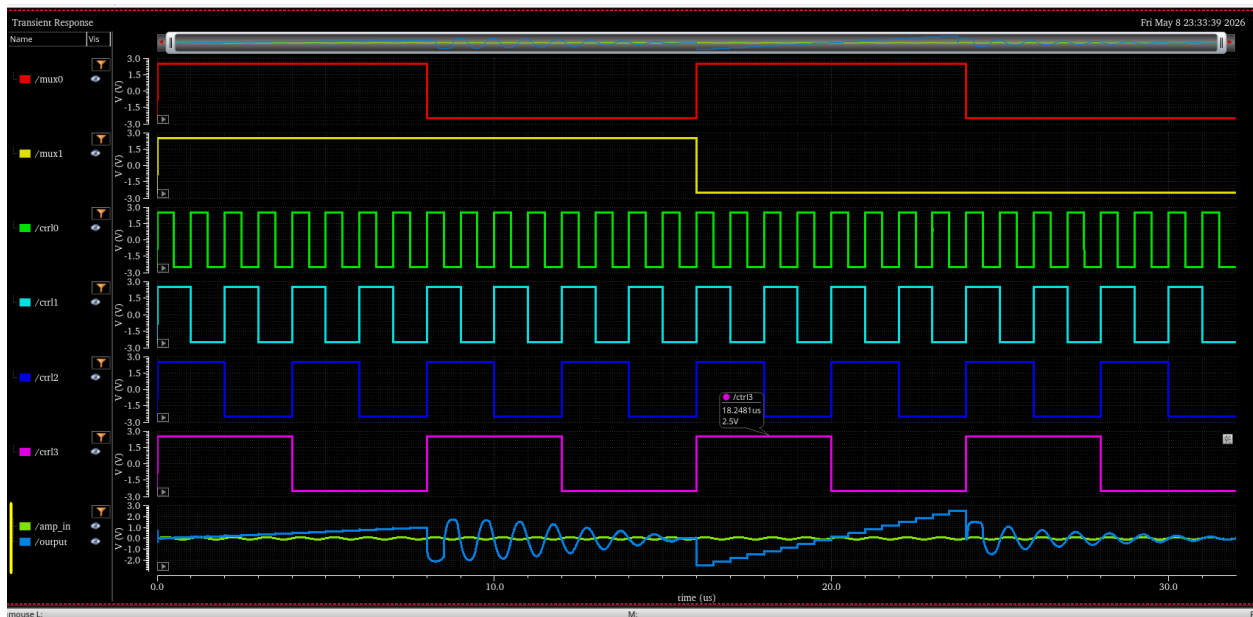
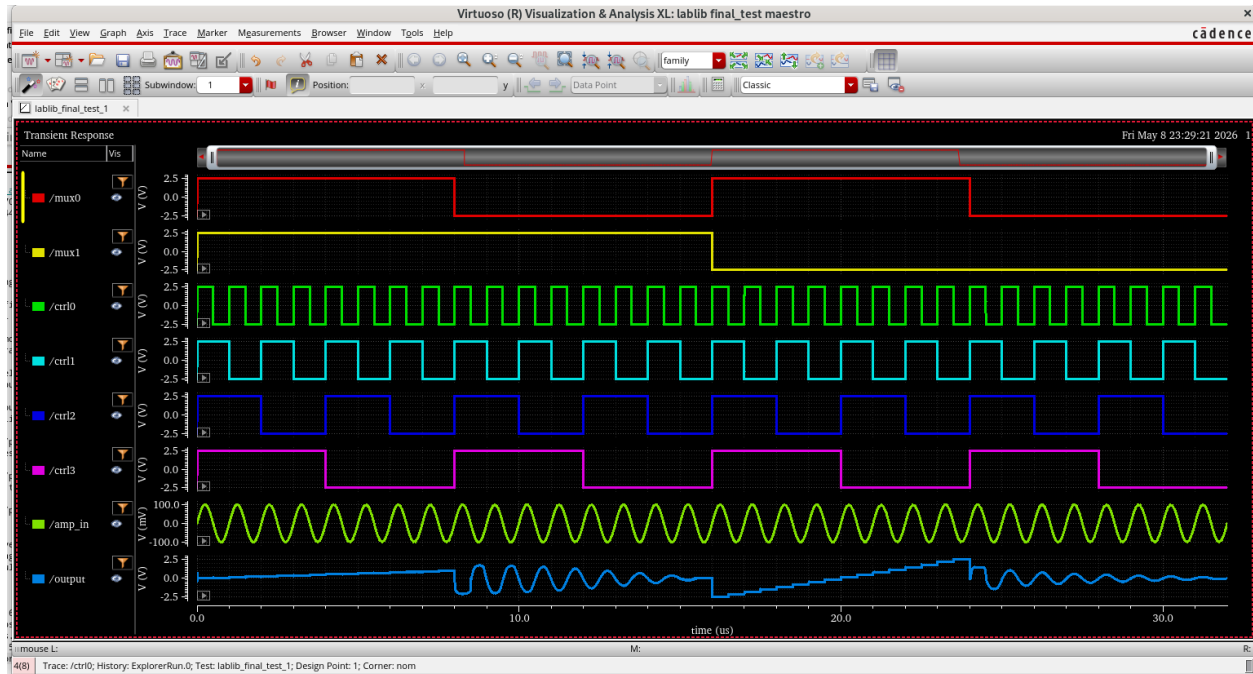


(Figure 74: This is the schematic for Final Project)



(Figure 75: This is the test bench circuit for Final Project)

We also made a tesbench for the circuit . What we expeceted from the output is what I described in the introduction, our circuit should behave differently based on the Mux input :



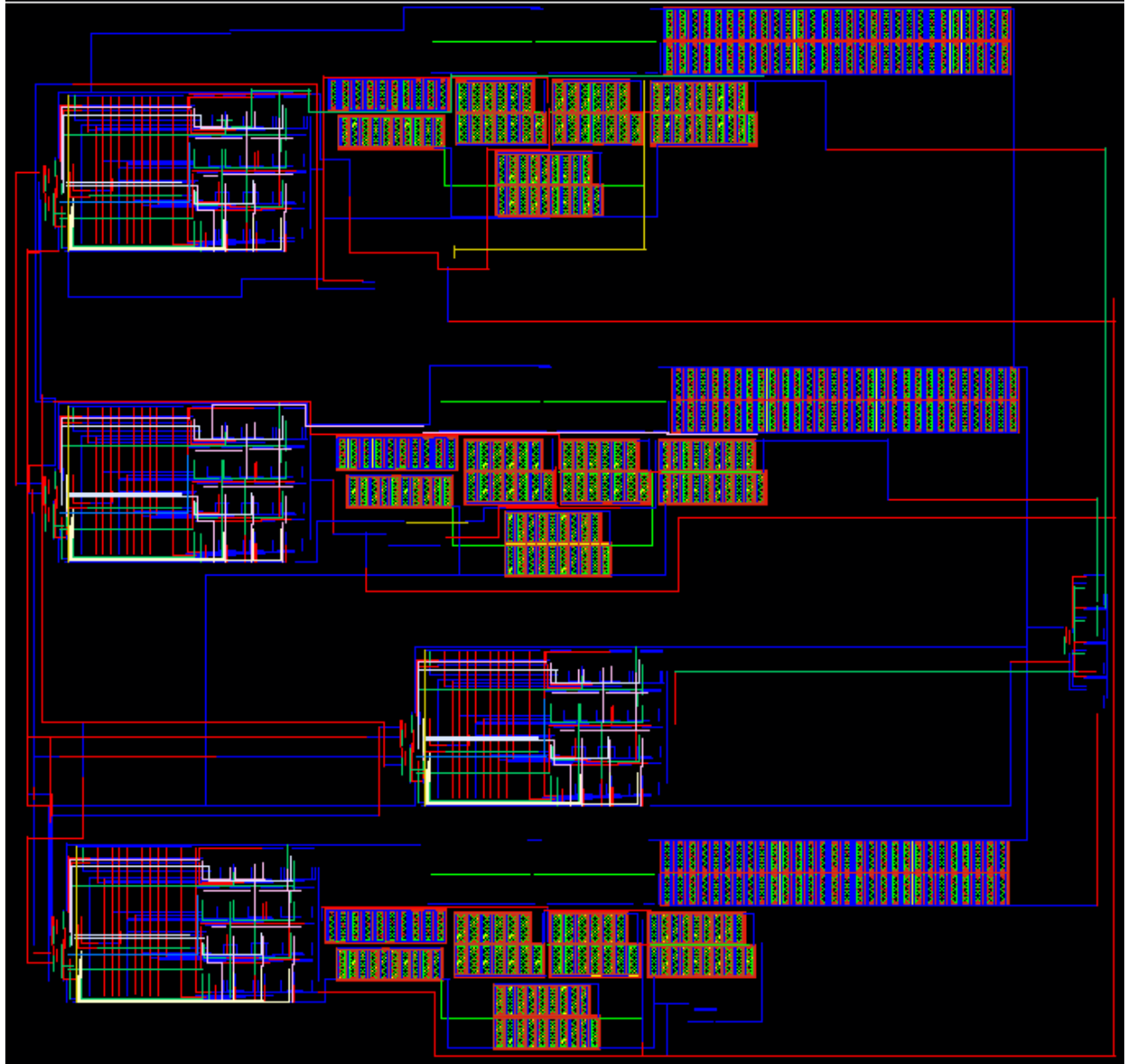
(Figures 76 & 77: This is the result for the test bench for Final Project)

In the wave form when Mux0 and mux 1 is 00 , the circuit behaved as an non-inverting amplifier, while when the input is 01, the circuit behaved as a inverting amplifier (opposite waveform) .

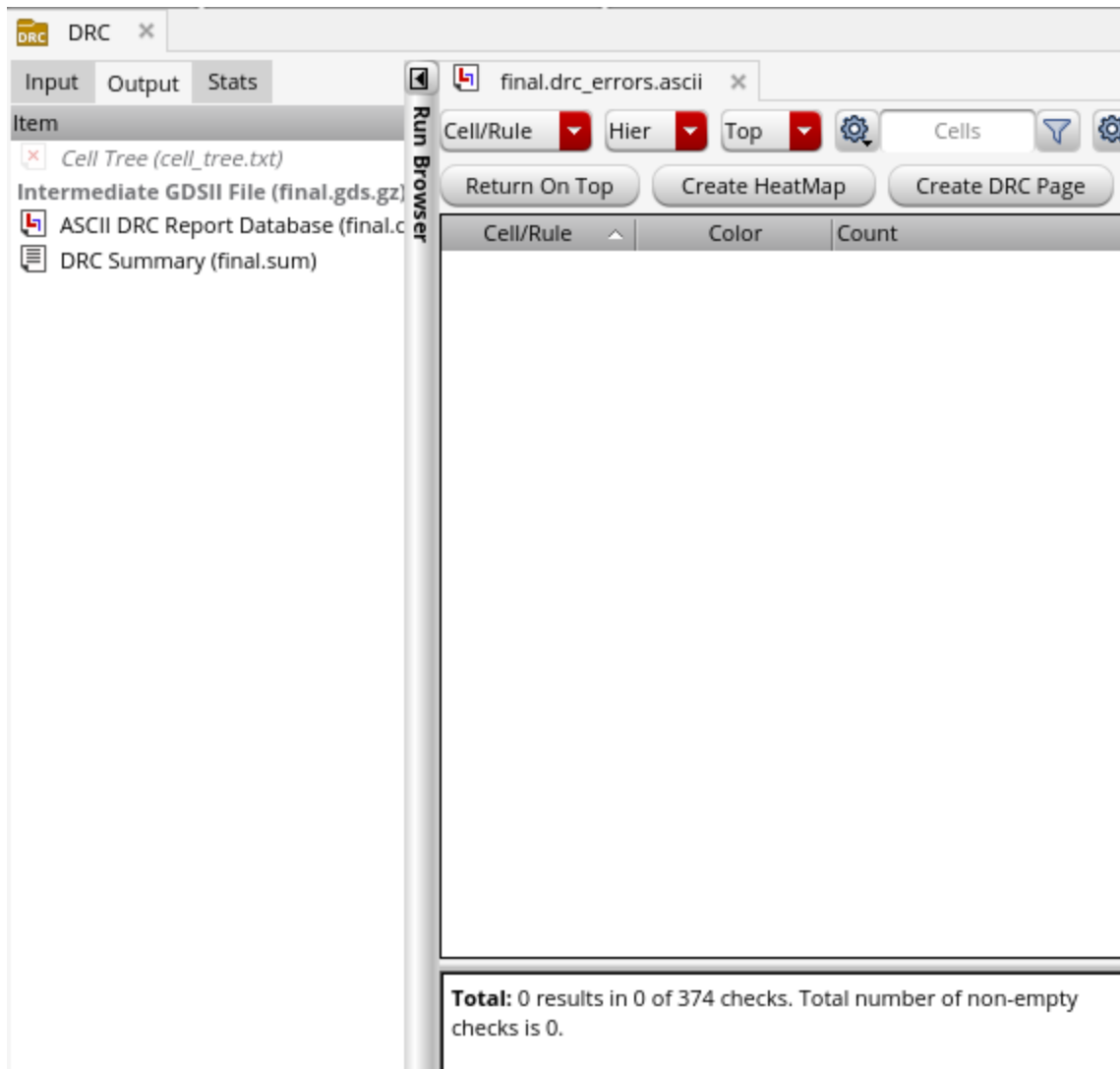
When the input is 10 and 11 (A0 is bit 1 and A1 is bit 0) , the circuit behaves as a potentiometer and a DAC.

We put the second picture for the waveform because we want to show the output and input waveform. It is obvious that both output waveforms for the circuit has bigger amplitude compared to the input. However, the non-inverting waveform has a bit smaller gain although it is usually the opposite case. This is because we use the “resistor-system” to stabilize the circuit, they make the output voltage a bit off. Additionally ,in this case since we set the R_{in} of the inverting configuration 250 , the circuit can have the gain up to $7500/250 = 30$, but for the non-inverting gain, because we set the R_{in} a bit too careful (1Kohm) ,even when input is 1111 (full resistor string), the gain can just be $1 + R_f/R_{in} = 1 + 7.5 = 8.5$ (V/v) which is way more humble compared to the inverting configuration.

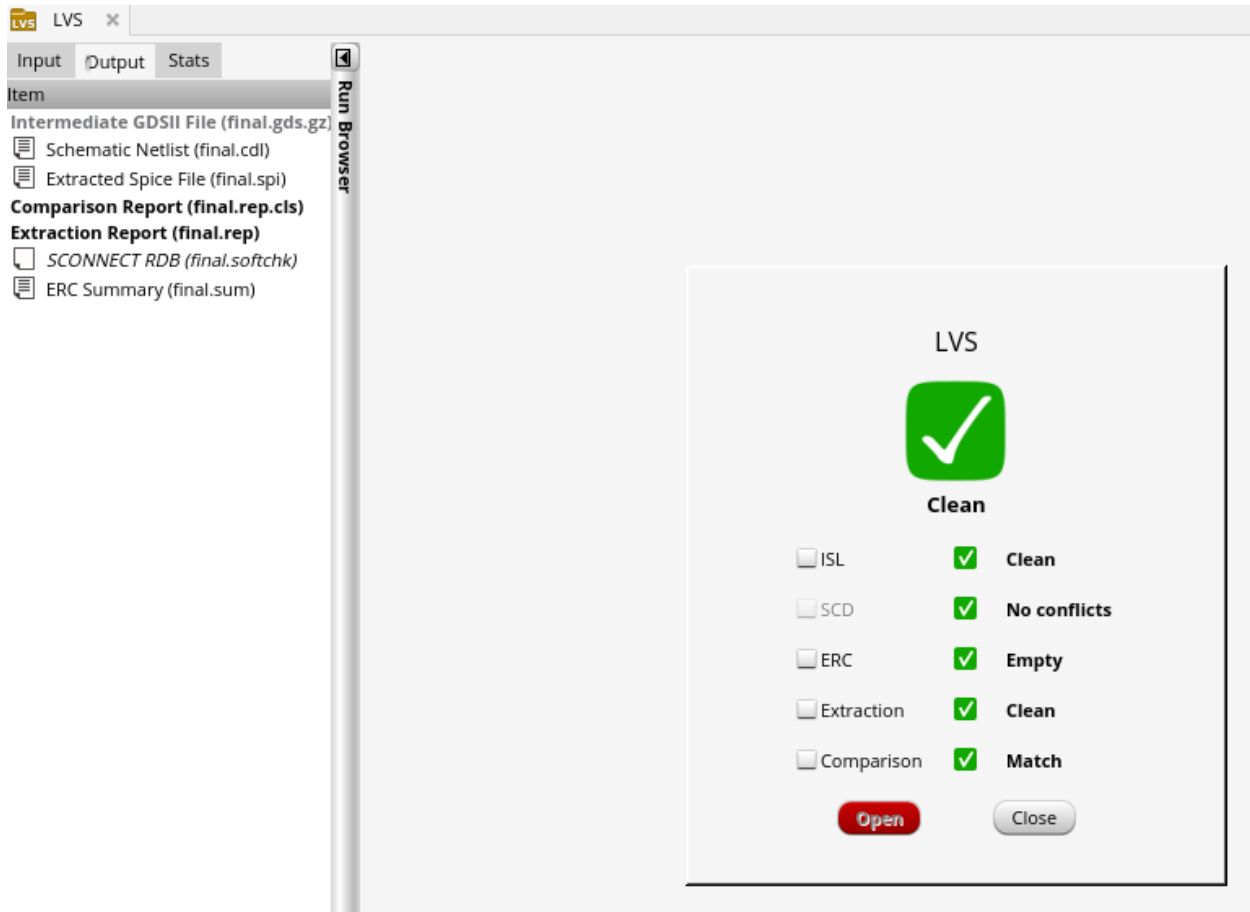
After making sure all the logic is correct , we went on to complete the final layout and get some deep sleep after 3 days of not sleeping (thanks god!)



(Figure 78: This is the layout for Final Project)



(Figure 79: This is the DRC result for the layout for Final Project)



(Figure 80: This is the LVS result for the layout for Final Project)

After that, we next built a test bench to see if the logic of our final schematic worked as intended as the logic stated for our Final Project in the reading from the project list.

III) Mistake and Lesson:

The very first mistake and biggest lesson we have learnt from this project is how to test our opamp decently. We should have tested it with unity case (the most extreme case) as Sam said since it would allow us to know exactly how good our design is and is it possible for our opamp to be functional in other configuraitons.

We just tested with a differential configuration (soft hoopers I gotta say), so when we have to deal with more extreme configurations , we

had a lot of troubles and had to come up with a solution that although is quite efficient , it does impact our circuit's overall performance.

Additionally, we have also learnt a lesson of testing the circuit and tried to interpret the waveform more carefully. At the beginning , we did not realize that the decoder is making in a different order. As a consequence , we wasted a lot of time on tryign to get the ladder waveform result (which we could have got way easier since our circuit worked properly).

Last but not least, we have also learnt a huge lesson about the importance of choosing the process. At first we thought we may need the verilog conversion to schematic , so we just deiced to use gpdk045 right away without thinking about the weakness of this process. Consequently, we did not have a proper capacitor (and resisor behavior is weird too) . Additionally, pegasus is not really a good LVS and DRC running software .

IV) Conclusion:

Through this project ,we have learnt a tons of new things from building new circuit that we have never actually thought of such as potentiometer, programmable amplifiers, or even a Digital to analog converter. We also gain a lot of experience in teamworking and debugging . Last but not least , 24 hours of just sleeping and working with cadence also gave us a lot of hands-on experience with this design software.

After all, we wanted to thank Sam , Yash, Professor Geiger, and other peers too for helping us throughout the semester and completing this project.